



ARDUINO INICIANTE

PROJETOS PRÁTICOS PARA COMEÇAR



Conselho Editorial

Luan Silva Santana

Wellington Assunção Azevedo

Direção de Arte

Edvan da Silva Oliveira

Diagramação e Capa

Carol Correia Viana

Produção e Revisão

Carol Correia Viana

Kleber Rocha Bastos

Luan Silva Santana

Wellington Assunção Azevedo

Colaboradores

Carlos Dyorgenes Santana

Clismann Silva Santana

Contato

contato@casadarobotica.com

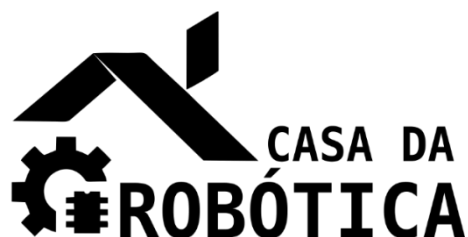


[@casadaroboticaoficial](https://www.facebook.com/casadaroboticaoficial)



[@casadarobotica](https://www.instagram.com/casadarobotica)

casadarobotica.com



Copyright © 2020 da WL Componentes Eletrônicos Ltda.

Todos os direitos reservados e protegidos pela Lei 9.610 de 19/02/1998. É proibida a reprodução desta obra, mesmo parcial, por qualquer processo, sem prévia autorização, por escrito dos autores.

Editores: Luan Silva Santana e Welligton Assunção Azevedo

Direção de arte: Edvan da Silva Oliveira

Diagramação e capa: Carol Correia Viana

Produção e revisão: Carol Correia Viana, Kleber Rocha Bastos, Luan Silva Santana e Welligton Assunção Azevedo

Colaboração: Carlos Dyorgenes Santana e Clismann Silva Santana

Primeira Edição

WL Componentes Eletrônicos

CNPJ: 29.495.665/0001-03

Avenida Brumado 1400

46052-000 - Vitória da Conquista, BA - Brasil

Tel.: +55 77 99151-2820

E-mail: contato@casadarobotica.com

Site: www.casadarobotica.com

APRESENTAÇÃO

Olá,

Obrigada por adquirir o Kit Iniciante.

Este material constitui um guia de apoio para seus estudos de eletrônica e programação. Nele, preparamos uma série de textos e exemplos práticos que consideramos importantes para os seus primeiros passos com o Kit.

Inicialmente, apresentamos a plataforma Arduino e a placa microcontroladora UNO SMD R3 Atmega328, compatível com ao projeto Arduino UNO, ressaltando suas principais características e forma de programação através do software Arduino IDE.

Posteriormente, foram expostas as principais características e especificações dos demais componentes eletrônicos que foram adquiridos em conjunto com o Kit.

Em seguida, fornecemos a você algumas noções da linguagem de programação para microcontroladores do projeto Arduino, desde a declaração de variáveis até o uso de bibliotecas.

Logo após, exibimos uma série de exemplos práticos para utilização dos componentes disponíveis no Kit, apresentando o esquemático do circuito e a programação necessária. Como forma de facilitar o entendimento das ligações elétricas do circuito e modo de programação disponibilizamos alguns destes exemplos práticos no Tinkercad, ferramenta online gratuita que permite a simulação de circuitos elétricos e programação.

Posteriormente, para que você aprenda se divertindo propomos a construção de um jogo da memória similar ao game Genius, jogo eletrônico de habilidade de memória que foi sucesso na década de 80.

E mais, disponibilizamos um exemplo prático BÔNUS utilizando a plataforma Blynk para criar um aplicativo de controle e monitoramento para seus projetos de hardware a partir de dispositivos móveis Android e iOS.

Esperamos que você aproveite esse material com entusiasmo e ele auxilie a sua jornada de estudos.

Um grande abraço,

Equipe Casa da Robótica.

GRUPO DE SUPPORTE NO TELEGRAM

ACESSE AGORA MESMO



***CLIQUE
AQUI!***



SUMÁRIO

1. INTRODUÇÃO	8
2. CONHECENDO A PLATAFORMA ARDUINO	10
O QUE É O ARDUINO?	10
EXPLORANDO UMA PLACA UNO SMD R3 ATMEGA328	12
PRIMEIROS PASSOS.....	15
EXPLORANDO O ARDUINO IDE.....	21
3. CONHECENDO OS DEMAIS COMPONENTES DO KIT INICIANTE.....	27
PROTOBOARD	27
JUMPER	28
PIN HEADER	29
CABO PARA BATERIA 9 V COM PLUG P4.....	29
RESISTOR	30
POTENCIÔMETRO	31
SENSOR DE LUZ - LDR.....	31
SENSOR DE TEMPERATURA – TERMISTOR NTC.....	32
LED.....	33
LED RGB	34
DISPLAY DE LED DE 7 SEGMENTOS.....	35
BOTÃO PUSH BUTTON.....	36
BUZZER.....	37
SENSOR INFRAVERMELHO TCRT5000.....	38
REED SWITCH.....	39
4. FUNDAMENTOS DE PROGRAMAÇÃO	41
OPERADORES E ESTRUTURA DE CONTROLE DE FLUXO	48
COMO PROGRAMAR A PLACA UNO	56
PROJETO BLINK – PISCA LED INTERNO DA PLACA UNO	56
PROJETO BLINK – PISCA LED EXTERNO.....	59
PROJETO LIGAR E DESLIGAR LED COM BOTÃO PUSH BUTTON.....	63
PROJETO INTERRUPTOR COM BOTÃO PUSH BUTTON	66
PROJETO SENSOR DE LUMINOSIDADE – APRENDENDO USAR O LDR.....	69
LIGAR E DESLIGAR LED UTILIZANDO SENSOR LDR	72
PROJETO TOCAR BUZZER 5 VEZES	77

PROJETO MÚSICA DÓ RÉ MÍ FÁ NO BUZZER.....	80
PROJETO PISCAR O LED RGB – VERMELHO, VERDE E AZUL	83
PROJETO PISCAR O LED RGB – COMBINAÇÃO DE CORES.....	87
PROJETO PISCAR O LED RGB – TODAS AS CORES	89
PROJETO PISCAR O LED RGB – TODAS AS CORES USANDO FUNÇÕES	91
PROJETO ESCOLHER A COR DO LED RGB PELO MONITOR SERIAL.....	94
PROJETO PISCAR LED COM INTERVALO DEFINIDO PELO POTENCIÔMETRO.....	98
PROJETO FADE LED COM POTENCIÔMETRO.....	101
PROJETO CONTADOR DE 0 A 9 COM DISPLAY DE 7 SEGMENTOS CÁTODO COMUM	104
PROJETO CONTADOR DE 0 A 9 COM DISPLAY DE 7 SEGMENTOS ÂNODO COMUM	113
PROJETO INCREMENTO E DECREMENTO – 0 A 9 COM DISPLAY DE 7 SEGMENTOS CÁTODO COMUM.....	120
PROJETO INCREMENTO E DECREMENTO – 0 A 9 COM DISPLAY DE 7 SEGMENTOS ÂNODO COMUM.....	127
PROJETO ILUMINAÇÃO SEQUENCIAL COM LEDS	134
PROJETO MEDIR TEMPERATURA DO AMBIENTE COM TERMISTOR NTC.....	137
PROJETO DETECTAR LINHA COM SENSOR INFRAVERMELHO.....	140
PROJETO DETECTAR CAMPO MAGNÉTICO COM REED SWITCH.....	144
5. JOGO DA MEMÓRIA.....	147
JOGO DA MEMÓRIA COM DISPLAY CÁTODO COMUM.....	147
JOGO DA MEMÓRIA COM DISPLAY ÂNODO COMUM.....	160
6. BÔNUS: PLATAFORMA BLYNK.....	173
PRIMEIROS PASSOS.....	174
CRIANDO O PRIMEIRO PROJETO NO BLYNK	179
7. CONSIDERAÇÕES FINAIS.....	191

1.INTRODUÇÃO

Neste material de apoio você encontrará todo o suporte necessário para iniciar seus estudos em eletrônica e programação. Para isto, inicialmente, você precisa conhecer um pouco sobre a plataforma Arduino e o microcontrolador UNO R3 Atmega328, compatível ao projeto Arduino UNO, e os demais componentes eletrônicos disponíveis no Kit, conforme a Tabela 1.

Tabela 1 - Componentes do Kit Iniciante.

COMPONENTES	QUANTIDADE
Placa UNO SMD R3 Atmega328 compatível com o projeto Arduino UNO	01
Cabo USB (tipo A para tipo B)	01
Cabo para bateria 9 V com plug V4	01
Cabos jumpers macho-macho	20
Pin header macho	01
Protoboard 400 furos M	01
LED difuso	15
LED RGB	02
Resistores de 220 Ω	20
Resistores de 1 kΩ	10
Resistores de 10 kΩ	10
Sensor de luz - LDR	02
Botões push button com capas	06
Sensor de temperatura – Termistor NTC	02
Potenciômetro 10 kΩ	01
Sensor infravermelho TCRT 5000	02
Buzzer ativo	01
Reed switch	03
Display de LED de 7 segmentos	01

Desta forma, os dois capítulos seguintes expõem a estrutura, características, pinagens e especificações técnicas destes componentes.

O Capítulo 4 foi preparado para auxiliar você a aprender os principais fundamentos da linguagem de programação utilizada pelo Arduino, como declaração e tipos de variáveis, estrutura de condição e repetição, funções e biblioteca. Este capítulo também expõe uma série de exemplos práticos para aplicação dos conhecimentos de programação e utilização dos componentes eletrônicos do Kit, sendo detalhado passo a passo a construção do esquemático elétrico e a

programação. Para facilitar o entendimento das ligações elétricas do circuito e permitir a simulação da programação disponibilizamos alguns destes exemplos práticos no Tinkercad.

O Capítulo 5 deste material de apoio tem como objetivo reunir os conhecimentos adquiridos e nos divertir com a criação de um jogo da memória similar ao game Genius (também conhecido como Simon Game), jogo eletrônico de habilidade de memória que foi sucesso da década de 80.

Por fim, no Capítulo BÔNUS apresentamos a plataforma Blynk e disponibilizamos um exemplo prático para criação de um aplicativo para smartphones Android e iOS para você controlar e monitorar seus projetos de hardware.

2.CONHECENDO A PLATAFORMA ARDUINO

O QUE É O ARDUINO?

O Arduino é uma plataforma eletrônica de código aberto baseada em software e hardware de fácil utilização, sendo ideal para iniciantes e para qualquer pessoa que deseja construir projetos eletrônicos.

As placas Arduino permitem a conexão de circuitos eletrônicos aos seus terminais, o que possibilita a leitura de entradas – luz em um sensor, o acionamento de um botão ou uma mensagem SMS, e transformar estas informações em uma saída controlando algum dispositivo – por exemplo ligando um LED, ativando um motor ou enviando uma mensagem.

As placas Arduino podem ser conectadas ao computador por meio do barramento serial universal (USB), possibilitando sua utilização como placa de interface e controlar dispositivos por meio do seu computador.

A plataforma Arduino oferece uma série de vantagens em relação a outras plataformas, o que o tornou popular entre professores, alunos, amadores e projetistas, tais como:

- Possuir ambiente multiplataforma, ou seja, pode ser executado nos principais sistemas operacionais comercializáveis;
- Contar uma IDE de programação própria;
- Poder ser programado utilizando um cabo USB;
- Possuir hardware e software de fonte aberta;
- Ter sido desenvolvido em um ambiente educacional, sendo ideal para iniciantes.

Diante da sua popularização, a plataforma Arduino cresceu e atualmente conta com diversas versões de mercado. A Figura 1 ilustra algumas versões da placa Arduino.

Figura 1 - Algumas versões da placa Arduino



Além disso, existem uma série de placas compatíveis com o projeto Arduino, uma vez que seu hardware é aberto a replica destas placas são permitidas e possuem as mesmas características, pinagens e forma de uso. A Placa UNO SMD R3 Atmega328 disponível neste Kit, por exemplo, é compatível ao projeto do Arduino UNO.

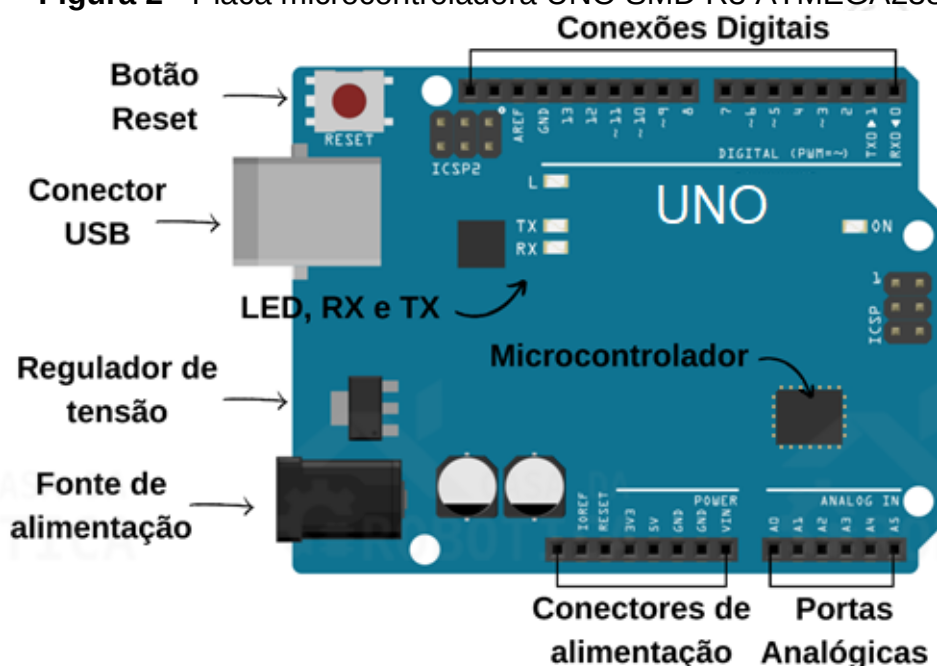
Existem placas Arduino bem pequenas (Nano, micro, mini), de tamanho médio e tradicional (Uno, Duemilanove, Leonardo), e as placas de maiores dimensões (Mega, Due). Diante de tanta variedade, você deve estar se perguntando: Qual placa devo usar no meu projeto? A escolha da versão ideal vai depender das necessidades de seu projeto, mas recomendamos:

- Placas de Arduino pequenas para projetos que precisam ser leves e ocupar pouco espaço;
- Placas de Arduino tamanho médio e tradicional para projetos de tamanho padrão como robôs, interfaces homem-máquina, central de monitoramento, entre outros;
- Placas de Arduino maiores dimensões para projetos que demandem de maior memória e número de portas de entrada e saída.

EXPLORANDO UMA PLACA UNO SMD R3 ATMEGA328

Conhecer os elementos que compõe a placa UNO SMD R3 ATMEGA328 é de suma importância antes de iniciar os nossos projetos. Desta forma, vamos explorar esta placa microcontroladora (Figura 2) para nos familiarizar com seus vários componentes.

Figura 2 - Placa microcontroladora UNO SMD R3 ATMEGA238.



Fonte de alimentação

O circuito interno da placa UNO deve ser alimentado com uma tensão contínua de 5V. Você pode alimentá-lo conectando-o a uma porta USB do computador, que fornecerá a alimentação e também a comunicação de dados, ou por meio de uma fonte de alimentação externa, que forneça uma saída contínua entre 7 V e 12 V, por meio da utilização de um plug P4 ou o pino Vin.

Regulador de tensão

Na placa UNO, o regulador de tensão tem como finalidade transformar qualquer tensão (entre 7 V e 12 V) que esteja sendo fornecida pelo conector de alimentação externa em uma tensão contínua de 5V.

Conectores de alimentação elétrica

Os conectores de alimentação elétrica fornecem energia para dispositivos externos e são constituídos pelos pinos:

- Reset que possui a mesma função do botão Reset;
- 3,3 V e 5 V que fornecem tensão de 3,3 e 5 V, respectivamente;
- GND fornece potencial de terra aos dispositivos externos;
- Vin fornece ao dispositivo externo a mesma tensão que está sendo recebida pelo pino de alimentação externa.

Entradas analógicas

A placa UNO possui 6 portas analógicas que estão indicados como Analog In, de A0 a A5. Esses pinos são dedicados a receber valores de grandezas analógicas, por exemplo, a tensão de um sensor. As grandezas analógicas variam continuamente no tempo dentro de uma faixa de valores.

Conexões digitais

A placa UNO possui 14 portas digitais que estão indicados como Digital, de 0 a 13. Estas portas podem ser utilizadas como receber ou enviar dados de grandezas digitais. Ao contrário das grandezas analógicas, as grandezas digitais não variam continuamente no tempo, mas sim em saltos entre valores definidos (0 ou 1, ligado ou desligado, sim ou não, 0 V ou 5 V).

Estes pinos digitais operam em 5V e corrente máxima de 40 mA. Além disso, alguns deles possuem funções especiais, como:

- Pinos 3, 5, 6, 9, 10 e 11 podem ser usados como saídas PWM, simulando uma porta analógica;
- Pinos 0 e 1 (RX e TX) podem ser utilizados para comunicação serial;
- Pinos 2 e 3 podem ser configurados para gerar uma interrupção externa.

Microcontrolador

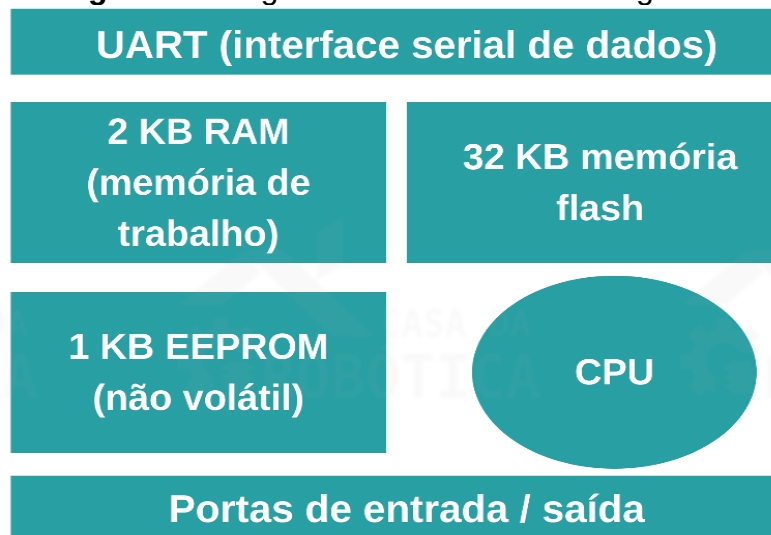
O microcontrolador utilizado na placa UNO é o ATmega328, um pequeno chip de 28 pinos que se encontra no centro da placa e é considerado o cérebro deste

dispositivo. Esse único chip é um pequeno computador, contendo memória, processador e toda eletrônica necessária aos pinos de entrada e saída.

É no microcontrolador que tudo acontece, é nele que fica gravado o código desenvolvido para execução. O microcontrolador permite que a placa Uno funcione de forma autônoma, em outras palavras, uma vez transferido o código não existe mais a necessidade de comunicação com o computador. Um fato que deve ser lembrado é que ao gravar um código, o anterior é descartado, ficando apenas o último código gravado.

Algumas características do microcontrolador ATmega328 encontra-se detalhado na Figura 3 abaixo.

Figura 3 - Diagrama de blocos do ATmega328.



Fonte: Adaptado de Monk (2017).

Botão Reset

O botão Reset tem como única função reinicializar a placa microcontroladora.

Outros componentes

Além dos componentes citados, a placa UNO também conta com um oscilador a cristal, capaz de realizar 16 milhões de ciclos ou oscilações por segundo, conector serial de programação, outro meio de programar a placa UNO, e um chip de interface USB, que converte os níveis de sinal usados pelo padrão USB em níveis que podem ser usados pela placa UNO.

PRIMEIROS PASSOS

Para que a placa UNO execute qualquer ação você precisará escrever um código ou Sketch em linguagem C/C++ utilizando gratuitamente o software Arduino IDE, que se encontra disponível na versão online e offline, e depois fazer o upload deles para a placa.

O Arduino Web Editor é a interface de desenvolvimento online do Arduino, com ele é possível codificar, salvar os esboços na nuvem, fazer backup e enviar o código feito para qualquer placa compatível com o Arduino a partir do navegador de internet. Por estar hospedado online, o Arduino Web Editor estará sempre atualizado com os recursos, bibliotecas e suporte mais recente. Além disto, esta interface de desenvolvimento permite que você acesse um código salvo a partir de qualquer dispositivo conectado à internet. O Arduino Web Editor encontra-se disponível no link <https://create.arduino.cc/editor>, em que será necessário a realização de um cadastro de acesso.

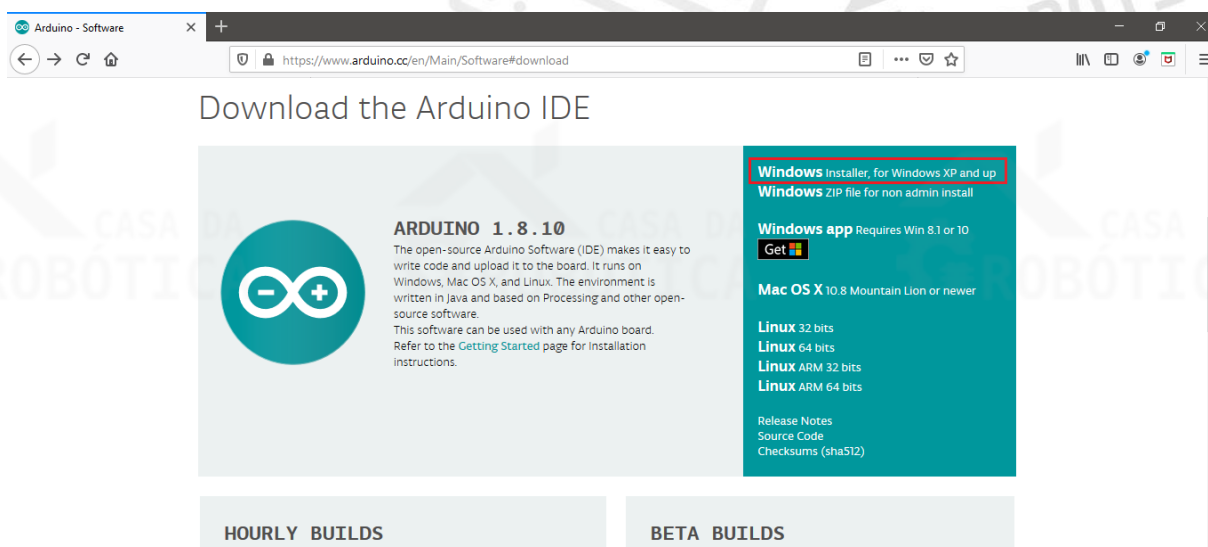
O Arduino IDE é a versão offline desta ferramenta de desenvolvimento e pode ser executado no Windows, Mac OS X e Linux. O download do Arduino IDE encontra-se disponível no link <https://www.arduino.cc/en/Main/Software#download>, em que faz-se necessária a escolha da versão apropriada para seu sistema operacional.

Download e Instalação do Arduino IDE no Windows

A seguir, você encontrará o passo a passo para instalar o Arduino IDE no seu computador Windows.

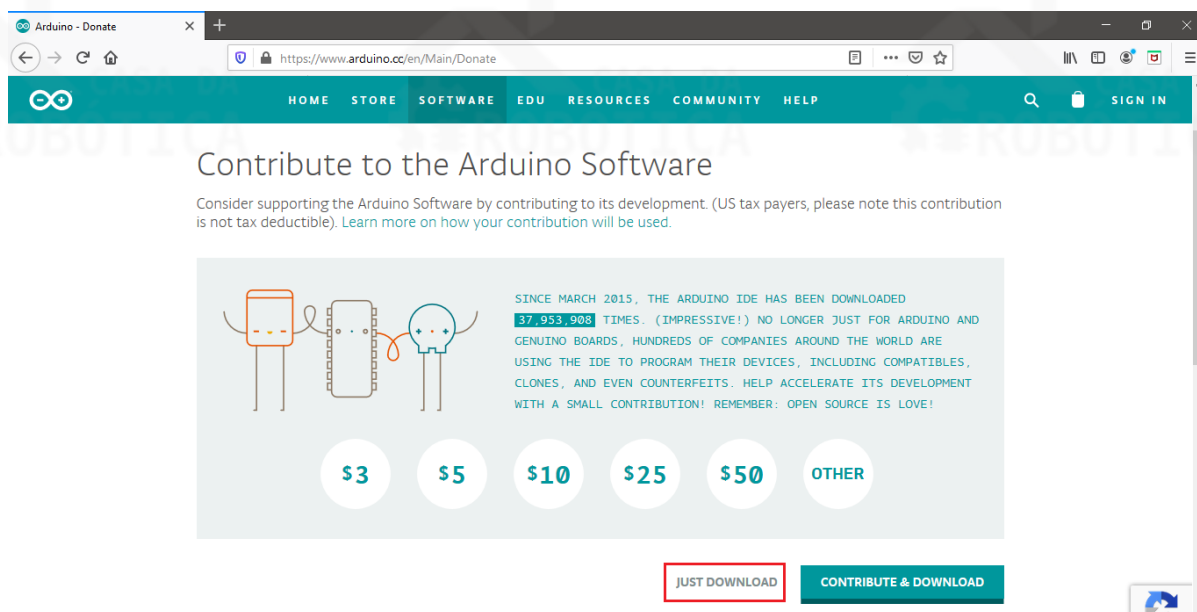
1 - Acesse o link <https://www.arduino.cc/en/Main/Software#download> e escolha a opção “Windows Installer, for Windows XP and up”, conforme ilustra a Figura 4.

Figura 4 – Passo 1 para instalação do Arduino IDE.



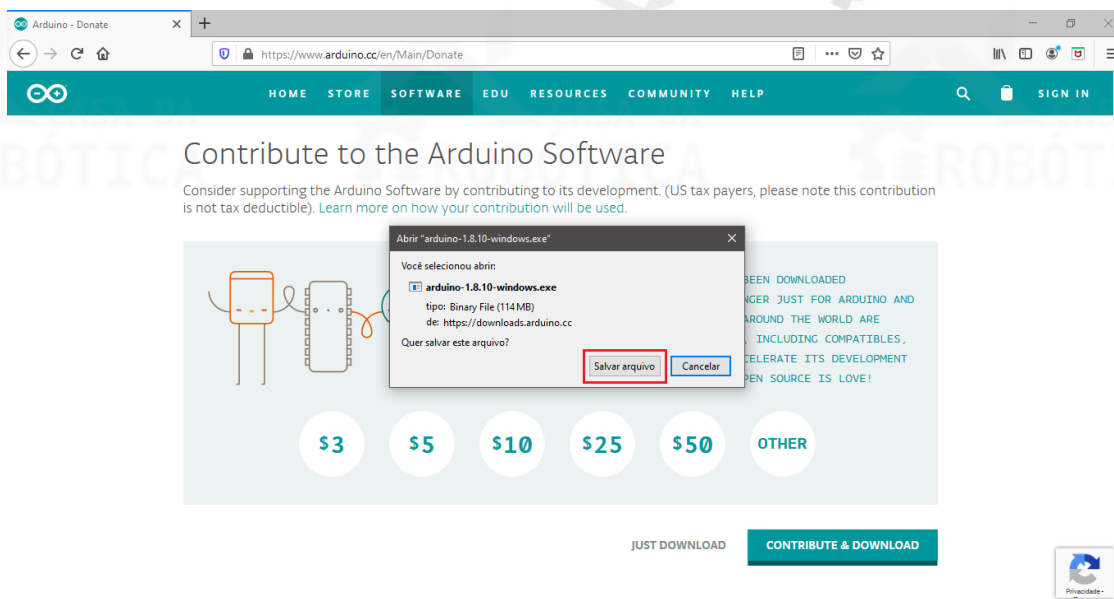
2 – Em seguida, para baixar gratuitamente o software Arduino IDE selecione a opção “Just download”, conforme Figura 5.

Figura 5 - Passo 2 para instalação do Arduino IDE.



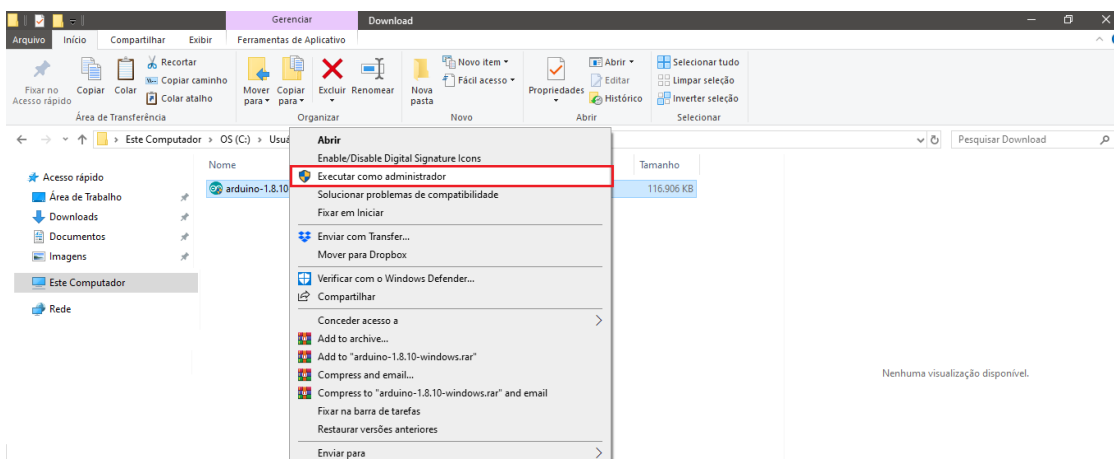
3 – Salve o arquivo do download e aguarde.

Figura 6 - Passo 3 para instalação do Arduino IDE.



4 – Após conclusão do download, clique com o botão direito sobre o arquivo baixado e o execute como administrador, conforme Figura 7.

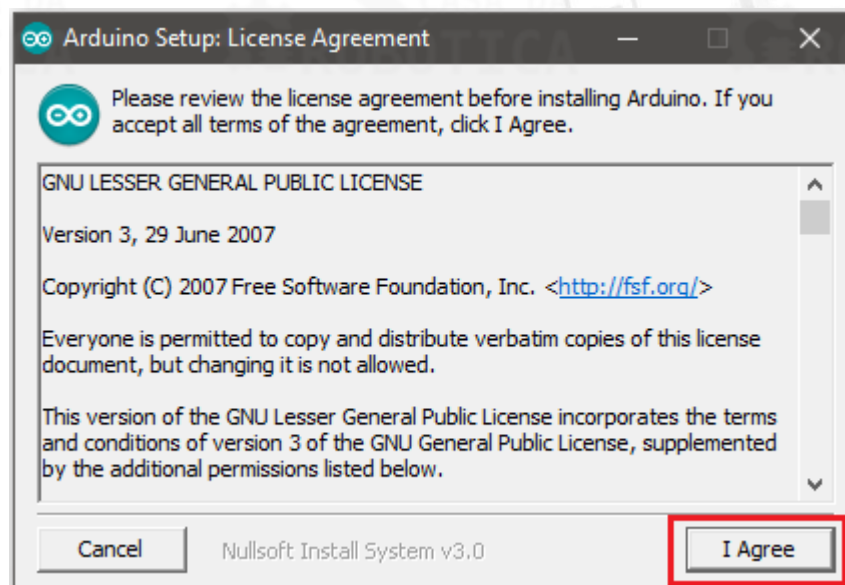
Figura 7 - Passo 4 para instalação do Arduino IDE.



5 – Após isto, aparecerá uma tela do Controle de Conta do Usuário solicitando permissão para instalação do Arduino IDE com a seguinte mensagem “Deseja permitir que este aplicativo faça alterações no seu dispositivo?”. Clique em SIM para iniciar a instalação.

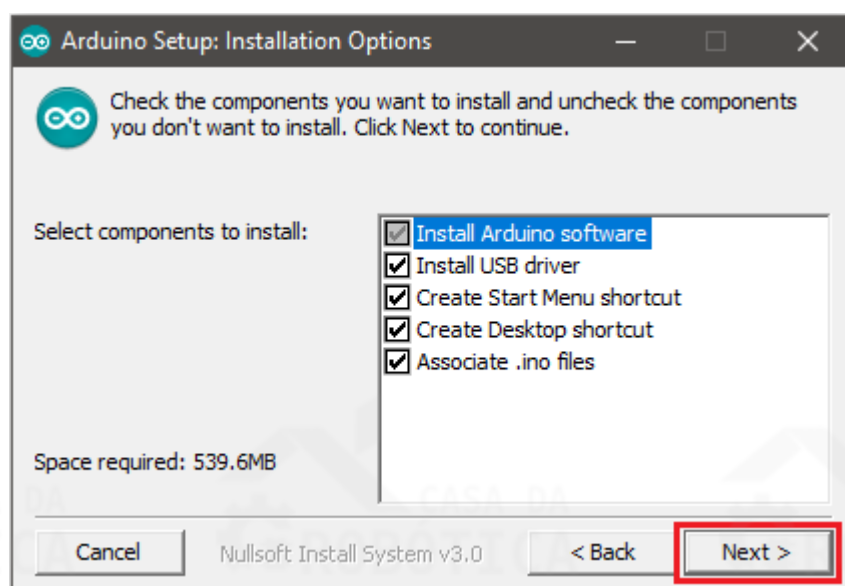
6 – A partir de então a tela de instalação do Arduino IDE será iniciada. A primeira ação que deve ser realizada para instalação deste software é aceitar os termos de licença clicando no botão “I Agree”, mostrado na Figura 8.

Figura 8 - Passo 6 para instalação do Arduino IDE.



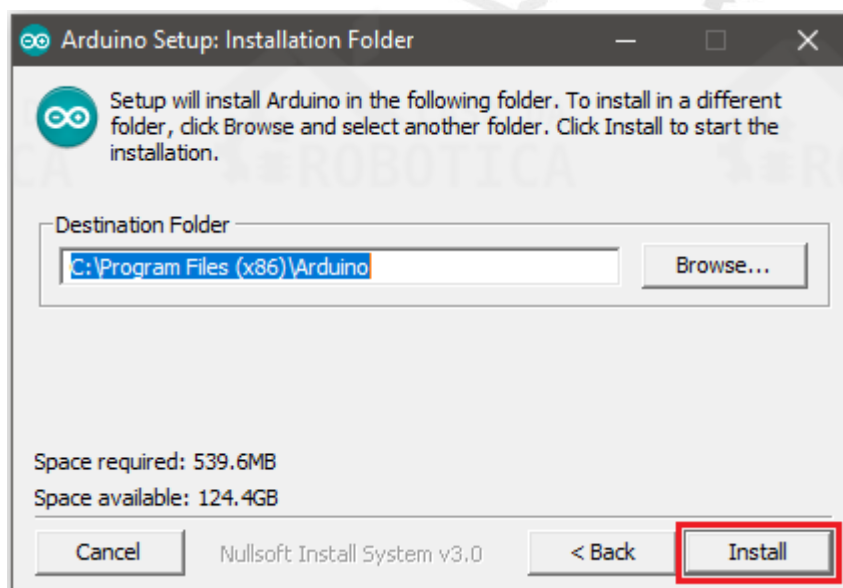
7 – Em seguida, você deve verificar se todos os itens estão selecionados para instalação e clicar no botão “Next >”, conforme a Figura 9.

Figura 9 - Passo 7 para instalação do Arduino IDE.



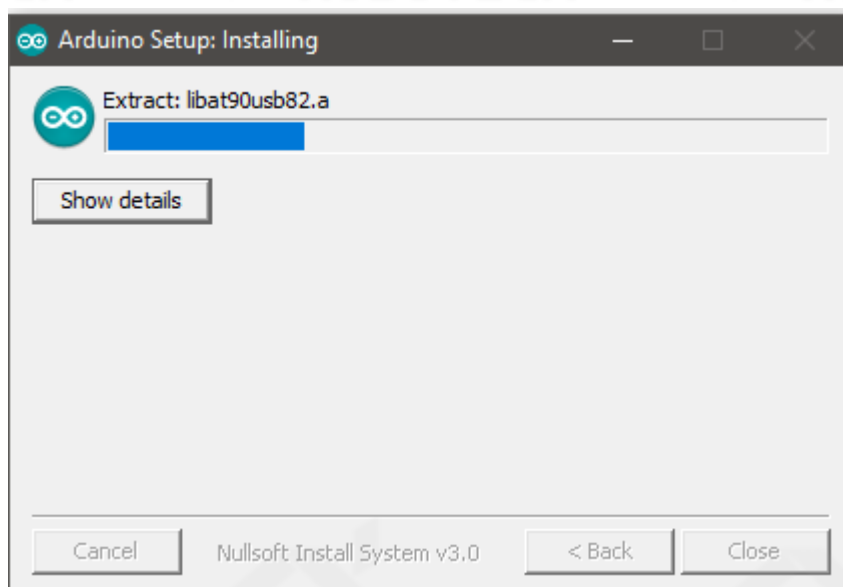
8 – Logo após, selecione a opção “Install” para prosseguir com a instalação.

Figura 10 - Passo 8 para instalação do Arduino IDE.



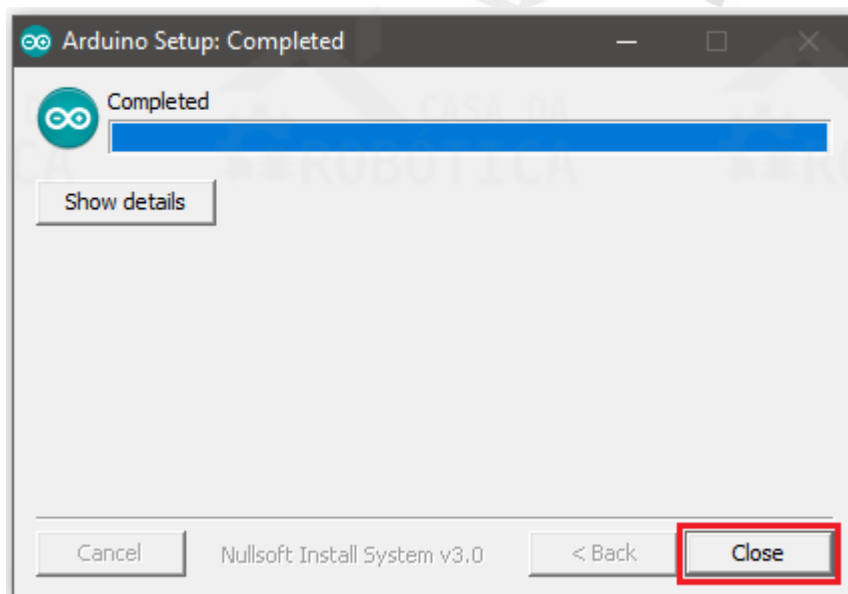
9 – Aguarde a conclusão da instalação. Este processo poderá ser acompanhado, conforme mostra a Figura 11.

Figura 11 - Passo 9 para instalação do Arduino IDE.



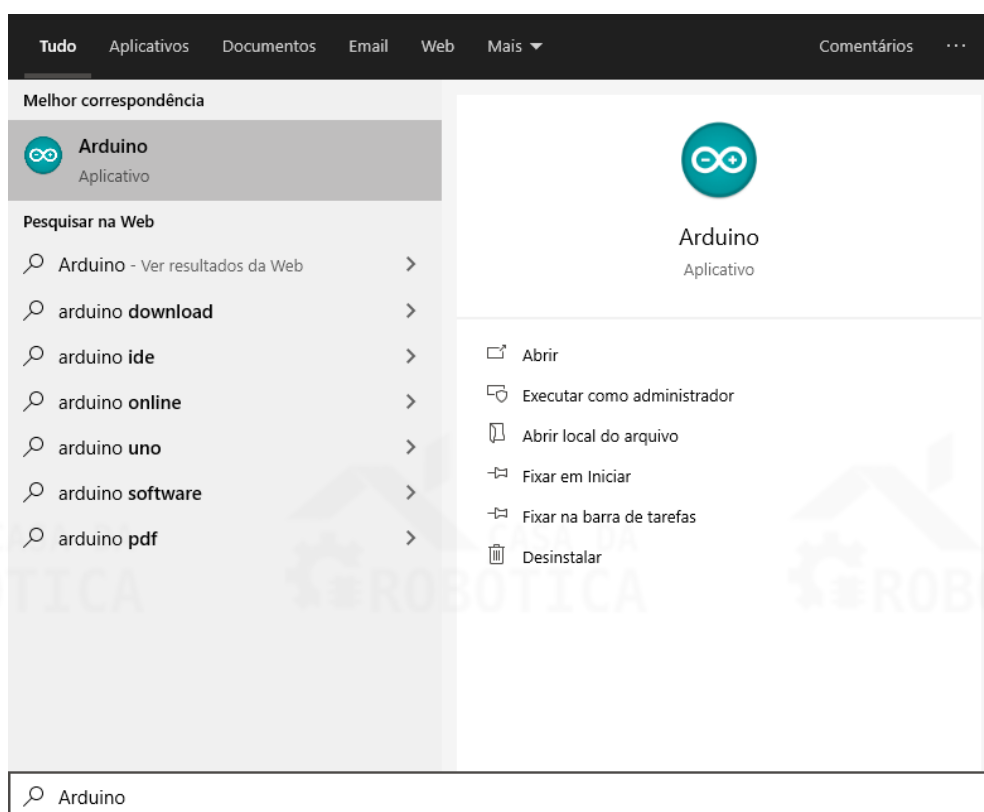
10 – Quando a instalação estiver completada, clique no botão “Close”.

Figura 12 - Passo 10 para instalação do Arduino IDE.



Ocorrendo tudo bem na instalação do Arduino IDE, você pode inicializá-lo através do atalho criado na área de trabalho ou buscando por Arduino no menu iniciar, conforme Figura 13.

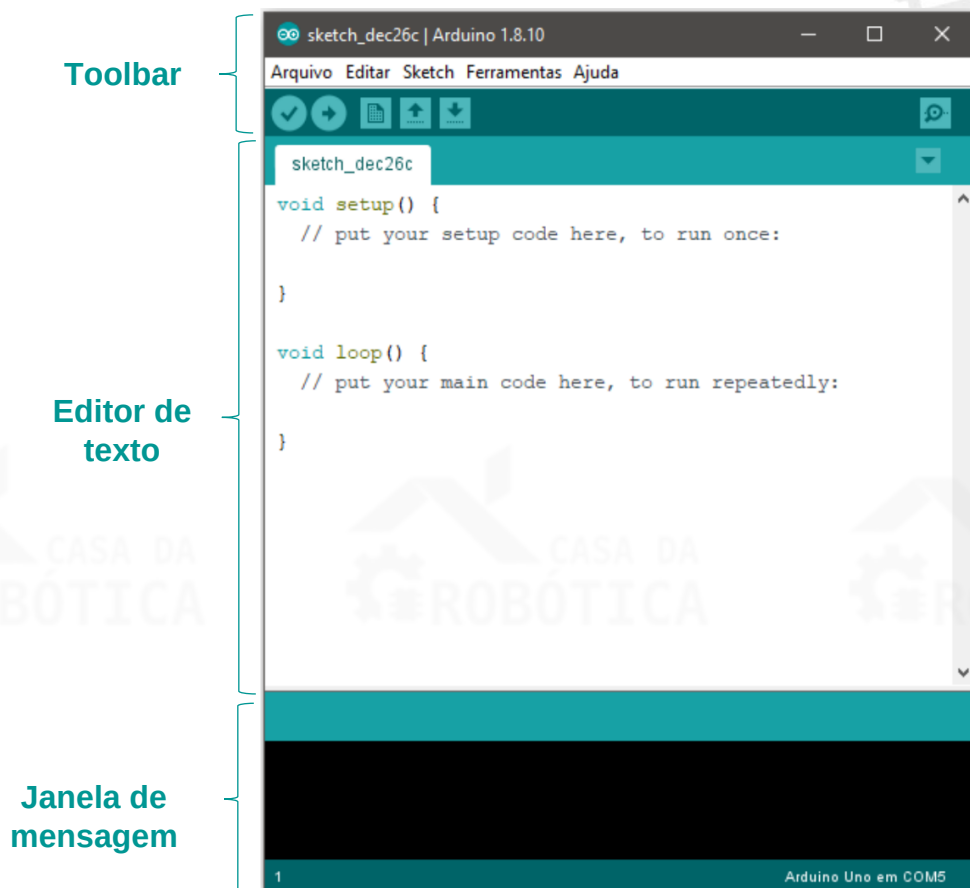
Figura 13 - Inicializando o Arduino IDE.



EXPLORANDO O ARDUINO IDE

Ao abrir o Arduino IDE você verá uma tela semelhante à Figura 14. Caso esteja utilizando o Linux ou Mac OS X, pode haver pequenas diferenças, mas o IDE é basicamente o mesmo para todos os sistemas operacionais.

Figura 14 - Visual do Arduino IDE.



O Arduino IDE pode ser dividido em três partes: A Toolbar no início da tela, o editor de texto no centro e a janela de mensagens na base.

No top da Toolbar há uma barra de menus contendo comandos comuns com os itens: Arquivo, Editar, Sketch, Ferramentas e Ajuda. Os comandos e funções disponíveis na barra de ferramenta podem ser consultados ao acessar o comando Ajuda > Ambiente.

Ainda na Toolbar encontra-se os botões de atalho, que fornecem acesso rápido às funções mais utilizadas. A seguir são mostrados os ícones e o detalhamento de suas funções.

- ✔ **Verificar:** Analisa se há erros em seu código
- ➔ **Upload:** Compila seu código e o envia para a placa microcontroladora
- 📄 **Novo:** Cria um novo Sketch
- ⬆ **Abrir:** Mostra uma lista de Sketch existentes
- ⬇ **Salvar:** Salva o seu Sketch atual
- 🔍 **Monitor serial:** Exibe os dados seriais enviados pela placa microcontroladora

O editor de texto é o campo destinado a escrita dos códigos. Os códigos escritos usando Arduino ou placas compatíveis são conhecidos como Sketches e são salvos com a extensão de arquivo .ino. Este editor de texto tem características de um editor tradicional, contendo funções de cortar, copiar, colar, selecionar tudo, entre outras.

A janela de mensagem fornece mensagens de feedback ao salvar e exportar arquivos, bem como exibe informações de erros no código ou ao compilar.

Conectando a placa UNO ao computador

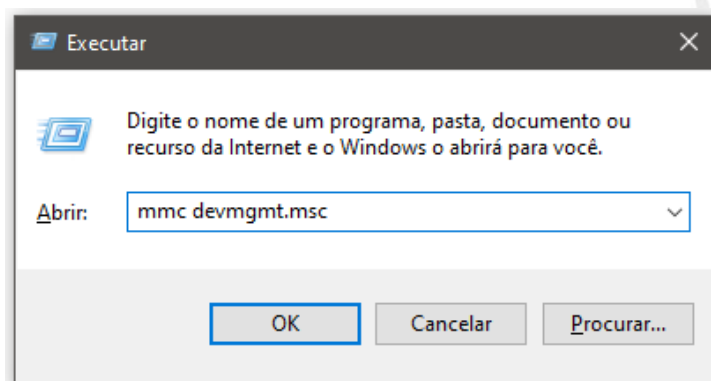
Agora que já conhecemos a placa UNO e sua interface de desenvolvimento, vamos conectá-lo ao computador. Esta conexão é realizada por meio de um cabo USB do tipo A para o tipo B, igual ao da Figura 15.

Figura 15 - Cabo USB do tipo A para o tipo B.



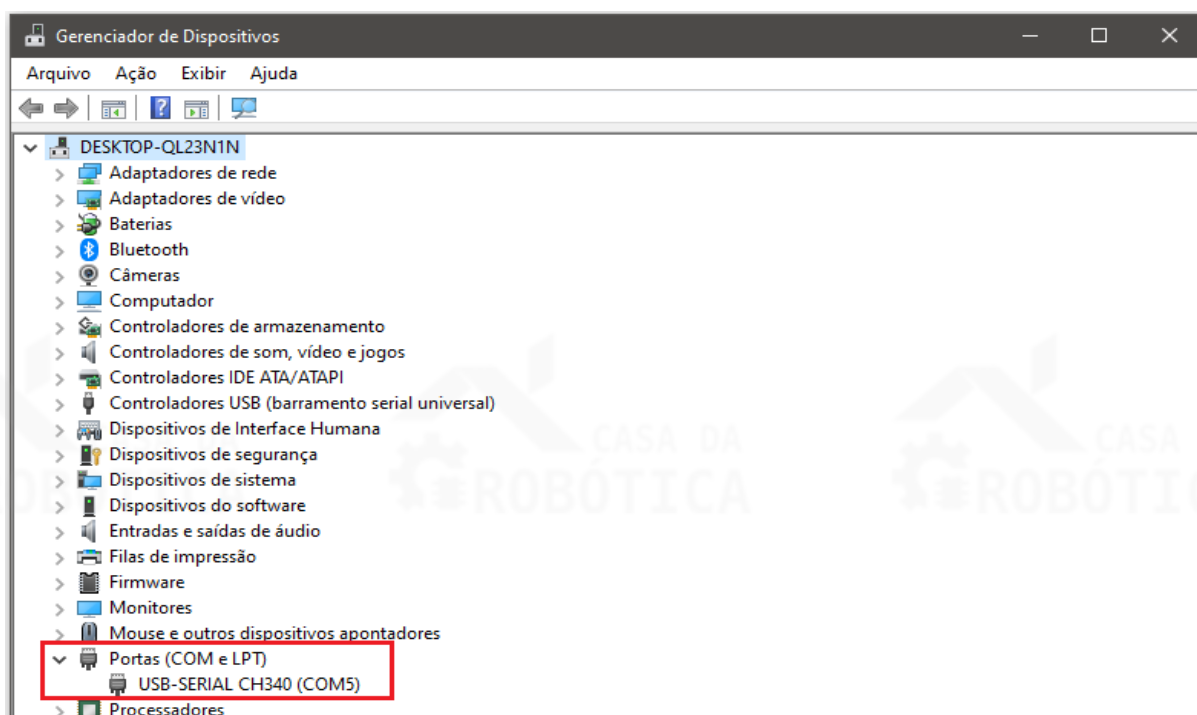
Conecte sua placa UNO ao seu computador através da USB. Para saber se o seu computador Windows está identificando a placa vamos realizar um teste acessando o gerenciador de dispositivos. Uma opção para se chegar neste painel é pressionar as teclas “Windows + r”. Assim que o menu executar abrir digite “mmc devmgmt.msc” sem as aspas, como se pode ser observado na Figura 16.

Figura 16 - Atalho para acessar o gerenciador de dispositivos.



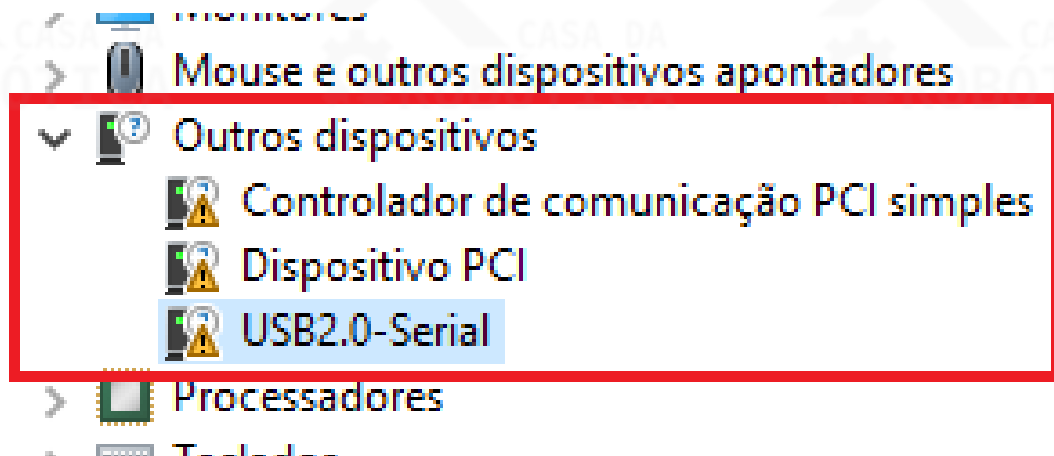
Após digitar esse comando e clicar em “OK” será aberta a tela da Figura 17. Para verificar se o driver da placa UNO foi reconhecido navegue até a opção Portas (COM e LPT) e expanda clicando na setinha do lado do nome. No exemplo abaixo a placa UNO foi reconhecida com sucesso pela porta COM de número 5, essa informação será útil posteriormente.

Figura 17 - Tela do gerenciador de dispositivos.



Caso a placa UNO não seja reconhecida pelo seu computador, ela pode aparecer, com o ícone , em “Outros dispositivos”, como na Figura 18.

Figura 18 - Indicador de que a placa Arduino não foi reconhecida.



Isso acontece devido à falta de um driver para a interpretação do dispositivo. Para resolver esse problema basta baixar o driver disponível no link a seguir e executá-lo:

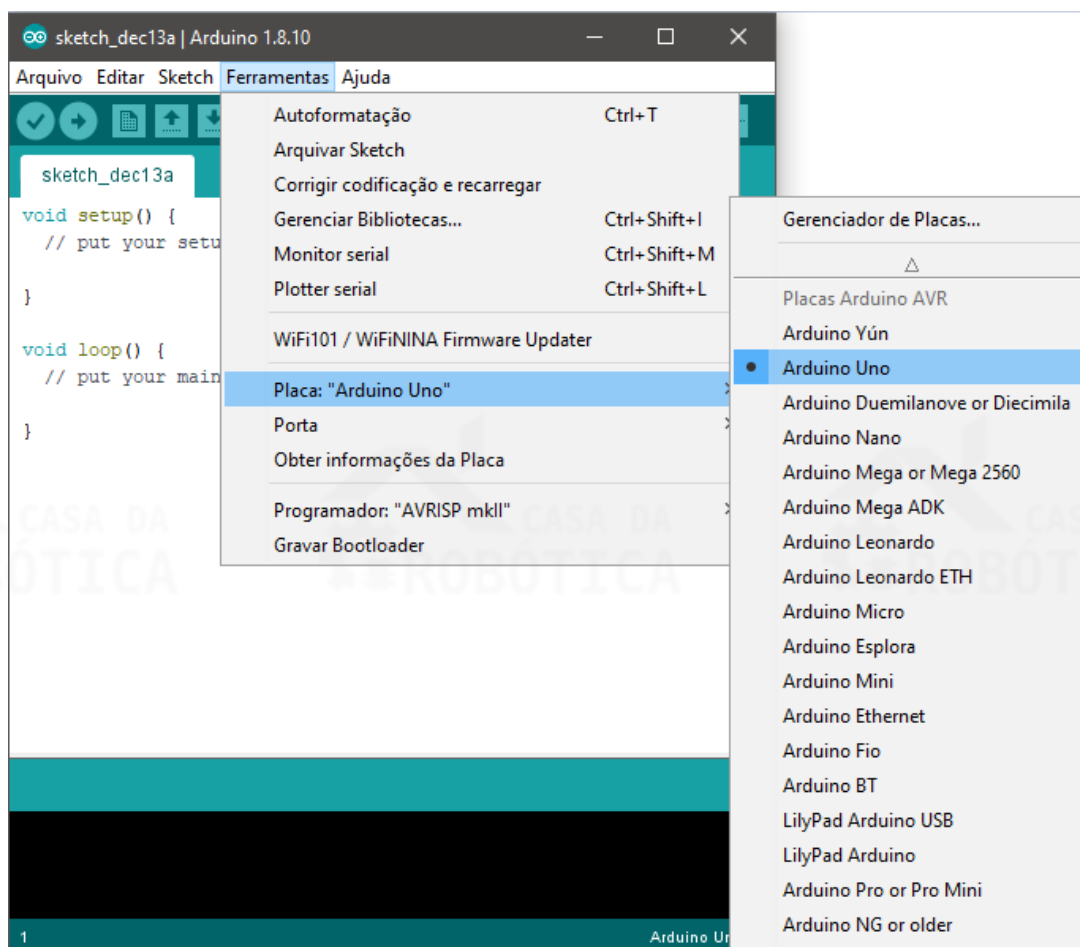
www.blogdarobotica.com/instalando-o-driver-serial-para-arduino/

Após a conclusão do download, execute os arquivos baixados, instale os drivers e reinicie o computador.

Seleção da placa e da porta de comunicação da placa UNO

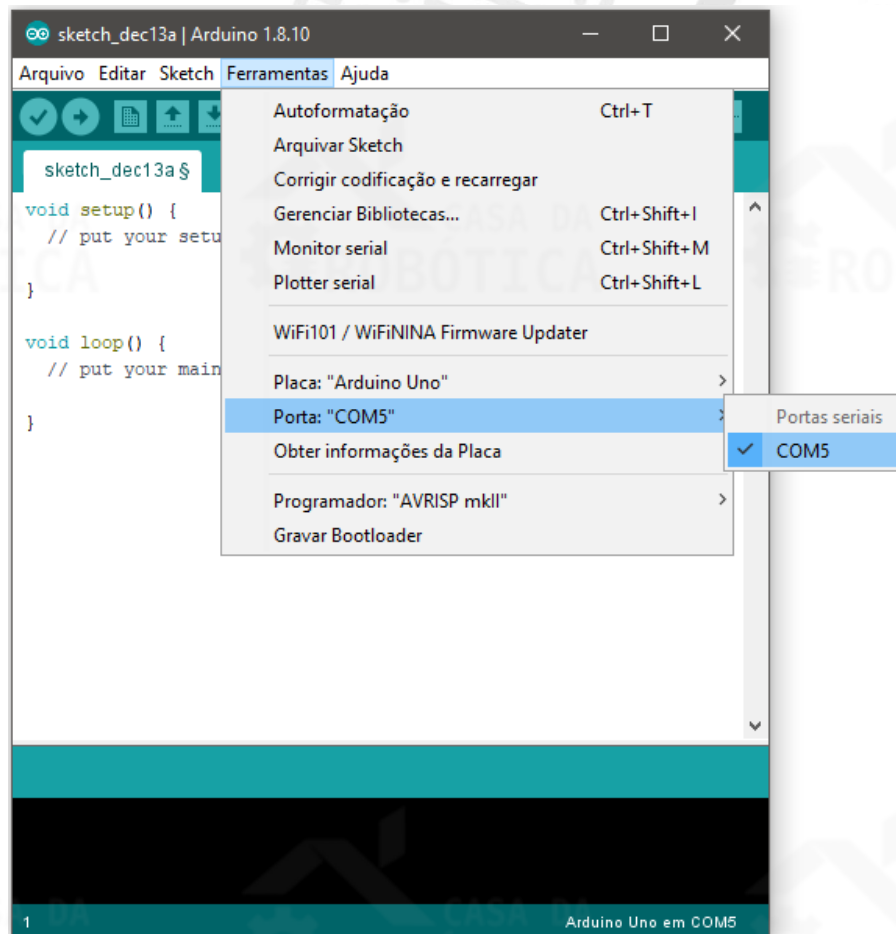
Agora que já temos a placa UNO conectada ao computador, vamos selecionar a placa e a porta de comunicação no Arduino IDE. Para tal, deve-se selecionar o modelo da placa utilizada no menu Ferramentas, para nossos exemplos usaremos o Arduino Uno, conforme Figura 19.

Figura 19 - Seleção da placa Arduino.



Após a seleção do modelo, deve-se selecionar a porta de comunicação a placa foi atribuída, ou seja, a porta que a placa UNO foi reconhecida. Como vimos anteriormente, em nosso exemplo, a placa UNO foi reconhecida pela COM de número 5. A Figura 20 mostra a seleção da COM através do menu Ferramentas.

Figura 20 - Seleção da porta COM.



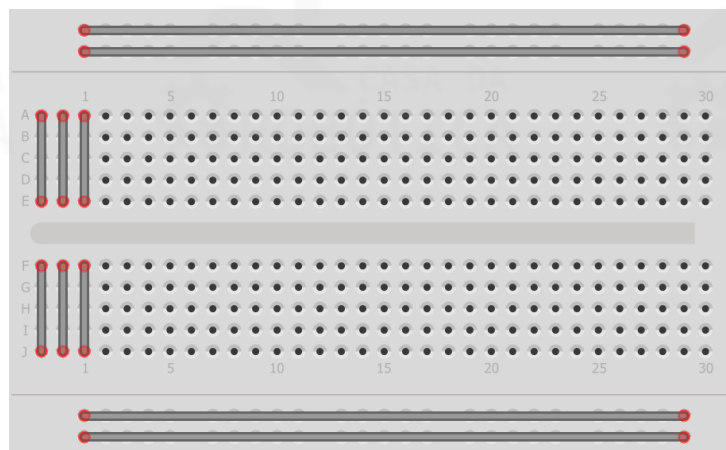
Após isto, sua placa UNO estará pronta e o ambiente de desenvolvimento configurado para uso.

3.CONHECENDO OS DEMAIS COMPONENTES DO KIT INICIANTE

PROTOBOARD

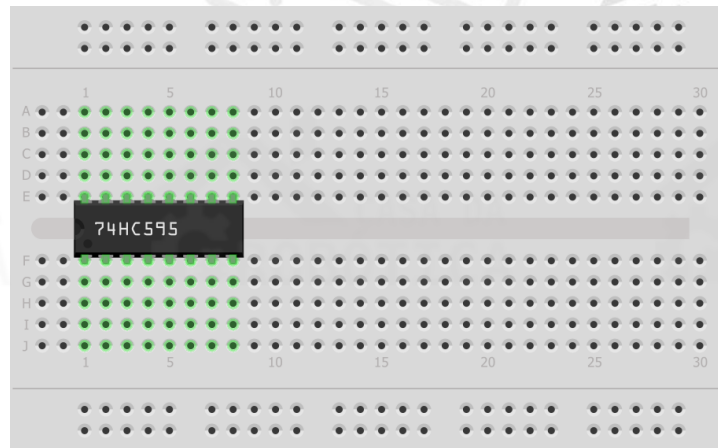
A protoboard, ou placa de ensaios, é um dispositivo reutilizável para montagem e prototipagem circuitos elétricos experimentais, sendo amplamente utilizada devido a facilidade de inserção de componentes e por não necessitar de soldagens. Este componente é formado por uma série de furos dispostos em grades. Esses furos são conectados por uma tira de metal condutivo. A forma como essas tiras são dispostas pode ser visualizada na Figura 21.

Figura 21 - Modo de disposição das tiras de metal em uma protoboard.



As tiras dispostas no topo e na base da protoboard são longas e possuem contato horizontal, sendo geralmente utilizadas para carregar o barramento de alimentação e o barramento terra. No entanto, as tiras no centro são agrupadas de 5 em 5 furos, possuem contato na vertical e contam com um espaço vazio para encaixe de circuitos integrados (CIs), de forma que cada pino do mesmo se conecte a um conjunto diferente de furos, ou seja, para um barramento diferente, conforme pode ser visto na Figura 22.

Figura 22 - Circuito integrado conectado em uma protoboard.



JUMPER

Os jumpers (Figura 23) são pequenos fios condutores utilizados para conectar dois pontos de um circuito eletrônico. Os jumpers facilitam a conexão entre componentes elétricos, sendo uma excelente escolha para montagem de projetos e interligação da placa UNO com a protoboard.

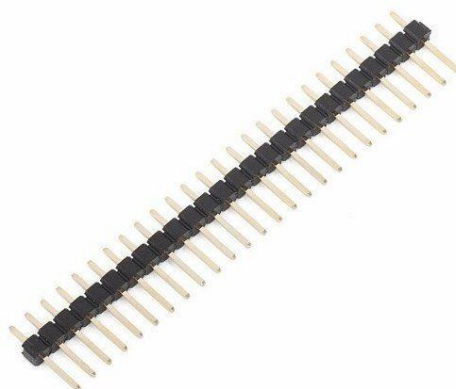
Figura 23 - Jumpers.



PIN HEADER

O pin header é um tipo de conector elétrico constituído por uma ou mais fileiras de pinos fêmea ou macho. A Figura 24 ilustra um pin header macho de uma fileira.

Figura 24 - Pin header macho.



O pin header pode ser soldado na placa UNO de modo que a conexão das portas analógicas e digitais com o circuito do projeto seja efetuada utilizando jumpers fêmea-macho.

CABO PARA BATERIA 9 V COM PLUG P4

O cabo para bateria 9 V com plug V4 (Figura 25) é um adaptador que permitirá a alimentação da placa UNO por meio de uma bateria de 9V.

Figura 25 - Cabo para bateria 9 V com plug V4.



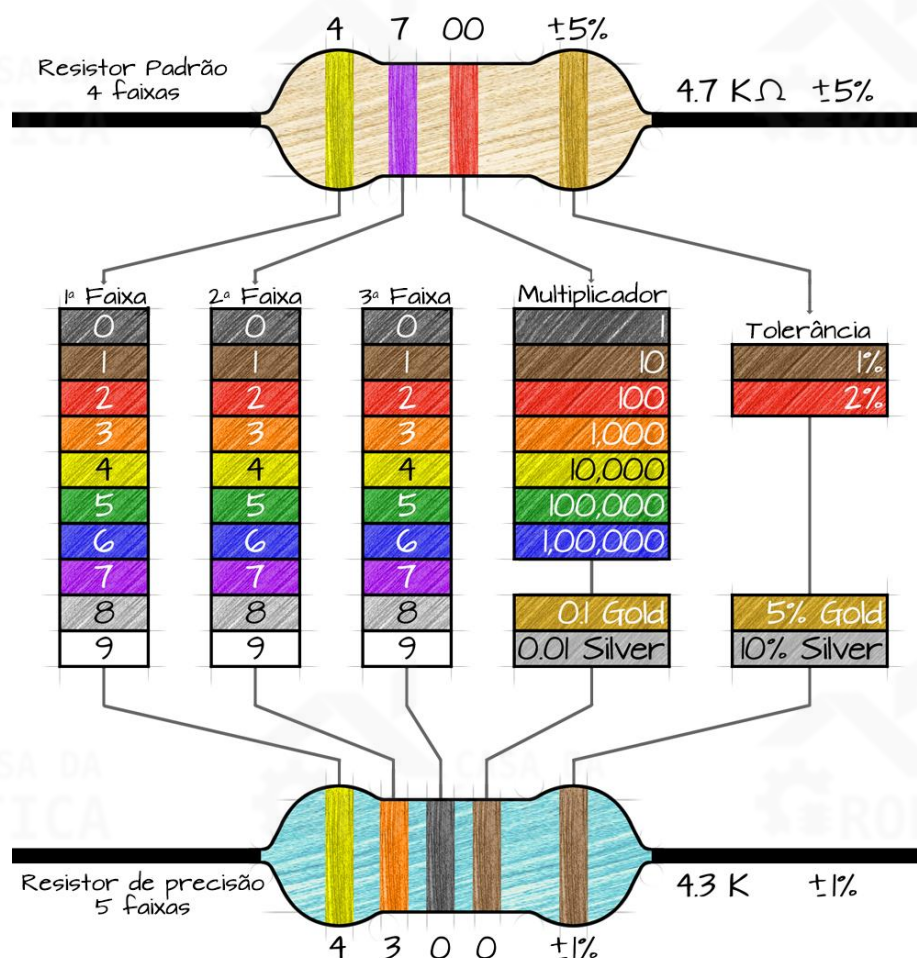
RESISTOR

O resistor é um dispositivo elétrico projetado com a finalidade de limitar a corrente elétrica em um circuito, causando uma queda de tensão em seus terminais.

Para facilitar o entendimento, você pode pensar em um resistor como um cano estreito conectado no meio da tubulação de água. Conforme a água (ou corrente elétrica) entra no resistor, o cano se torna mais estreito e o volume de água (corrente elétrica) irá reduzir. Os resistores são utilizados para diminuir a tensão ou a corrente. O valor da resistência é medido em ohm e seu símbolo é a letra grega ômega (Ω).

O valor do resistor é identificado por meio do código de cores, que consiste em faixas coloridas no corpo do mesmo. As três primeiras faixas servem para indicar o valor nominal e a última faixa, a porcentagem na qual a resistência pode variar. A Figura 26 ilustra o código de cores dos resistores.

Figura 26 - Código de cores dos resistores.



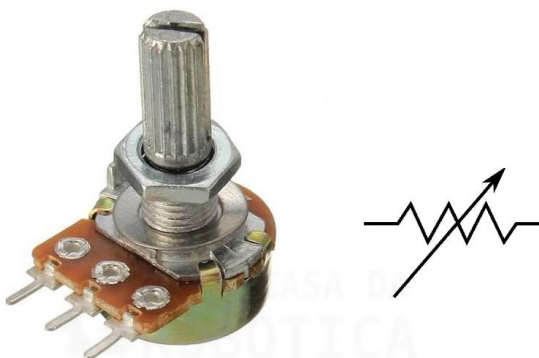
Fonte: Ohm's Law Calculator.2019.

POTENCIÔMETRO

O potenciômetro é um componente eletrônico utilizado para limitar o fluxo de corrente elétrica. Desta forma, este dispositivo impõe resistência elétrica em um circuito, assim como um resistor, no entanto esta resistência pode ser variada manualmente.

O potenciômetro normalmente possui três terminais e um eixo giratório para ajuste da sua resistência, que é medida em ohms (Ω). A Figura 27 ilustra um tipo de potenciômetro e sua simbologia.

Figura 27 - Potenciômetro e sua simbologia.



Os potenciômetros são amplamente utilizados em amplificadores de áudio, instrumentos musicais eletrônicos, mixers de áudio, eletrodomésticos, equipamentos industriais, joysticks, entre outros.

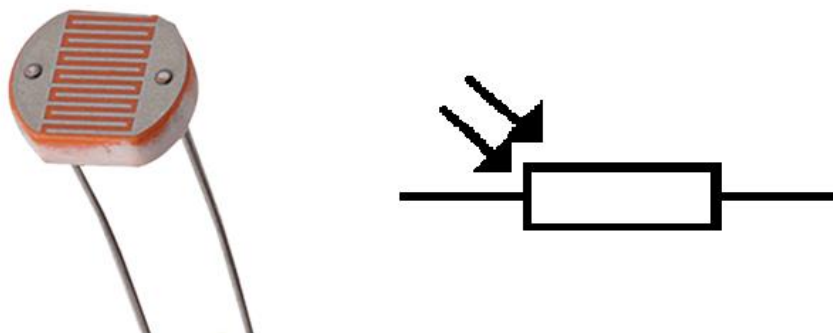
SENSOR DE LUZ - LDR

O LDR (*Light Dependent Resistor*, ou Resistor Dependente de Luz) é um componente eletrônico cuja resistência elétrica varia de acordo com a luminosidade que incide sobre ele, ou seja, quando ocorre a ausência de luminosidade a resistência do LDR é muito grande, no entanto, quando este é iluminado, a resistência diminui, resultando em um grande aumento da corrente elétrica nos terminais.

O LDR (Figura 28), também conhecido como fotoresistor, é um dispositivo eletrônico amplamente difundido e utilizado em circuitos controladores de iluminação,

em fotocélulas, medidores de luz, entre outros, devido ao seu baixo custo e facilidade de utilização. Em conjunto com a placa UNO, o LDR pode ser aplicado em projetos nos quais se deseja controlar o acionamento de uma carga em função da presença ou ausência de luminosidade sobre a superfície do sensor.

Figura 28 - Sensor LDR e sua simbologia.



SENSOR DE TEMPERATURA – TERMISTOR NTC

Um termistor é um dispositivo elétrico que tem sua resistência alterada termicamente, em outras palavras, sua resistência muda conforme a temperatura do ambiente.

Esse dispositivo é amplamente utilizado para controlar e/ou alterar a temperatura em dispositivos eletroeletrônicos, como termômetros, circuitos eletrônicos de compensação térmica, dissipadores de calor, ar condicionados, entre outros.

Os termistores podem ser classificados em PTC e NTC. No termistor do tipo PTC (do inglês, *Positive Temperature Coefficient*) sua resistência elétrica aumenta sensivelmente com o aumento da temperatura. Por sua vez, no NTC (do inglês, *Negative Temperature Coefficient*) sua resistência diminui sensivelmente à medida que a temperatura aumenta.

Para medir temperatura normalmente utiliza-se os termistores do tipo NTC. Existem termistores NTC de diversos valores disponíveis no mercado, no entanto o mais comum é o de 10K. A Figura 29 ilustra um termistor NTC e sua simbologia.

Figura 29 - Termistor NTC e sua simbologia.



Algumas especificações técnicas do termistor NTC 10K encontram-se detalhadas a seguir:

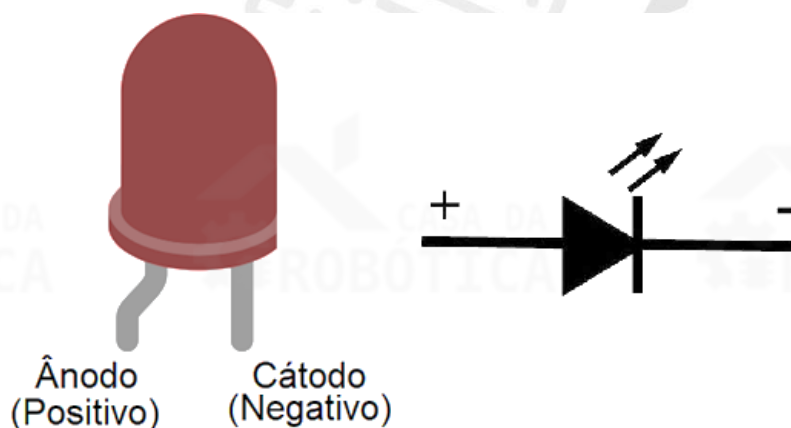
- Resistência a 25°C: 10 k Ω ;
- Faixa de temperatura: -55°C ~ 125°C;
- Corrente máxima de estado estacionário: 5 mA;
- Constante de dissipação térmica: 2mW/°C;
- Constante de tempo térmico: 15 segundos.

LED

O LED, do inglês *Light Emitting Diode* (Diodo Emissor de Luz), é um dispositivo eletrônico capaz de emitir luz visível através da transformação da energia elétrica em energia luminosa. Os LEDs estão disponíveis em todos os tipos de cores e níveis de luminosidade, incluindo ultravioleta e infravermelho.

Ao examinar um LED você perceberá que os terminais têm comprimentos diferentes e que um lado do LED é chanfrado, em vez de cilíndrico. O terminal mais comprido é o ânodo (positivo) e deve ser conectado à alimentação positiva e o terminal chanfrado é o cátodo (negativo) e deve ser ligado ao terra. A Figura 30 ilustra os terminais do LED e sua simbologia.

Figura 30 – LED, seus terminais e simbologia.

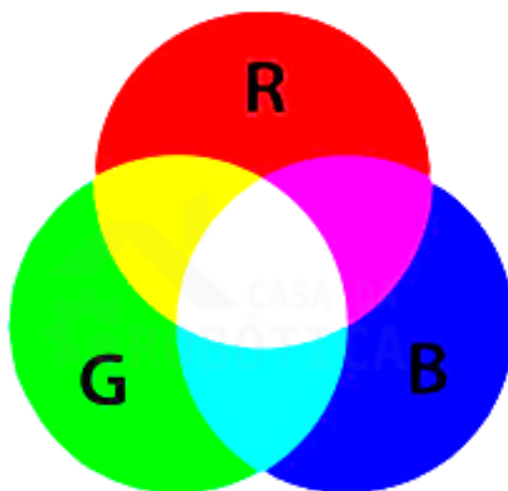


LED RGB

Os LEDs RGB consistem na junção de três LEDs em um só dispositivo, mas que podem ser controlados individualmente. Cada um destes LEDs possuem uma cor distinta: Um vermelho (*Red*), um verde (*Green*) e um azul (*Blue*), que, quando associadas, podem formar outras cores.

A definição dessas cores é baseada na combinação aditiva das cores. Por exemplo, ao adicionar a cor verde a vermelha, obteremos amarelo, acrescentar a cor azul a vermelha, obteremos a cor magenta (violeta-púrpura), entre outras, conforme pode ser observada na Figura 31.

Figura 31 - Combinação de cores.

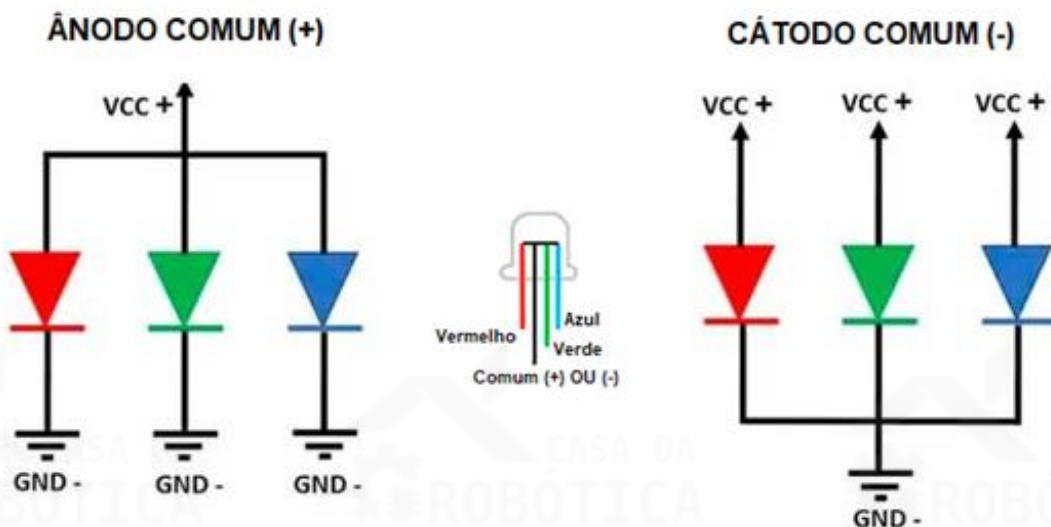


Os LEDs RGB podem ser:

- **Ânodo comum:** quando os terminais das cores vermelha, verde e azul são conectados ao terminal negativo ou terra da fonte de energia;
- **Cátodo comum:** quando os terminais das cores vermelha, verde e azul são conectados ao terminal positivo da fonte de energia.

A Figura 32 ilustra a diferença de modo de ligação dos dois tipos de LEDs RGB.

Figura 32 - Modo de ligação dos LEDs RGB cátodo comum e ânodo comum.



DISPLAY DE LED DE 7 SEGMENTOS

O display de 7 segmentos é formado por sete LEDs retangulares dispostos de forma que, por meio da combinação de LEDs ligados e desligados, possa ser mostrada informações alfanuméricas (números e letras).

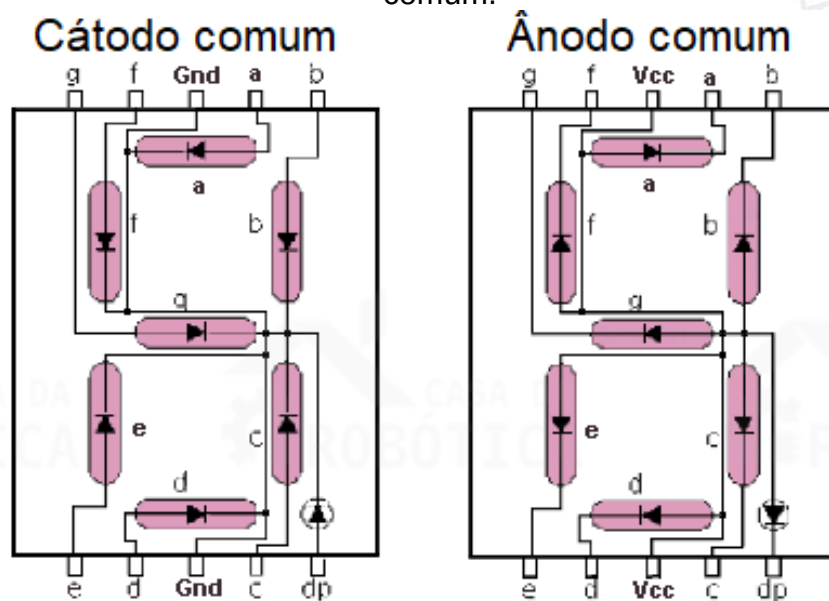
Esse componente é muito utilizado em projetos em que faz-se necessária a apresentação de informações alfanuméricas de forma visual, como em placares, calculadoras, contadores de produtos, relógios digitais, termômetro digital, entre outras aplicações.

Há dois tipos de displays de 7 segmentos definidos em função da sua alimentação, podendo ser **cátodo comum** ou **ânodo comum**. No display de 7 segmentos **cátodo comum** todos os terminais cátodos dos LEDs são interligados

internamente, esse terminal comum deve ser ligado ao terra (GND) e para acionar o segmento é necessário aplicar uma tensão (V_{cc}) ao terminal do mesmo. Por sua vez, o display de 7 segmentos **ânodo comum** é o contrário, ou seja, o terminal comum é conectado em 5 V (V_{cc}) e para acionar o segmento é necessário ligá-lo ao terra (GND).

A Figura 33 ilustra o esquemático interno do display de 7 segmentos cátodo comum e ânodo comum.

Figura 33 – Esquemático dos displays de 7 segmentos cátodo comum e ânodo comum.



BOTÃO PUSH BUTTON

O botão push button é um interruptor pulsador que conduz corrente elétrica apenas quando pressionado. Este componente eletrônico é muito utilizado na prototipagem de projetos eletrônicos tanto na protoboard quanto soldado na placa de circuito impresso.

O botão push button pode ser utilizada para acionamento de um circuito elétrico. A Figura 34 ilustra uma chave tátil.

Figura 34 – Botão push button com capa.



BUZZER

O buzzer é um componente eletrônico que converte um sinal elétrico em onda sonora. Este dispositivo é utilizado para sinalização sonora, sendo aplicado em computadores, despertadores, carros, entre outros.

O buzzer (Figura 35) é composto por duas camadas de metal, uma terceira camada de cristal piezelétrico, envolvidas em um invólucro de plástico, e dois terminais para ligação elétrica.

Figura 35 - Buzzer.



Existem dois tipos de buzzer: o buzzer ativo e o buzzer passivo. Embora estes sejam idênticos visualmente sua forma de funcionamento e aplicações são bem diferentes.

O buzzer ativo possui um circuito mais complexo que o passivo, no entanto seu uso é mais simples necessitando apenas de ser energizado para emitir um sinal sonoro. Este componente é apropriado para alarmes, avisos e sinais de alerta.

Por sua vez, o buzzer passivo é um pouco mais difícil de ser utilizado, pois sua forma de operação depende da frequência de onda enviada pelo microcontrolador. Dependendo da frequência dessa onda a frequência do som se altera. Esse componente é ideal para construção de melodias, visto que é possível ter o controle dos tons gerados.

SENSOR INFRAVERMELHO TCRT5000

O sensor infravermelho é um dispositivo eletrônico que emite e/ou detecta radiação infravermelha com o intuito de revelar alguns aspectos do ambiente a seu redor. Esses sensores podem ser utilizados para detectar movimentos, medir o calor de um objeto, em leitores de código de barras, alarmes de passagem, entre outros.

Para aplicações de robótica, o sensor infravermelho é muito utilizado na construção de robô seguidor de linha. Este robô possui como objetivo percorrer um determinado trajeto sendo direcionado por uma linha preta, branca ou cores intermediárias. Para esta aplicação, o TCRT5000 (Figura 36) é amplamente utilizado.

Figura 36 – Sensor TCRT5000.



O sensor infravermelho TCRT5000 inclui um emissor infravermelho e um fototransistor em uma embalagem que bloqueia a luz visível.

Algumas características técnicas do TCRT5000 encontram-se listadas a seguir:

- Faixa de alcance: 0.2mm ~ 15 mm;
- Comprimento de onda: 950 nm;
- Altura do cabeçote: 7 mm;
- Método de funcionamento: Reflexão;
- Tensão inversa: 5V;
- Corrente de trabalho: 60mA;
- Potência total: 200mW;
- Temperatura de operação: -25°C ~ +85°C.

REED SWITCH

O reed switch, também conhecido como interruptor de lâmina, é um dispositivo eletrônico formado por um bulbo de vidro com duas lâminas flexíveis, hermeticamente seladas, que se movimentam com a ação de campo magnético.

Em condições normais estas lâminas encontram-se separadas e não conduzem corrente elétrica, operando como uma chave aberta. Por sua vez, quando um gerador de campo magnético, como um ímã, é posto próximo deste componente, o campo magnético magnetiza as lâminas e gera uma atração entre elas, o que ocasiona no fechamento dos contatos e, conseqüentemente, na passagem de corrente elétrica, passando a funcionar como uma chave fechada.

Figura 37 - Reed switch.



O reed switch encontra-se ilustrado na Figura 37. Dentre suas aplicações estão alarmes, sensor de presença ou passagem de determinado objeto, sensor de nível, entre outras.

Algumas características técnicas do reed switch incluso no Kit Iniciante:

- Potência máxima: 4 W;
- Tensão máxima de comutação: 60VDC/VAC;
- Corrente máxima de comutação: 0.25A;
- Corrente de carga máxima: 0.5A;
- Máxima resistência de contato: 115 M;
- Tempo de ação: 0.25 ms;
- Resposta de frequência: 3000Hz.



OBSERVAÇÃO:

- Por possuir um invólucro de vidro, o reed switch se torna bastante frágil e deve ser manuseado com bastante cuidado.

4.FUNDAMENTOS DE PROGRAMAÇÃO

Programação pode ser definida como o processo de projetar, escrever, verificar e manter um código ou programa que comande as ações de uma máquina, como computador, celular, microcontrolador, entre outras.

Um código, programa ou sketch – como são denominados os códigos em Arduino, são compostos por um conjunto de instruções sequenciais definido pelo programador que descrevem as tarefas que a máquina deve realizar. Em outras palavras, para que a máquina execute comandos específicos é necessário elaborar um programa escrito contendo todas as instruções, ordenadas sequencialmente, em uma linguagem de programação.

Uma linguagem de programação é um idioma artificial desenvolvido para prover a comunicação entre uma pessoa (programador) e uma máquina (computador, microcontrolador, etc.). Atualmente, existem centenas de linguagens de programação e em todas elas o programador pode definir como a máquina deve atuar, como armazenar ou transmitir os dados, quais ações tomar em diferentes situações, quando finalizar a sua operação, entre outras. Dentre as linguagens de programação mais populares pode-se citar o C, C++, C#, Java, JavaScript, Python, PHP.

Os programas para microcontroladores compatíveis com o projeto Arduino, como o microcontrolador UNO R3 Atmega328 incluso neste Kit Iniciante, são implementados tendo como referência a linguagem de programação C++, mantendo preservada sua sintaxe clássica de declaração de variáveis, nos operadores, nas estruturas e em muitas outras características.

Antes de aprendermos a linguagem de programação do Arduino devemos conhecer alguns elementos básicos que compõem um Sketch. Desta forma, a seguir apresentaremos a estrutura de um sketch, variáveis, funções e bibliotecas.

Estruturas básica de um Sketch

Todos os Sketch Arduino devem ter a estrutura composta pelas funções `setup()` e `loop()`, conforme ilustra o exemplo a seguir.


```
1 void setup()
2 {
3     Comando 1;
4     Comando 2;
5     ...
6 }
7 void loop()
8 {
9     Comando 3;
10    Comando 4;
11    ...
12 }
```

A função `setup()` é executada apenas uma vez na inicialização do programa, e é nela que você deverá descrever as configurações e instruções gerais para preparar o programa antes que o loop principal seja executado. Em outras palavras, a função `setup()` é responsável pelas configurações iniciais da placa microcontroladora, tais como definições de pinos de entrada e saída, inicialização da comunicação serial, entre outras.

A função `loop()` é a função principal do programa e é executada continuamente enquanto a placa microcontroladora estiver ligada. É nesta função que todos os comandos e operações deverão ser escritos.



OBSERVAÇÕES:

- As instruções dadas em um Sketch são realizadas de forma sequencial;
- É necessário incluir o sinal ; (ponto e vírgula) para que o programa identifique onde uma instrução deve ser finalizada.

Variáveis

As variáveis são expressões que podemos utilizar em programas para nomear e armazenar um dado para uso posterior, como dados de um sensor ou um valor calculado.

Antes de serem utilizadas, todas as variáveis devem ser declaradas, definindo seu tipo e, opcionalmente, definindo seu valor inicial. Alguns tipos de variáveis encontram-se listadas na Tabela 2:

Tabela 2 - Tipos de variáveis e suas descrições.

Tipo	Descrição
<code>char</code>	Utilizado para armazenar um valor de caractere.
<code>byte</code>	Usado para armazenar um número entre 0 e 255.
<code>int</code>	Utilizado para armazenar números inteiros.
<code>bool</code>	Empregado para armazenar dois valores: <code>true</code> (verdadeiro) ou <code>false</code> (falso).
<code>float</code>	Armazena números decimais que ocupam 32 bits (4 bytes).
<code>double</code>	Armazena números decimais que ocupam 64 bits (8 bytes).
<code>void</code>	Usada apenas em declarações de funções
<code>String</code>	Utilizada para armazenar cadeias de texto.

A declaração de uma variável pode ser melhor entendida com o exemplo a seguir. Neste caso, declaramos uma variável com nome `a` do tipo `int`, uma variável com nome `b` do tipo `float` e uma variável de nome `c` do tipo `char`. Quando as variáveis são declaradas antes da função `setup()`, significa que elas são variáveis globais, que podem ser usada em todas ao longo de todo o programa. Por sua vez, as variáveis declaradas em funções específicas, como no `loop()`, são variáveis locais e só podem ser usadas dentro da sua função de origem.

```
1  int a;  
2  float b;  
3  char C;  
4  
5  void setup()  
6  {  
7      ...  
8  }
```

Para definir um valor inicial a variável utilizamos o comando de atribuição, representado pelo símbolo `=`. Desta forma, quando escrevemos `int a = 10;` estamos atribuindo o valor `10` a variável de nome `a` do tipo `int`.



OBSERVAÇÕES:

- O nome das variáveis deve seguir algumas regras: Devem iniciar com letras ou sublinhado `_` e não podem ter nome idêntico a alguma das palavras reservadas pelo programa ou biblioteca;
- Letras maiúsculas e minúsculas fazem diferença. Se o nome da variável definida for algo como, por exemplo, `ledPin`, não será a mesma coisa que `LedPin` ou `LEDPIN`;
- O símbolo `=` tem o papel exclusivo de atribuição. A igualdade matemática é representada pela dupla igualdade `==`.

Funções

Funções são blocos de instruções que podem ser chamados em qualquer parte do seu Sketch. A principal motivação para o uso de funções em um programa é quando precisamos executar a mesma ação várias vezes. Desta forma, a segmentação do código em funções permite a criação de trechos de código modulares que executam uma tarefa definida e retornam à área do código a partir da qual a função foi chamada.

O uso de funções possui várias vantagens, entre elas:

- As funções ajudam o programador a manter o sketch organizado;
- As funções codificam uma ação em um só lugar, de forma que o trecho do código precise ser pensado e escrito apenas uma vez;
- Reduz as chances de erros na modificação quando o código precisa ser alterado;
- Facilitam a reutilização de código em outros programas;
- Tornam o código mais legível.

As duas funções principais na criação de um sketch no Arduino são `void setup()` e `void loop()`, mas existem algumas outras funções predefinidas para controlar uma placa microcontroladora, conforme mostra a Tabela 3:

Tabela 3 – Funções e suas descrições.

Controle	Função	Descrição
Entrada e saída digitais	<code>digitalRead()</code>	Lê o valor de um pino digital especificado
	<code>digitalWrite()</code>	Escreve no pino digital HIGH ou LOW
	<code>pinMode()</code>	Configura o pino para funcionar como saída ou entrada
Entrada e saída analógicas	<code>analogRead()</code>	Lê o valor de um pino analógico especificado
	<code>analogReference()</code>	Configura a tensão de referência para a entrada analógica
	<code>analogWrite()</code>	Aciona uma onda PWM em um pino
Funções temporizadoras	<code>delay()</code>	Pausa o programa por um período (em milissegundos)
	<code>millis()</code>	Retorna o número de milissegundos passados desde que a placa Arduino começou a executar o programa.
Entradas e saídas avançadas	<code>tone()</code>	Gera uma onda quadrada na frequência especificada em um pino
	<code>noTone()</code>	Interrompe a função <code>tone()</code>
Funções matemáticas	<code>map()</code>	Mapeia um intervalo numérico em outro intervalo desejado
	<code>sq()</code>	Calcula o quadrado de um número
	<code>sqrt()</code>	Calcula a raiz quadrada de um número
Funções trigonométricas	<code>cos()</code>	Calcula o cosseno de um ângulo (em radianos)
	<code>sin()</code>	Calcula o seno de um ângulo (em radianos)
	<code>tan()</code>	Calcula a tangente de um ângulo (em radianos)
Números aleatórios	<code>random()</code>	Gera números pseudoaleatórios.
	<code>randomSeed()</code>	Inicializa o gerador de números pseudoaleatórios
Comunicação	<code>Serial</code>	Usada para comunicação entre uma placa Arduino e um computador ou outros dispositivos
Interrupções Externas	<code>attachInterrupt()</code>	Cria interrupção externa
	<code>detachInterrupt()</code>	Desativa a interrupção externa
Interrupções	<code>interrupts()</code>	Reativa interrupções
	<code>noInterrupts()</code>	Desativa interrupções



OBSERVAÇÕES:

- Para saber mais sobre as funções predefinidas do Arduino acesse o site: <https://www.arduino.cc/reference/pt/>. Nele você encontrará outras funções

não especificadas neste material de apoio e poderá consultar a descrição, sintaxe e parâmetros das funções que desejar;

- Os sinais analógicos são aqueles que variam continuamente ao longo do tempo. Por sua vez, os sinais digitais assumem valores discretos (0 ou 1);
- Configurar um pino digital em HIGH significa colocar o pino digital em nível lógico alto (1), ou seja 5 V. Definir um pino digital como LOW significa colocar o pino digital em nível lógico baixo (0), ou seja, 0 V.
- Outros conceitos técnicos descritos na Tabela 3 serão detalhados ao decorrer desse material.

Além destas funções, você também pode escrever suas próprias funções, que devem ser escritas fora das funções `setup()` e `loop()`. A criação de uma função deve seguir a sintaxe descrita abaixo:

```
1  Tipo nome_da_funcao(declaração de parâmetros)
2  {
3      Declaração de variável; //opcional
4      Comando 1;
5      Comando 1;
6      ...
7  }
```

Há dois tipos de função: As que não retornam nenhum valor e as que retornam algum valor para a função onde está inserida.

A função que não retornam nenhum valor são do tipo `void`. As funções do tipo `void` não retorna nenhum valor para a função que a chamou, ou seja, as ações executadas nessa função não retornam números, variáveis ou caracteres para a função principal.

Por sua vez, as funções que retornam valor podem ser do tipo `int`, `float`, `string`, etc. Uma função do tipo `int`, por exemplo, retorna um valor inteiro para a função que a chamou. Existem duas formas de retorno de uma função, uma delas é quando o finalizador de função (`}`) é encontrado e a outra é usando a declaração `return`.

A declaração `return` termina uma função e retorna um valor desejado.



OBSERVAÇÕES:

- Se a função retorna um valor é obrigatório que seja determinado o tipo de retorno, que pode ser um número inteiro, um caractere ou um número real;
- As variáveis declaradas no parâmetro são variáveis de entrada, cujos tipos também devem ser especificados;
- Barra dupla (//) pode ser utilizada para fazer um breve comentário em alguma linha do código. A função do comentário é deixar o código claro tanto para o programador quanto para outras pessoas. Os comentários são ignorados pelo compilador do código;
- Também é possível comentar várias linhas do código, para isso você deve incluir os comandos /* na linha de início e */ ao final da linha que finaliza o trecho a ser comentado.

Bibliotecas

As bibliotecas são um conjunto de instruções desenvolvidas para executar tarefas específicas relacionadas a um determinado dispositivo. O uso de bibliotecas facilita a conexão a sensores, a uma tela, a um módulo, etc., além de poupar tempo do programador.

Algumas bibliotecas já vêm instaladas com o Arduino IDE, outras podem ser incluídas a partir de download e você também pode criar a sua própria. Muitas vezes o fabricante de um sensor, módulos, atuador, etc. fornece uma biblioteca para facilitar a programação.

Uma biblioteca padrão é chamada através da seguinte instrução:

```
1 #include <nome_da_biblioteca>
```

Por sua vez, uma biblioteca criada pelo usuário segue a sintaxe:

```
1 #include "nome_da_biblioteca.h"
```




OBSERVAÇÕES:

- Usamos a diretiva `#include` quando desejamos incluir uma biblioteca externa ao nosso Sketch. Com isso, teremos acesso a um grande número de bibliotecas escritas especialmente para a linguagem Arduino;
- Outra diretiva que será utilizada em nossos projetos é a `#define`, que permite dar um nome a um valor constante antes de o programa ser compilado. Uma vantagem da utilização desta diretiva é que as variáveis definidas a partir dela não ocupam espaço na memória de programa do chip;
- Instruções com `#include` e `#define` não são terminadas com ; (ponto e vírgula).

OPERADORES E ESTRUTURA DE CONTROLE DE FLUXO

Operadores aritméticos

Os operadores aritméticos são as representações que utilizamos para realizar as operações aritméticas básicas, como somar, subtrair, dividir, multiplicar, entre outras.

Tabela 4 - Operadores aritméticos.

Operador	Nome	Sintaxe	Resultado
+	Adição	$x = y + z$	x é igual a soma de y mais z
-	Subtração	$x = y - z$	x é igual a subtração de y menos z
*	Multiplicação	$x = y * z$	x é igual a multiplicação de y vezes z
/	Divisão	$x = y / z$	x é igual a divisão de y por z
%	Resto da divisão	$x = y \% z$	x é igual ao resto da divisão de y por z

Operadores de comparação e booleanos

Para programas corretamente a placa microcontroladora UNO é necessário aprender a usar de forma adequada os operadores de comparação e booleanos.

Os operadores de comparação, como o próprio nome diz, compara dois valores retornando verdadeiro (true) ou falso (false). Observe na Tabela 5 a seguir os operadores de comparação.

Tabela 5 - Operadores de comparação.

Operador	Nome	Sintaxe	Resultado
==	Igual	x == y	Retorna true (verdadeiro) se x for igual a y
!=	Diferente	x != y	Retorna true (verdadeiro) se x for diferente de y
<	Menor que	x < y	Retorna true (verdadeiro) se x for menor que y
>	Maior que	x > y	Retorna true (verdadeiro) se x for maior que y
<=	Menor ou igual a	x <= y	Retorna true (verdadeiro) se x for menor ou igual a y
>=	Maior ou igual a	x >= y	Retorna true (verdadeiro) se x for maior ou igual a y

Os operadores booleanos são utilizados para testes lógicos entre elementos em um teste condicional. Assim como os operadores de comparação, os operadores booleanos também retornam verdadeiro (true) e falso (false) conforme o resultado dos testes. Os operadores booleanos são:

Tabela 6 - Operadores booleanos.

Operador	Nome
&&	E lógico
	OU lógico
!	NÃO lógico

Operador E lógico

O E lógico resulta em verdadeiro apenas se **ambos** os operandos forem verdadeiros. Representamos o E lógico com &&.

Exemplo: Se quisermos verificar se um determinado valor de temperatura se encontra entre uma faixa de valores (entre 30 e 50°C), podemos utilizar:

```
Se temperatura >=30 && temperatura <=50
```

Se o valor encontrado de temperatura for maior ou igual a 30 e menor ou igual a 50 a condição será satisfeita, retornando verdadeiro (true).

Operador OU lógico

O OU lógico resulta em verdadeiro se **pelo menos um** dos operadores for verdadeiro. Representamos o OU lógico com ||.

Exemplo: Se quisermos verificar se o valor de temperatura é igual a pelo menos um valor determinado (30°C ou 50°C), podemos utilizar:

```
Se temperatura == 30 || temperatura == 50
```

Neste caso, se o valor encontrado de temperatura for igual a 30 **ou** igual a 50 a condição será satisfeita, retornando verdadeiro (true).

Operador NÃO lógico

O NÃO lógico resulta em verdadeiro se o operando for falso. Representamos o NÃO lógico com !.

Exemplo: Se quisermos mudar o estado de uma variável x de verdadeiro para falso e vice-versa, podemos utilizar:

```
!x
```

Neste caso, se x for verdadeiro (true) o uso do NÃO lógico transformará seu estado para falso (false). De mesmo modo, se x for falso (false) o NÃO lógico transformará seu estado para verdadeiro (true).

Operadores de incremento e decremento

Os operadores de incremento e decremento são operadores unitários utilizados com a finalidade de adicionar ou subtrair em uma unidade ao valor de uma variável. O operador de incremento é representado por ++ e o de decremento por --.

Os operadores de incremento e decremento podem ser pré-fixos ou pós-fixos dependendo de serem posicionados antes ou depois do nome da variável a ser implementada ou decrementada.

Tabela 7 - Operadores de incremento e decremento.

Tipo	Operador	Significado
Prefixo	++x	Incrementa x em um e retorna o novo valor de x
	--x	Decrementa x em um e retorna o novo valor de x
Pós-fixos	x++	Incrementa x em um e retorna o valor antigo de x
	x--	Decrementa x em um e retorna o valor antigo de x

Exemplo:

```
1  x=2;
2  y=++x; //x agora contém 3 e y também
3  y=x++; //x contém 4, mas y ainda contém 3
```

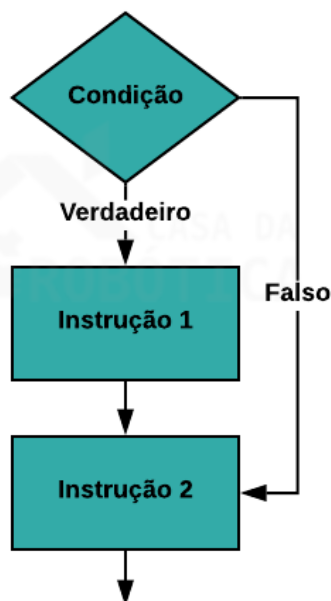
Estrutura de controle de fluxo

As estruturas de controle são blocos de programação que analisam variáveis e decidem como o programa deve se comportar com base em parâmetros pré-definidos. Em outras palavras, a estrutura de controle de fluxo determina como o programa responderá diante de certas condições ou parâmetros.

A Figura 38 ilustra um exemplo de estrutura de controle, em que dependendo do resultado obtido na condicional (verdadeiro ou falso) o programa executará diferentes instruções.

Neste caso, o resultado da condicional for verdadeiro o programa executará a Instrução 1 e, em sequência, a instrução 2. No entanto, se o resultado da condicional for falso apenas a Instrução 2 será executada pelo programa.

Figura 38 - Estrutura de controle de fluxo.



IF

O comando if (se) é uma estrutura de controle de fluxo de seleção. Usamos esse comando para checar uma condição. Se a condição for satisfeita (verdadeiro) uma lista de instrução delimitada por {} (chaves) serão executadas. No entanto, se a condição não for satisfeita (falso) esta lista de instruções não será executada e o programa seguirá a sequência de comandos escritos depois do if.

A sintaxe do comando if na programação Arduino é:

```
1  if (condição){
2      Comando 1;
3      Comando 2;
4      ...
5  }
```

IF...ELSE

A combinação dos comandos if...else permite maior controle sobre o fluxo de código que o comando if, por possibilitar que múltiplos testes sejam agrupados. O

comando else (senão) será executado se a condição do comando if (se) resultar em falso.

A sintaxe dos comandos if...else na programação Arduino é:

```
1  if (condição1){
2      Comando 1;
3  }
4  else{
5      Comando 2;
6  }
```

Dentro do comando else podemos adicionar outro comando if, de modo que múltiplos testes podem ser executados ao mesmo tempo. Desta forma, a sintaxe poderá ser a seguinte:

```
1  if (condição1){
2      Comando 1;
3  }
4  else if (condição2){
5      Comando 2;
6  }
7  else{
8      Comando 3;
9  }
```

FOR

O comando for (para) é uma estrutura de controle de fluxo de repetição. Este comando permite que certo trecho do código seja executado um determinado número de vezes. O comando for é útil para qualquer operação repetitiva e é usado frequentemente com vetores para operar em coleções de dados.

A sintaxe do comando for é a seguinte:

```
1  for (inicialização; condição; incremento){
2      Comando 1;
3      ...
4  }
```


A inicialização ocorre primeiro e apenas uma vez. A cada repetição do loop, a condição é testada, se verdadeira o bloco de comandos e o incremento são executados. Por sua vez, quando a condição for falsa o loop termina.

SWITCH...CASE

Da mesma forma que o comando if, o comando switch...case (selecione...caso) é uma estrutura de controle de fluxo de seleção. Este comando permite especificar códigos diferentes para serem executados em várias condições, funcionando da seguinte maneira: um comando switch compara o valor de uma variável aos valores especificados nos comandos case. Quando um comando case é encontrado, cujo valor é igual ao da variável, o código para esse comando case é executado.

A sintaxe do comando switch...case é a seguinte:

```
1  switch (var){
2      case valor1:
3          comando1;
4          break;
5      case valor2:
6          comando2;
7          break;
8      default:
9          comando3;
10         break;
11 }
```

A variável `var` será comparada ao valor dos vários cases, podendo ser do tipo `int` ou `char`. Os parâmetros `valor1` e `valor2` são constantes do tipo `int` ou `char`.



OBSERVAÇÕES:

- A palavra-chave `break` é utilizada para interromper o comando switch, devendo ser escrito ao final de cada case. Sem o comando `break` o comando switch continuará executando as expressões seguintes, até encontrar um `break`.

- A palavra-chave default é opcional e é executado quando a variável de comparação do switch não corresponde a nenhum valor constate dos cases.

WHILE

O while (enquanto) é uma estrutura de controle de fluxo de repetição. As instruções contidas em um while serão repetidas continuamente, e infinitamente, até que a sua condicional se torne falsa. Em outras palavras, as instruções contidas no código while serão executadas enquanto a condição for satisfeita (verdadeiro)

A forma geral do while é:

```
1 while (condição){  
2     Comando 1;  
3     ...  
4 }
```

DO...WHILE

A estrutura do...while (faça...enquanto) funciona de forma semelhante às estruturas while e for, com exceção de a condição ser testada ao final do loop, de tal modo que as instruções contidas no do...while serão executadas pelo menos uma vez.

A sintaxe do comando do...while é a seguinte:

```
1 do{  
2     Comando 1;  
3 } while (condição);
```

COMO PROGRAMAR A PLACA UNO

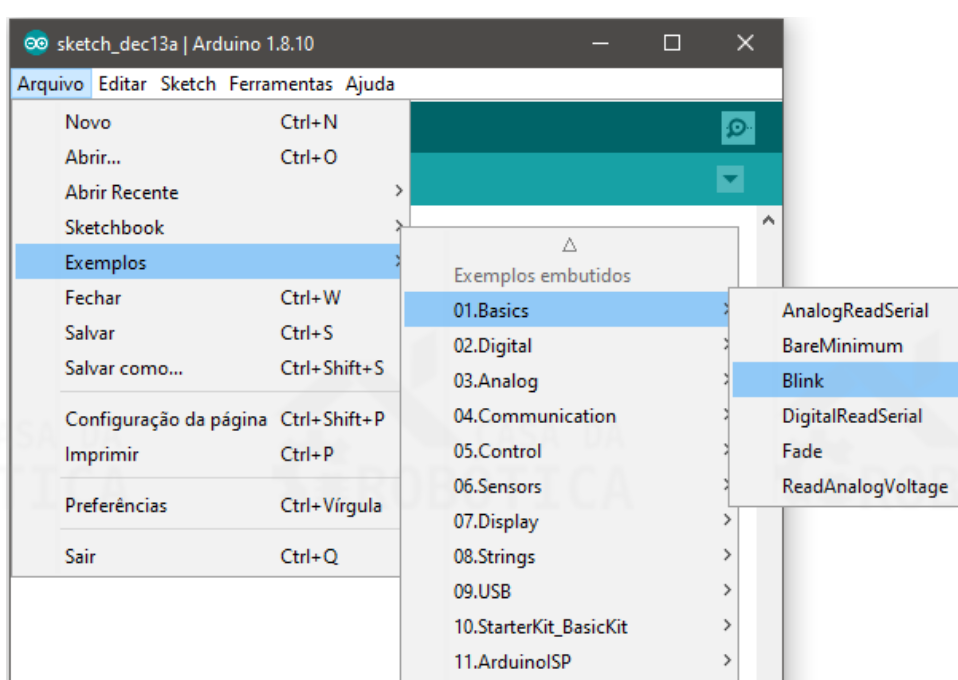
Agora que já aprendemos alguns elementos básicos que compõem um Sketch, os operadores e as estruturas de controle de fluxo, podemos começar a programar em Arduino.

PROJETO BLINK – PISCA LED INTERNO DA PLACA UNO

O exemplo mais básico e clássico para iniciar a programação do Arduino e placas compatíveis é o Blink (ou Pisca Led), que consiste em acionar um LED por meio de um sinal digital. A placa UNO conta com um LED conectado ao pino Digital 13 que pode ser utilizado para este teste. Desta forma, não há a necessidade de componentes adicionais.

Este e outros exemplos básicos encontram-se disponíveis no próprio Arduino IDE e pode ser acessado através do menu Arquivos ao clicar em Exemplos, conforme mostrado na Figura 39. O Blink pode ser acessado através do caminho: Arquivo > Exemplos > 01. Basics > Blink.

Figura 39 - Caminho de acesso ao exemplo Blink.

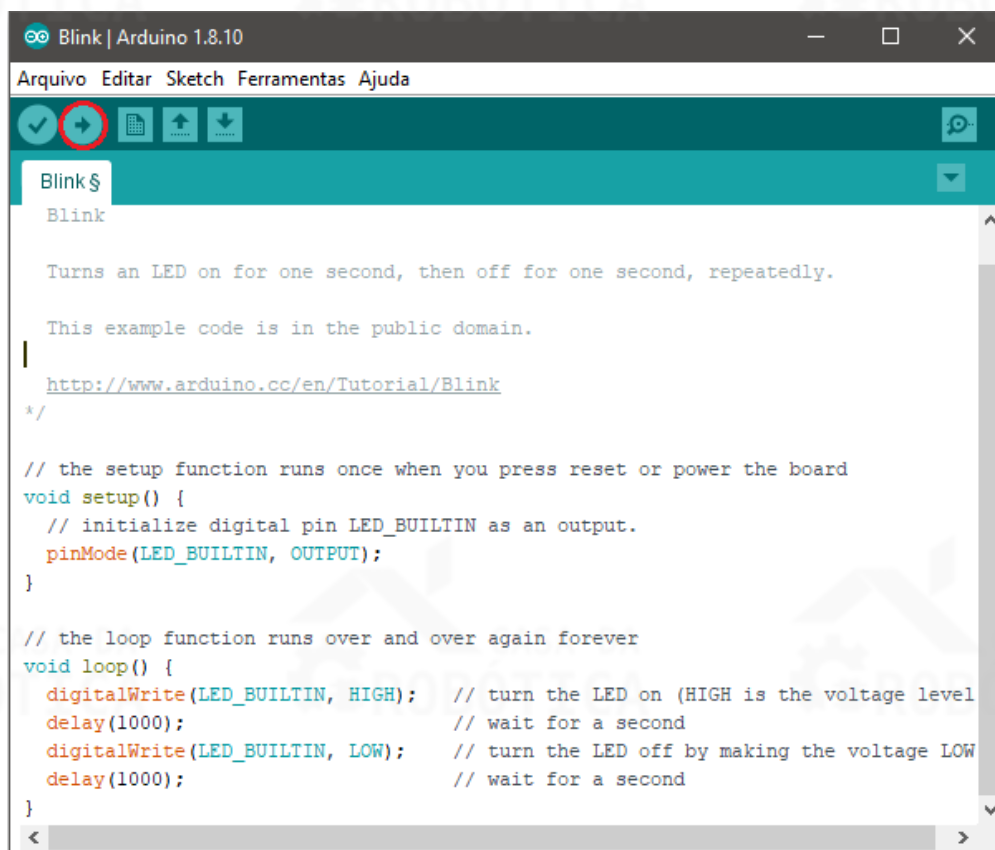


Ao selecionar o Sketch Blink uma nova janela será aberta contendo o seguinte código:

```
void setup() {  
  // initialize digital pin LED_BUILTIN as an output.  
  pinMode(LED_BUILTIN, OUTPUT);  
}  
  
// the loop function runs over and over again forever  
void loop() {  
  digitalWrite(LED_BUILTIN, HIGH); // turn the LED on (HIGH is the voltage  
  level)  
  delay(1000); // wait for a second  
  digitalWrite(LED_BUILTIN, LOW); // turn the LED off by making the voltage  
  LOW  
  delay(1000); // wait for a second  
}
```

Para carregar o código na placa UNO é necessário configurar a placa e a porta de comunicação, conforme Figura 19 e Figura 20. Em seguida, basta clicar no ícone Upload, como pode ser observado na Figura 40.

Figura 40 - Realizando upload do código Blink.



A transferência do código demorará alguns segundos, mas, logo em seguida, o LED ligado ao pino 13 começará a piscar em intervalos de 1 segundo.

ENTENDENDO O CÓDIGO

Apesar de simples, o código Blink nos ajudará a compreender sobre a estrutura básica de um programa sequencial desenvolvido no Arduino IDE e como escrever na porta digital da placa UNO.

Conforme vimos anteriormente, a estrutura do código feito no Arduino IDE é composta por duas funções obrigatórias, que são `setup()` e `loop()`, sem elas o Sketch não funcionará.

No Sketch Blink, a função `setup()` inicializa a configuração do programa. Para isso, faz uso da função `pinMode()`, responsável por configurar o modo como um pino especificado irá funcionar, podendo ser como saída ou entrada. No exemplo, o LED embutido na porta Digital 13 (`LED_BUILTIN`) está configurado como porta de saída (`OUTPUT`).

A função `loop()` é a função principal do programa e é executada continuamente enquanto a placa estiver ligada. No Sketch Blink desejamos que o LED acenda, permaneça aceso por um segundo, apague, fique apagado por um segundo e repita continuamente o processo. Desta forma, estas informações deverão ser escritas dentro da função `loop()`.

A primeira instrução do `loop()` do Sketch Blink deve comandar a placa UNO a acender o LED embutido na porta Digital 13 (`LED_BUILTIN`). Para isso, utilizaremos a função `digitalWrite(LED_BUILTIN, HIGH)`, que escreve um valor `HIGH` para a porta Digital 13. Definir um pino como `HIGH` significa que estamos colocando o pino em nível lógico 1, enviando 5 V para que o LED seja ligado. Ao contrário, quando definimos um pino como `LOW` significa que estamos colocando o pino em nível lógico 0, enviando 0 V ou conectado ao terra.

A próxima instrução escrita foi a função `delay(1000)`. Esse comando diz ao microcontrolador para esperar um intervalo 1000 milissegundos, equivalente a 1 segundo, antes de executar a instrução seguinte.

Em seguida, a função `digitalWrite(LED_BUILTIN, LOW)` é utilizada para apagar o LED embutido na porta Digital 13. Então, outra instrução de espera por mais

1000 milissegundos é enviada, finalizando a função `loop()`. No entanto, como esta é a função principal, o programa reiniciará e a executará repetidamente.



OBSERVAÇÕES:

- Configuraremos como saída (OUTPUT) todos os dispositivos que desejamos controlar, como: LEDs, buzzer, motores, displays, relés, entre outros;
- Configuraremos como entrada (INPUT) todos os dispositivos que desejamos receber dados, como: LDR, botões, sensores infravermelhos, sensores ultrassônicos, termistores, reed switch, entre outros.

PROJETO BLINK – PISCA LED EXTERNO

Neste projeto, vamos repetir o projeto anterior. No entanto, desta vez, usaremos componentes externos: conectaremos um LED a um dos pinos digitais ao invés de utilizar o LED embutido na porta Digital 13. Neste momento, aprenderemos um pouco sobre eletrônica e codificação na linguagem do Arduino.

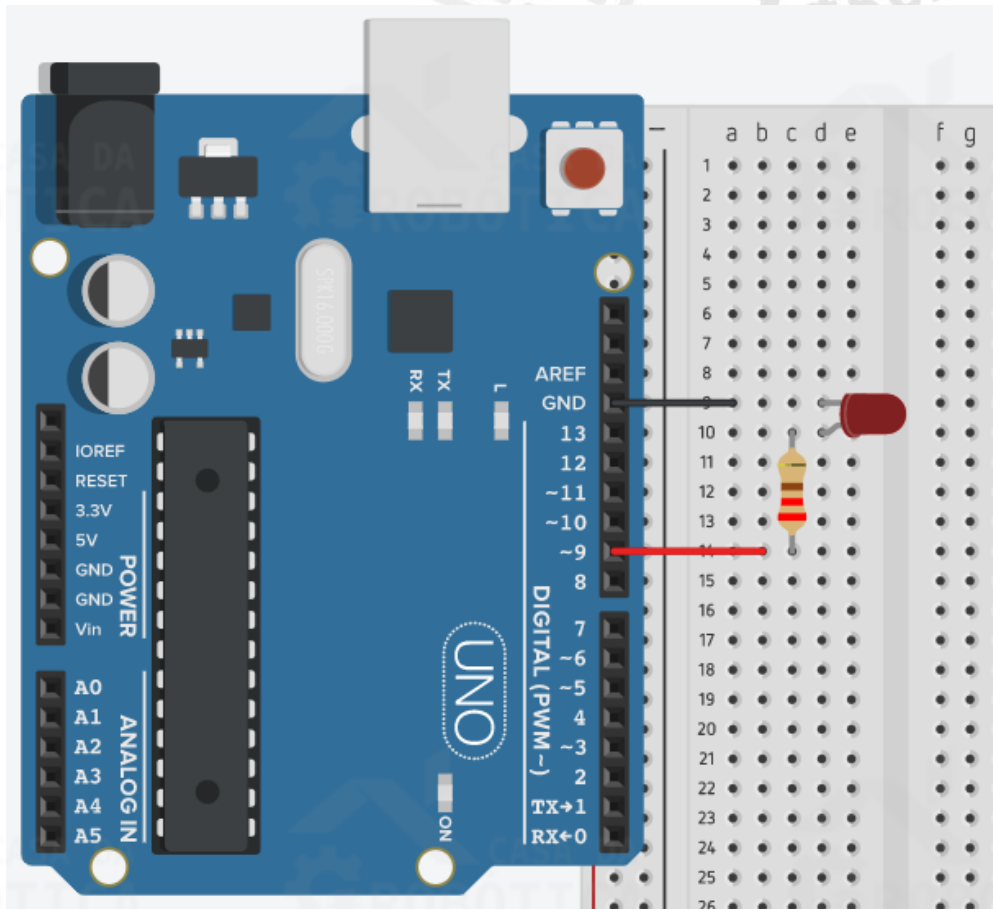
MATERIAIS NECESSÁRIOS

- 1 x Placa UNO SMD R3 Atmega328 compatível com Arduino UNO;
- 1 x Cabo USB;
- 1 x Protoboard;
- 1 x Resistor de 220 Ω ;
- 1 x LED difuso;
- Fios de jumper macho-macho.

ESQUEMÁTICO DE LIGAÇÃO DOS COMPONENTES

Antes de iniciar a montagem elétrica do circuito, certifique-se que a sua placa UNO esteja desligada. Em seguida, monte o circuito da Figura 41 utilizando a protoboard, o LED, o resistor e os fios.

Figura 41 - Circuito para o projeto Piscar o LED.



Ao montar seu circuito na protoboard preste atenção nos seguintes pontos:

- Você pode utilizar fios de cores diferentes ou furos diferentes na protoboard, mas deve assegurar que os componentes e fios estejam conectados na mesma ordem da Figura 41;
- O terminal mais longo do LED encontra-se conectado ao pino Digital 9. Este terminal longo é o ânodo (positivo) do LED e deve ser conectado na alimentação de 5V, neste caso representado pelo pino Digital 9. O terminal mais curto é o cátodo (negativo) e deve ser conectado ao terra (GND);
- Em nosso projeto, utilizaremos o resistor de $220\ \Omega$ para reduzir a tensão e a corrente de alimentação do LED. O LED será acionado por uma porta digital da placa UNO que emite 5V de tensão e 40 mA de corrente contínua. No entanto, o LED difuso vermelho necessita de uma tensão de 2V e uma corrente máxima de 35 mA. Portanto, utilizaremos o resistor para reduzir os 5V para 2V, e a corrente de 40 mA para uma corrente inferior a 35 mA.



OBSERVAÇÕES:

- Em nosso exemplo, utilizaremos um resistor de 220 Ω . Para saber a corrente no circuito utilizaremos a seguinte equação: $I = (V_S - V_L)/R$, em que V_S é a tensão fornecida, V_L é a tensão requerida pelo LED, I é a corrente no circuito e R é o valor da resistência. Aplicando esta equação ao nosso projeto temos: $I = (5 - 2)/220 = 13,64$ mA. Desta forma, o resistor de 220 Ω atende as especificações de corrente do LED e da placa UNO.
- Você também pode utilizar outro valor de resistor desde que as especificações de corrente sejam satisfeitas. Lembrando que a corrente no LED e na placa UNO deve ser sempre inferior a corrente máxima especificada pelo fabricante.

Assim que você tiver certeza de que tudo foi conectado corretamente, ligue sua placa UNO ao computador.

ELABORANDO O CÓDIGO

Após a montagem do circuito elétrico, realize as configurações da placa e da porta de comunicação da placa UNO.

Tal qual o projeto anterior, a proposta deste projeto é comandar a placa UNO para acender um LED por 1 segundo e, em seguida, apagá-lo por 1 segundo. Desta forma, vamos utilizar a mesma lógica de programação, conforme o código a seguir:

```
// Projeto - Piscar LED
int ledPin = 9; // Atribui o valor 9 a variável inteira ledPin, que irá
representar o pino digital 9

void setup() {
    pinMode(ledPin, OUTPUT); // Define ledPin (pino 9) como saída
}

void loop() {
    digitalWrite(ledPin, HIGH); // Coloca ledPin em nível alto (5V)
    delay(1000); // Espera 1000 milissegundos (1 segundo)
    digitalWrite(ledPin, LOW); // Coloca ledPin em nível baixo (0V)
    delay(1000); // Espera 1000 milissegundos (1 segundo)
}
```

Você pode simplesmente copiar este código no Arduino IDE, mas vai ser muito mais proveitoso se você montar o seu próprio. Ao elaborar o código observe os seguintes pontos:

- A primeira linha do código `// Projeto - Piscar LED` trata-se apenas de um comentário que será ignorado pelo compilador;
- A instrução `int ledPin = 9;` atribui o valor 9 a variável inteira `ledPin`, que será utilizada para representar a porta digital 9. As variáveis são utilizadas para armazenar dados. Em nosso exemplo, a variável é do tipo `int`, ou inteiro. A vantagem da utilização da variável é que se você decidir utilizar outro pino, não será necessário alterar o código em vários locais, basta alterar o valor da variável;
- A variável deve ser declarada antes da função `void setup()`;
- No `loop()`, por meio da função `digitalWrite(ledPin, HIGH)` colocar o pino 9 em nível alto (5V), acendendo o LED. Em seguida, damos um intervalo de 1 segundo através da função `delay(1000)` ;.
- Para apagar o LED novamente usamos a função `digitalWrite(ledPin, LOW)`, colocando o pino 9 em nível baixo (0V). Logo após, adicionamos um delay de 1 segundo com a função `delay(1000)` ;.

Com o código escrito no Arduino IDE pressione o botão Verificar para certificar-se de que não há erros. Se não houver erros, clique no botão Upload para transferir o código para a placa UNO. Caso tudo tenha sido feito corretamente, o LED vermelho se acenderá e apagará em intervalo de 1 segundo.

TINKERCAD

Este projeto encontra-se disponível no Tinkercad, ferramenta online gratuita de design de modelos 3D e de simulação de circuitos elétricos, desenvolvida pela Autodesk.

Para melhor visualizar e/ou simular o circuito e a programação deste projeto basta se acessar o seguinte link: www.blogdarobotica.com/tinkercad-Blink .

PROJETO LIGAR E DESLIGAR LED COM BOTÃO PUSH BUTTON

A proposta desse projeto é ligar e desligar um LED com um botão do tipo push button. Neste projeto vamos aprender como ler uma porta digital da placa UNO e forma de funcionamento do botão push button.

Além disso, esse projeto também visa colocar em prática o uso da estrutura de repetição *if...else* (Se/senão), que torna possível múltiplos testes agrupados. Uma instrução escrita no comando *else* será executada se a condição do comando *if* for falsa.

MATERIAIS NECESÁRIOS

- 1 x Placa UNO SMD R3 Atmega328 compatível com Arduino UNO;
- 1 x Cabo USB;
- 1 x Protoboard;
- 1 x Resistor de 220 Ω ;
- 1 x Resistor de 10 k Ω ;
- 1 x Botão tipo push button;
- 1 x LED difuso;
- Fios de jumper macho-macho.

ESQUEMÁTICO DE LIGAÇÃO DOS COMPONENTES

Inicialmente, certifique-se que a sua placa UNO esteja desligada. Em seguida, monte o circuito da Figura 42 utilizando o LED, os resistores e o botão.

com botão push button.

- O

- O qu

Apó
corretas

ELABORANDO O CÓDIGO

Após a montagem do circuito, vamos a programação do Sketch. Conforme, mencionado anteriormente, o objetivo deste projeto é ligar e desligar um LED com um botão push button. Desta forma, quando o botão for pressionado o LED deverá acender e retornará ao estado desligado quando o botão deixar de ser pressionado.

Vamos entender a lógica de programação deste projeto a partir dos seguintes passos:

1- Declarar as variáveis:

O primeiro passo na construção do Sketch será a declaração de variáveis. Desta forma, definimos a porta digital 7, em que o botão está conectado, a variável `buttonPin`, definimos a porta digital 10, em que o LED está conectado, a variável `ledPin`, e criamos a variável `estadoButton`, do tipo inteiro, para armazenar o estado (HIGH ou LOW) do botão.

2- Configurar as portas de saída e entrada:

Na função `setup()` configuraremos as portas de entrada e saída da placa UNO. A porta 10 (`ledPin`) deve ser configurada como saída e a porta 7 (`buttonPin`) deve ser configurada como entrada.

3- Realizar a leitura da porta digital:

Na função `loop()` escreveremos todos os comandos e operações para ligar e desligar o LED com o botão. Iniciamos o `loop()` realizando a leitura da porta digital 7 (`buttonPin`), para isso utilizaremos a função `digitalRead(buttonPin)`, e armazenaremos este valor na variável `estadoButton`.

4- Realizar a comparação

Utilizaremos a lógica do `if...else` para comparar se a variável `estadoButton` encontra-se em nível alto ou baixo (chave pressionada ou chave não pressionada).

Se a variável `estadoButton` estiver em nível lógico alto (chave pressionada) o LED será ligado através do comando `digitalWrite(ledPin, HIGH);`. Senão, o LED deve ser desligado por meio da instrução `digitalWrite(ledPin, LOW);`.

Desta forma, o Sketch deste projeto ficará da seguinte maneira:


```

int buttonPin = 7; //Define buttonPin no pino digital 7
int ledPin = 10; //Define ledPin no pino digital 10
int estadoButton = 0; //Variável responsável por armazenar o estado do botão
(ligado/desligado)

void setup() {
  pinMode(ledPin, OUTPUT); //Define ledPin (pino 10) como saída
  pinMode(buttonPin, INPUT); //Define buttonPin (pino 7) como entrada
}

void loop() {
  estadoButton = digitalRead(buttonPin); //Lê o valor de buttonPin e
  armazena em estadoButton
  if (estadoButton == HIGH) { //Se estadoButton for igual a HIGH ou 1
    digitalWrite(ledPin, HIGH); //Define ledPin como HIGH, ligando o LED
    delay(100); //Intervalo de 100 milissegundos
  }
  else { //Senão = estadoButton for igual a LOW ou 0
    digitalWrite(ledPin, LOW); //Define ledPin como LOW, desligando o LED
  }
}

```

TINKERCAD

Para melhor visualizar e/ou simular o circuito e a programação deste projeto basta se acessar o seguinte link: <http://blogdarobotica.com/tinkercad-ledbotao>.

EXTRA

Você também pode fazer este mesmo projeto substituindo o LED por um buzzer. Disponibilizamos esse projeto para você visualizar e simular no Tinkercad Para isso, acesse o seguinte link: www.blogdarobotica.com/tinkercad-buzzerbotao.

PROJETO INTERRUPTOR COM BOTÃO PUSH BUTTON

No projeto anterior aprendemos como ligar ou desligar um LED com um botão do tipo push button. Você deve ter percebido que o LED só permanece acionado enquanto mantemos o botão pressionado. Neste projeto, utilizaremos o botão push

button como um interruptor, de modo que uma vez pressionado o LED continue ligado até que ele seja pressionado novamente.

Neste projeto, utilizaremos novamente a estrutura de repetição `if...else` (Se/senão) e o operador booleano `NÃO` lógico.

MATERIAIS NECESÁRIOS E ESQUEMÁTICO DE LIGAÇÃO DOS COMPONENTES

Para construção deste projeto vamos utilizar os mesmos componentes e esquemático de ligação dos componentes do exemplo anterior (Figura 42).

ELABORANDO O CÓDIGO

Com o circuito montado, vamos a programação do nosso Sketch. O intuito deste projeto é utilizar o botão push button como um interruptor para acionar ou desligar um LED. Vamos entender a lógica de programação deste projeto com os seguintes pontos:

1- Declarar as variáveis

Para este projeto precisamos declarar 4 variáveis. Desta forma, atribuímos a porta digital 7, em que o botão está conectado, a variável `buttonPin`. Definimos a porta digital 10, em que o LED está conectado, a variável `ledPin`. Criamos a variável `estadoButton`, do tipo `int`, para armazenar o estado (HIGH ou LOW) do botão. Por fim, criamos a variável `estadoLed`, do tipo `bool`, para armazenar o estado do LED, inicializando-a como `false`. Uma variável do tipo `bool` é utilizada quando precisamos armazenar dois valores `true` (verdadeiro) ou `false` (falso).

2- Configurar o pino de entrada

A porta 10 (`ledPin`) deve ser configurada como saída e a porta 7 (`buttonPin`) deve ser configurada como entrada.

3- Realizar a leitura da porta digital:

Iniciamos o `loop()` realizando a leitura da porta digital 7 (`buttonPin`), para isso utilizaremos a função `digitalRead(buttonPin)`, e armazenaremos este valor na variável `estadoButton`.

4- Realizar comparações

Neste projeto utilizaremos a estrutura de repetição `if...else` realizando múltiplos testes agrupados.

Inicialmente, precisamos comparar se a variável `estadoButton` encontra-se em nível alto (chave pressionada). Uma vez pressionado o botão, a variável `estadoButton` muda seu estado para `HIGH`, a condição do `if` será satisfeita e o estado da variável `estadoLed` deverá ser invertida através da instrução: `estadoLed = !estadoLed;`, ou seja, passará de `false` para `true` ou de `true` para `false`.

Em seguida, iremos comparar se `estadoLed` é igual a `true`. Se esta condição for satisfeita o LED deve ser ligado, através do comando `digitalWrite(ledPin, HIGH);`. Senão, o LED deve ser desligado por meio da instrução `digitalWrite(ledPin, LOW);`.

Desta forma, o Sketch deste projeto ficará da seguinte maneira:

```
int buttonPin = 7; //Define buttonPin no pino digital 7
int ledPin = 10; //Define ledPin no pino digital 10
int estadoButton = 0; //Variável responsável por armazenar o estado do botão (ligado/desligado)
bool estadoLed = false; //Variável booleana responsável por armazenar o estado do LED (ligado = true/desligado = false)
void setup() {
    pinMode(ledPin, OUTPUT); //Define ledPin (pino 10) como saída
    pinMode(buttonPin, INPUT); //Define buttonPin (pino 7) como entrada
}
void loop() {
    estadoButton = digitalRead(buttonPin); //Lê o valor de buttonPin e armazena em estadoButton
    if(estadoButton == HIGH) { //Se estadoButton for igual a HIGH ou 1
        estadoLed = !estadoLed; //Inverte estadoLed
        delay(500); //Intervalo de 0,5 segundos
    }
    if(estadoLed == true) { //Se estadoLed for igual a true (verdadeiro)
        digitalWrite(ledPin, HIGH); //Define ledPin como HIGH, ligando o LED
    }
    else { //Senão
        digitalWrite(ledPin, LOW); //Define ledPin como LOW, desligando o LED
    }
}
```

TINKERCAD

Para melhor visualizar e/ou simular o circuito e a programação deste projeto basta se acessar o seguinte link: www.blogdarobotica.com/tinkercad-InterruptorPushButton.

PROJETO SENSOR DE LUMINOSIDADE – APRENDENDO USAR O LDR

Neste projeto, vamos aprender como utilizar o sensor de luminosidade. O LDR é um sensor analógico, ou seja, seu sinal de saída assume valores que variam ao longo do tempo e é proporcional à grandeza medida. O LDR tem sua resistência alterada conforme variamos a luminosidade que incide sobre ele.

Por se tratar de um sensor analógico para realizar sua leitura do LDR vamos utilizar a porta de entrada analógica da placa UNO. Desta forma, neste projeto aprenderemos como ler uma porta analógica e como visualizar os sinais recebidos por meio do monitor serial do Arduino IDE.

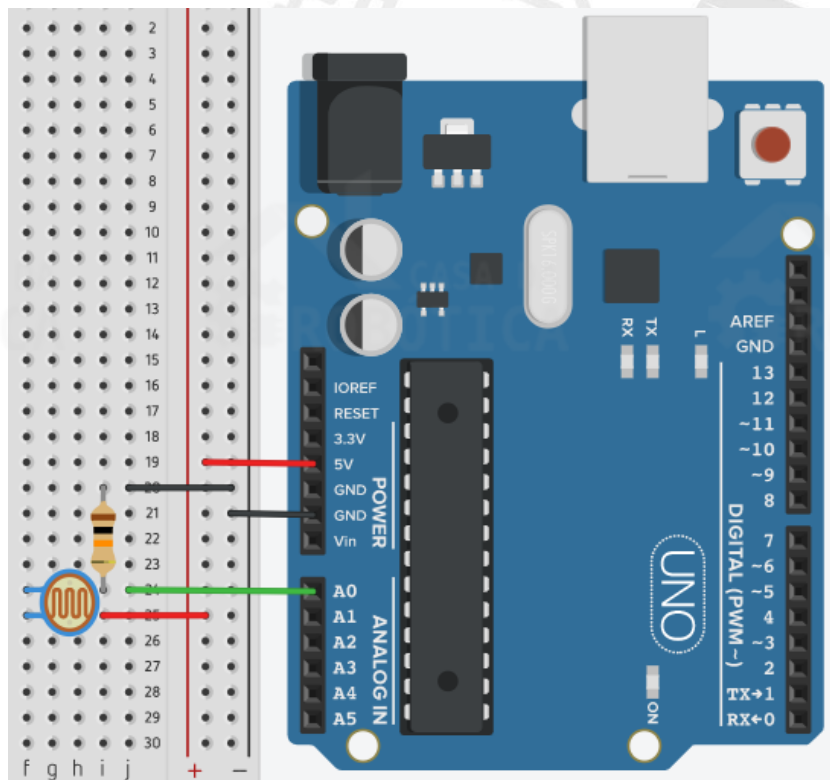
MATERIAIS NECESÁRIOS

- 1 x Placa UNO SMD R3 Atmega328 compatível com Arduino UNO;
- 1 x Cabo USB;
- 1 x Protoboard;
- 1 x Sensor de luminosidade LDR;
- 1 x Resistor de 10 k Ω ;
- Fios de jumper macho-macho.

ESQUEMÁTICO DE LIGAÇÃO DOS COMPONENTES

Antes de iniciar a montagem elétrica do circuito, certifique-se que sua placa esteja desligada. Monte o circuito da utilizando a protoboard, o LDR, o resistor e os jumpers.

Figura 43 - Circuito do projeto sensor de luminosidade – Aprendendo a usar o LDR.



Ao montar o circuito na protoboard observe os seguintes pontos:

- Assim como os resistores comuns, o LDR não possui polaridade e sua resistência é medida em ohms (Ω);
- Um terminal do LDR deve ser conectado ao 5V e o outro ao pino analógico da placa UNO, neste caso usamos o pino A0. Conectamos também uma resistência de 10 k Ω entre o pino A0 e o GND da placa UNO.

ELABORANDO O CÓDIGO

Com o circuito montado, vamos a programação do nosso Sketch. O objetivo deste projeto é ler o sensor de luminosidade LDR. Vamos entender a lógica de programação deste projeto com os seguintes pontos:

1- Declarar as variáveis

Neste projeto, precisamos de duas variáveis, uma de definição do pino em que o LDR está conectado e outra para armazenar o valor lido pelo sensor LDR. Desta forma, definimos o pino A0, em que o LDR está conectado, a variável `ldr` e criamos a variável `valorldr`, do tipo inteiro, para armazenar o valor lido do LDR.

2- Configurar a porta de entrada

Como vamos receber informações (dados) do sensor LDR devemos configurar a porta em que ele está conectado como entrada (INPUT) e fazemos isso através da instrução: `pinMode(ldr, INPUT);`.

3- Iniciar a comunicação serial

Através da comunicação serial é possível obter os dados que a placa UNO está gerando ou recebendo. Para ter acesso a esses dados e visualizá-los na tela do computador precisamos inicializar a comunicação serial por meio da função `Serial.begin(velocidade);`, em que velocidade é a taxa de transferência em bits por segundo.

A placa UNO consegue emitir dados nas seguintes taxas: 300, 1200, 2400, 4800, 9600, 14400, 19200, 28800, 38400, 57600, ou 115200 bits por segundo. Alguns equipamentos apenas receberão dados caso a placa UNO esteja configurada em uma taxa de transmissão específica, como é o caso do computador que é configurado para receber dados da USB numa velocidade de 9600.

4- Realizar leitura da porta analógica

Iniciamos o `loop()` realizando a leitura da porta analógica A0 (`ldr`), para isso utilizaremos a função `analogRead(ldr)`, e armazenaremos este valor na variável `valorldr`.

5- Imprimir dados

Para imprimir os dados de leitura do sensor LDR na porta serial utilizaremos a função `Serial.println(valorldr);`, que imprime os dados e salta para a próxima linha. Além disso, utilizaremos a instrução `Serial.print("Valor lido pelo LDR = ");` para escrever a mensagem "Valor lido pelo LDR = ".

Ao fim, o Sketch deste projeto ficará da seguinte maneira:

```
int ldr = A0; //Atribui A0 a variável ldr
int valorldr = 0; //Declara a variável valorldr como inteiro

void setup() {
    pinMode(ldr, INPUT); //Define ldr (pino analógico A0) como saída
    Serial.begin(9600); //Inicialização da comunicação serial, com taxa de
    transferência em bits por segundo de 9600
}

void loop() {
    valorldr = analogRead(ldr); //Lê o valor do sensor ldr e armazena na
    variável valorldr
```



```
Serial.print("Valor lido pelo LDR = "); //Imprime na serial a mensagem
Valor lido pelo LDR
Serial.println(valorldr); //Imprime na serial os dados de valorldr
}
```

O valor analógico lido pelo sensor LDR pode ser visualizado por meio do Monitor Serial. Para isto, basta clicar no ícone  , que se encontra na Toolbar.

TINKERCAD

Para melhor visualizar e/ou simular o circuito e a programação deste projeto basta se acessar o seguinte link: <http://blogdarobotica.com/tinkercad-LDRleitura>.

LIGAR E DESLIGAR LED UTILIZANDO SENSOR LDR

A proposta deste projeto é utilizar um sensor LDR em conjunto com a placa UNO para ligar e desligar um LED a partir da luminosidade que incide sobre a superfície do sensor, de modo que:

- Quando houver a presença de luminosidade incidindo na superfície do LDR, o LED deverá ser desligado;
- Quando não houver a presença de luminosidade incidindo na superfície do LDR, o LED deverá ser ligado.

O princípio de funcionamento deste projeto é o mesmo utilizado nos postes de iluminação da rua, que utilizam LDR para acionar as luzes da cidade quando anoitece.

MATERIAIS NECESSÁRIOS

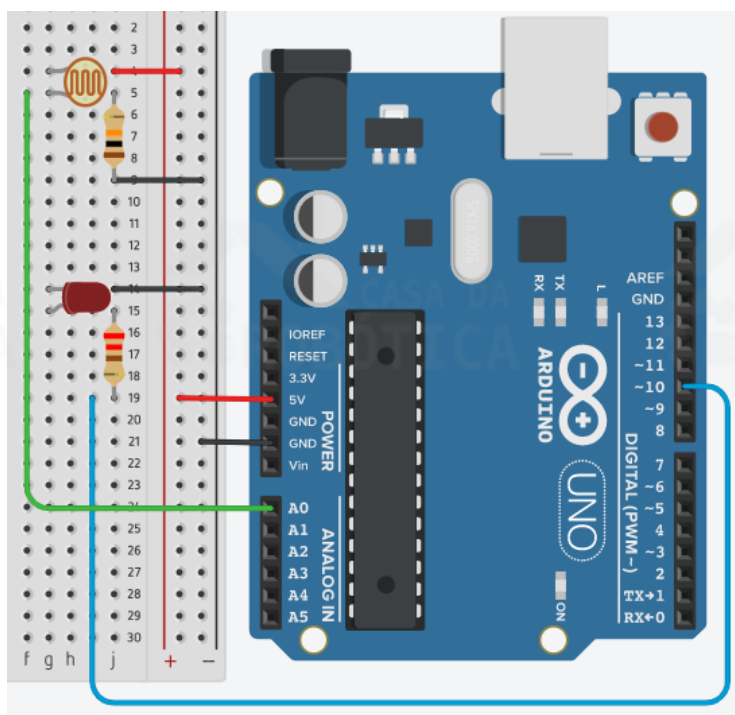
- 1 x Placa UNO SMD R3 Atmega328 compatível com Arduino UNO;
- 1 x Cabo USB;
- 1 x LDR;
- 1 x Protoboard;
- 1 x LED difuso;

- 1 x Resistor de 10 k Ω ;
- 1 x Resistor de 220 Ω ;
- Fios de jumper macho-macho.

ESQUEMÁTICO DE LIGAÇÃO DOS COMPONENTES

Antes de iniciar a montagem elétrica do circuito, certifique-se que a sua placa UNO esteja desligada. Monte o circuito da Figura 44 utilizando a protoboard, o LDR, o LED, os resistores e os fios.

Figura 44 - Circuito para projeto ligar e desligar um LED utilizando sensor LDR.



Ao montar o circuito na protoboard observe os seguintes pontos:

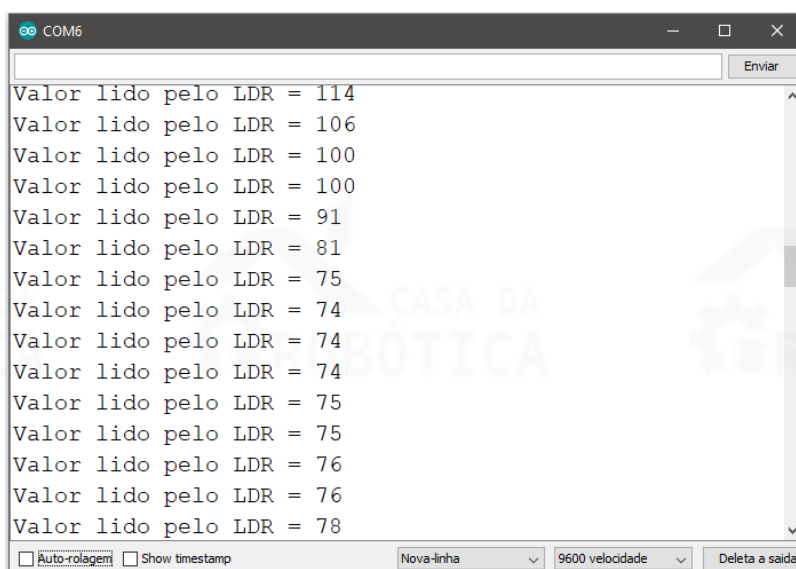
- Um terminal do LDR deve ser conectado ao GND e o outro ao pino analógico da placa UNO, neste caso usamos o pino A0. Conectamos também uma resistência de 10 k Ω entre o pino A0 e o 5V da placa UNO;
- O ânodo do LED encontra-se conectado a um resistor de 220 Ω e a porta digital 10 da placa UNO.

ELABORANDO O CÓDIGO

O objetivo deste projeto é controlar o acionamento de um LED a partir da luminosidade que incide sobre o sensor LDR. Em outras palavras, quando o não houver luminosidade incidindo sobre LDR, o LED deverá ser aceso. Caso contrário, o LED deverá ser apagado.

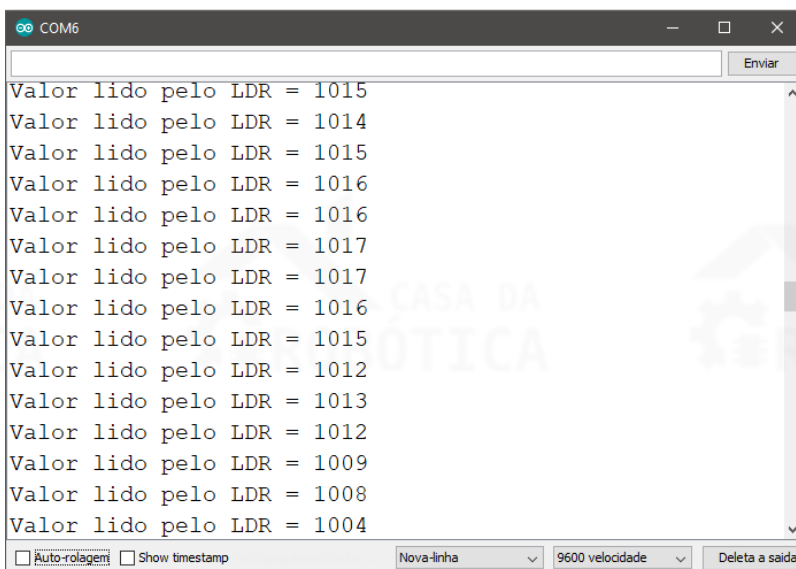
Antes de iniciar o código do exemplo prático proposto precisamos encontrar o valor de referência do LDR. Para isso, utilizaremos o código do projeto sensor de luminosidade – aprendendo usar o LDR. Desta forma, refaça o projeto anterior e observe o resultado no monitor serial para pouca luz e muita luz incidindo no LDR.

Figura 45 - Valor lido pelo LDR ao receber pouca luz.



```
COM6
Valor lido pelo LDR = 114
Valor lido pelo LDR = 106
Valor lido pelo LDR = 100
Valor lido pelo LDR = 100
Valor lido pelo LDR = 91
Valor lido pelo LDR = 81
Valor lido pelo LDR = 75
Valor lido pelo LDR = 74
Valor lido pelo LDR = 74
Valor lido pelo LDR = 74
Valor lido pelo LDR = 75
Valor lido pelo LDR = 75
Valor lido pelo LDR = 76
Valor lido pelo LDR = 76
Valor lido pelo LDR = 78
```

Figura 46 - Valor lido pelo LDR ao receber muita luz.



```
COM6
Valor lido pelo LDR = 1015
Valor lido pelo LDR = 1014
Valor lido pelo LDR = 1015
Valor lido pelo LDR = 1016
Valor lido pelo LDR = 1016
Valor lido pelo LDR = 1017
Valor lido pelo LDR = 1017
Valor lido pelo LDR = 1016
Valor lido pelo LDR = 1015
Valor lido pelo LDR = 1012
Valor lido pelo LDR = 1013
Valor lido pelo LDR = 1012
Valor lido pelo LDR = 1009
Valor lido pelo LDR = 1008
Valor lido pelo LDR = 1004
```

Em função da montagem do LDR na protoboard, podemos verificar que:

- Quanto menor a luminosidade que incide no LDR, mais baixo será o valor lido, conforme ilustra a Figura 45;
- Quanto maior a luminosidade que incide no LDR, mais alto será o valor lido, como pode ser visto na Figura 46.

A partir destas informações, encontramos a grandeza de valores para o LDR quando exposto a muita luz e a pouca luz. Com base nestes valores, podemos definir um parâmetro para acionamento do LED. No nosso exemplo, vamos utilizar 500 como valor de referência. Você pode alterar este valor da forma que achar mais adequado de acordo com os valores lidos no seu projeto, visto que dependendo da luminosidade e do fabricante do LDR estes valores poderão ser bem diferentes.

Feito isso, vamos entender a lógica de programação do exemplo prático proposto a partir dos seguintes passos:

1- Declarar as variáveis:

A variável `led` será utilizada para representar o pino digital 10, a variável `ldr` será utilizada para representar o pino analógico A0 e a variável `valorldr` será utilizada para armazenar das leituras do sensor LDR.

2- Definir os pinos de entrada e saída:

A variável `ldr` será definido como entrada, ou seja, `INPUT` e a variável `led` será definido como saída, ou seja, `OUTPUT`.

3- Iniciar a comunicação serial

Inicializamos a comunicação serial por meio da instrução `Serial.begin(9600);`.

4- Realizar a leitura da porta analógica e imprimir o valor lido

Como no projeto anterior, iniciamos o `loop()` realizando a leitura da porta analógica A0 (`ldr`), para isso utilizaremos a função `analogRead(ldr)`, e armazenaremos este valor na variável `valorldr`. Em seguida, imprimiremos o valor no monitor serial por meio da instrução `Serial.println(valorldr);`.

5- Realizar a comparação

Utilizaremos a lógica do `if...else` para comparar o valor lido pela porta analógica:

- Se o valor lido (`ValorLDR`) no sensor LDR for menor que 500 (valor de referência) então o `led` será ligado, recebendo nível lógico alto (`HIGH`).

Observação: O valor de comparação deverá ser ajustado de acordo com o seu circuito.

- Senão (se o valor lido no sensor LDR não for menor que 500) led receberá nível lógico baixo (**LOW**), sendo desligado ou permanecendo desligado.

Desta forma, o Sketch deste projeto ficará da seguinte maneira:

```
int led = 10; //Atribui a porta digital 10 a variável led
int ldr = A0; //Atribui A0 a variável ldr
int valorldr = 0; //Declara a variável valorldr como inteiro

void setup() {
  pinMode(led, OUTPUT); //Define led (pino digital 10) como saída
  pinMode(ldr, INPUT); //Define ldr (pino analógico A0) como saída
  Serial.begin(9600); //Inicialização da comunicação serial, com velocidade
  de comunicação de 9600
}

void loop() {
  valorldr = analogRead(ldr); //Lê o valor do sensor ldr e armazena na
  variável valorldr
  Serial.println(valorldr); //Imprime na serial os dados de valorldr

  if((valorldr) < 500){ //Se o valor de valorldr for menor que 500:
    digitalWrite(led, HIGH); //Coloca led em alto para acioná-lo
  }

  else{ //Senão:
    digitalWrite(led, LOW); //Coloca led em baixo para que o mesmo desligue
    ou permaneça desligado
  }
}
```



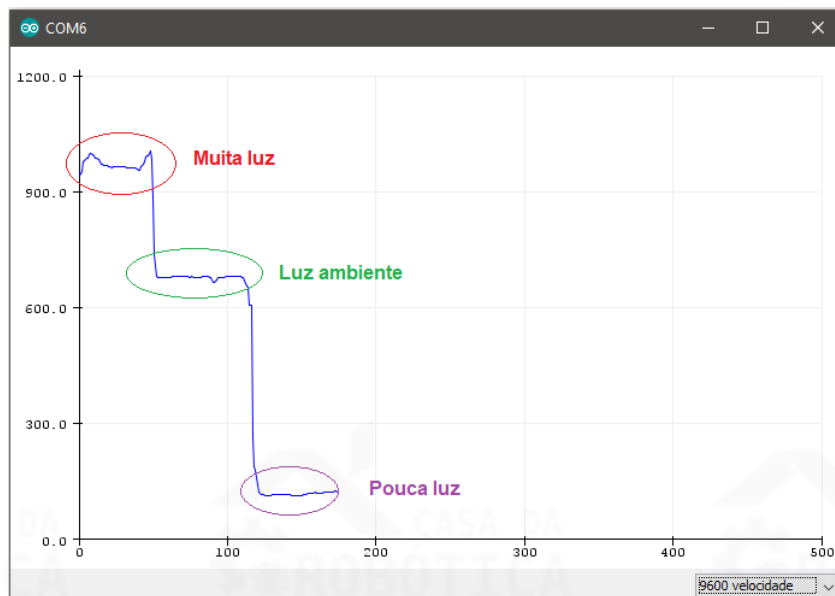
OBSERVAÇÕES:

- Os dados impressos no monitor serial podem ser visualizados em forma de gráfico atualizado em tempo real através da ferramenta Plotter Serial;
- O Plotter Serial é uma ferramenta vem pré-instalada no Arduino IDE (versão 1.6.6 ou superior) que pega os dados seriais recebidos e os exibe em forma de gráfico simples ou múltiplo (duas ou mais variáveis);
- O Plotter Serial está disponível no Arduino IDE por meio do seguinte caminho: Toolbar > Ferramentas > Plotter Serial ou pelo atalho Ctrl + Shift + L;
- A Figura 47 ilustra o gráfico do valor lido pelo LDR plotado no Plotter Serial. O gráfico mostra inicialmente os valores lidos pelo LDR com muita luz incidindo

sobre ele, em seguida com o LDR submetido a luz ambiente e, por fim, com o LDR submetido a pouca luz;

- O eixo Y do gráfico (vertical) se ajusta à medida que o valor dos seus dados seriais aumenta ou diminui, sendo possível plotar dados negativos. O eixo X (horizontal) tem 500 pontos (amostras) e cada dado é plotado conforme o comando `Serial.println()` ou `Serial.print()` é executado.

Figura 47 – Gráfico do valor lido pelo LDR feito pelo Plotter Serial.



TINKERCAD

Para melhor visualizar e/ou simular o circuito e a programação deste projeto basta se acessar o seguinte link: www.blogdarobotica.com/tinkercad-LDR.

PROJETO TOCAR BUZZER 5 VEZES

O propósito deste projeto será utilizar o buzzer ativo para emitir som cinco vezes. Apesar de bastante simples, este projeto nos ensinará sobre o acionamento deste componente em uma determinada frequência e intervalo de tempo. Para isso, aprenderemos como utilizar as funções `tone()` e `noTone()`.

Além disso, esse projeto também visa colocar em prática o uso da estrutura de repetição *for*, utilizada para que certo trecho do código seja executado um determinado número de vezes, cinco neste caso.

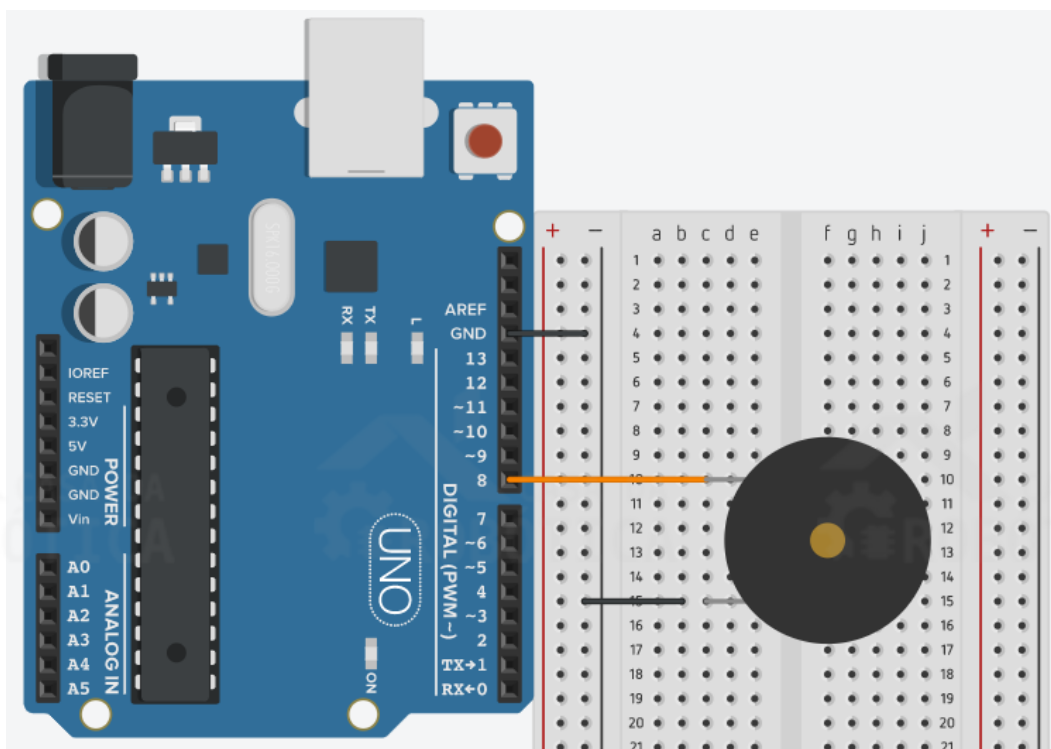
MATERIAIS NECESSÁRIOS

- 1 x Placa UNO SMD R3 Atmega328 compatível com Arduino UNO;
- 1 x Cabo USB;
- 1 x Protoboard;
- 1 x Buzzer ativo;
- Fios de jumper macho-macho.

ESQUEMÁTICO DE LIGAÇÃO DOS COMPONENTES

Antes de iniciar a montagem elétrica do circuito, certifique-se que a sua placa UNO esteja desligada. Monte o circuito da Figura 48 utilizando a protoboard, o buzzer ativo e os fios.

Figura 48 - Circuito para projeto tocar buzzer 5 vezes.



Ao montar seu circuito na protoboard preste atenção nos seguintes pontos:

- Assim como os LEDs, o buzzer possui polaridade. No corpo do componente você encontrará um símbolo “+” que indica a polaridade positiva;
- O terminal positivo do buzzer deve ser conectado à porta digital 8 da placa UNO e o outro terminal deve ser ligado ao GND.

ELABORANDO O CÓDIGO

Após a montagem do circuito, vamos a programação do Sketch. Conforme mencionado anteriormente, o objetivo deste projeto é fazer com o que o buzzer toque cinco vezes.

Vamos entender a lógica de programação deste projeto com os seguintes passos:

1- Declarar a variável

A variável `buzzer` será utilizada para representar o pino digital 8, onde o terminal positivo do buzzer está conectado;

2- Definir o pino de saída

A variável `buzzer` (pino 8) deve ser definida como saída, ou seja, `OUTPUT`;

3- Realizar repetição

Inicializaremos o loop incluindo a estrutura de repetição `for` através da instrução `for (i; i < 5; i++)`. Após o `for`, o primeiro parâmetro a ser incluído nos parênteses é a variável que será utilizada como contadores. O outro parâmetro é uma condição, que deve ser satisfeita para que as instruções do laço `for` sejam executadas. Neste projeto, as instruções do `for` serão realizadas enquanto `i` for menor que 5. Por fim, deve ser incluído o incremento.

4- Acionar o buzzer

Utilizaremos a função `tone()` para gerar uma onda quadrada na frequência de 1500 Hz no pino digital 8 da placa UNO e damos um intervalo de 500 milissegundos; Sintaxe da função `tone`: `tone(pino, frequência);`

Para interromper a geração da onda quadrada iniciada pela função `tone()` utilizaremos a função `noTone()`, desligando o buzzer e criamos um intervalo de 500 milissegundos;

Sintaxe da função noTone: `noTone(pino);`

Desta forma, o Sketch deste projeto ficará da seguinte maneira:

```
int buzzer = 8; //Atribui o valor 8 a variável buzzer, que representa o pino digital 8, onde o buzzer está conectado
int i = 0; //Variável para contar o número de vezes que o buzzer tocou

void setup() {
    pinMode(buzzer, OUTPUT); //Definindo o pino buzzer como de saída.
}

void loop() {
    for (i; i < 5; i++) { //Para i, enquanto i for menor que 5, realize o código e incremente 1 em i
        tone(buzzer, 1500); //Ligando o buzzer com uma frequência de 1500 Hz.
        delay(500); //Intervalo de 500 milissegundos
        noTone(buzzer); //Desligando o buzzer.
        delay(500); //Intervalo de 500 milissegundos
    }
}
```

TINKERCAD

Para melhor visualizar e/ou simular o circuito e a programação deste projeto basta se acessar o seguinte link: www.blogdarobotica.com/tinkercad-Buzzer5.

PROJETO MÚSICA DÓ RÉ MÍ FÁ NO BUZZER

Apesar do buzzer ativo não ser ideal para criar melodias, podemos fazer algumas músicas simples, como Dó Ré Mi Fá. Para reproduzir músicas no buzzer ativo utilizamos a função `tone()` alterando a frequência e o tempo das notas. Neste caso, a sintaxe que usaremos da função `tone()` será:

`tone(pino, frequência, duração)`

Em que:

Pino: É a porta em que buzzer está conectado;

Frequência: É frequência do tom em Hertz;

Duração: É a duração do tom em milissegundos (opcional).

Desta forma, a proposta desse projeto é reproduzir a música Dó Ré Mi Fá na placa UNO usando buzzer ativo.

MATERIAIS NECESÁRIOS E ESQUEMÁTICO DE LIGAÇÃO DOS COMPONENTES

Para construção deste projeto siga os passos do exemplo anterior (Figura 48).

ELABORANDO O CÓDIGO

Para elaborar a programação do código para reproduzir a música Dó Ré Mi Fá na placa UNO usando buzzer ativo precisamos conhecer a frequência das notas musicais. A Tabela 8 expõe as frequências de algumas notas musicais.

Tabela 8 - Notas músicas e suas frequências.

Nota	Frequência (Hz)
Dó	262
Ré	294
Mi	330
Fá	349
Sol	392
Lá	440
Si	494
Dó	523

Agora que já sabemos as frequências das notas, vamos a programação do Sketch. Vamos entender a lógica de programação deste projeto com os seguintes passos:

1- Declarar a variável

A variável `buzzer` será utilizada para representar o pino digital 8, onde o terminal positivo do buzzer está conectado;

2- Definir o pino de saída

A variável `buzzer` (pino 8) deve ser definida como saída, ou seja, `OUTPUT`;

3- Acionar o buzzer

Acionaremos o buzzer com a frequência da nota conforme a sequência: DÓ – RÉ – MI – FÁ – FÁ – FÁ – FÁ – DÓ – RÉ – DÓ – RÉ – RÉ – RÉ – DÓ – SOL – FÁ – MI – MI – MI – DÓ – RÉ – MI – FÁ – FÁ – FÁ.

A duração depende do tempo musical. Você pode verificar no sketch a seguir que algumas notas tem duração maior e outras tem duração menor.

Desta forma, o Sketch deste projeto ficará da seguinte maneira:

```
int buzzer = 8; //Atribui o valor 8 a variável buzzer, que representa o pino
digital 8, onde o buzzer está conectado

void setup()
{
    pinMode(buzzer, OUTPUT); //Definindo o pino buzzer como de saída.
}

void loop()
{
    tone(buzzer, 262, 200); //Frequência e duração da nota Dó
    delay(200); //Intervalo de 200 milissegundos
    tone(buzzer, 294, 300); //Frequência e duração da nota Ré
    delay(200); //Intervalo de 200 milissegundos
    tone(buzzer, 330, 300); //Frequência e duração da nota Mi
    delay(200); //Intervalo de 200 milissegundos
    tone(buzzer, 349, 300); //Frequência e duração da nota Fá
    delay(300); //Intervalo de 300 milissegundos
    tone(buzzer, 349, 300); //Frequência e duração da nota Fá
    delay(300); //Intervalo de 300 milissegundos
    tone(buzzer, 349, 300); //Frequência e duração da nota Fá
    delay(300); //Intervalo de 300 milissegundos
    tone(buzzer, 262, 100); //Frequência e duração da nota Dó
    delay(200); //Intervalo de 200 milissegundos
    tone(buzzer, 294, 300); //Frequência e duração da nota Ré
    delay(200); //Intervalo de 200 milissegundos
    tone(buzzer, 262, 100); //Frequência e duração da nota Dó
    delay(200); //Intervalo de 200 milissegundos
    tone(buzzer, 294, 300); //Frequência e duração da nota Ré
    delay(300); //Intervalo de 300 milissegundos
    tone(buzzer, 294, 300); //Frequência e duração da nota Ré
    delay(300); //Intervalo de 300 milissegundos
    tone(buzzer, 294, 300); //Frequência e duração da nota Ré
    delay(300); //Intervalo de 300 milissegundos
    tone(buzzer, 262, 200); //Frequência e duração da nota Dó
    delay(200); //Intervalo de 200 milissegundos
    tone(buzzer, 392, 200); //Frequência e duração da nota Sol
    delay(200); //Intervalo de 200 milissegundos
    tone(buzzer, 349, 200); //Frequência e duração da nota Fá
    delay(200); //Intervalo de 200 milissegundos
    tone(buzzer, 330, 300); //Frequência e duração da nota Mi
    delay(300); //Intervalo de 300 milissegundos
    tone(buzzer, 330, 300); //Frequência e duração da nota Mi
    delay(300); //Intervalo de 300 milissegundos
    tone(buzzer, 330, 300); //Frequência e duração da nota Mi
    delay(300); //Intervalo de 300 milissegundos
    tone(buzzer, 262, 200); //Frequência e duração da nota Dó
    delay(200); //Intervalo de 200 milissegundos
    tone(buzzer, 294, 300); //Frequência e duração da nota Ré
    delay(200); //Intervalo de 200 milissegundos
    tone(buzzer, 330, 300); //Frequência e duração da nota Mi
```

```
delay(200); //Intervalo de 200 milissegundos
tone(buzzer, 349, 300); //Frequência e duração da nota Fá
delay(300); //Intervalo de 300 milissegundos
tone(buzzer, 349, 300); //Frequência e duração da nota Fá
delay(300); //Intervalo de 300 milissegundos
tone(buzzer, 349, 300); //Frequência e duração da nota Fá
delay(2500); //Intervalo de 2,5 segundos
}
```

TINKERCAD

Para melhor visualizar e/ou simular o circuito e a programação deste projeto basta se acessar o seguinte link: www.blogdarobotica.com//tinkercad-BuzzerDoReMiFa.

EXTRAS

Você também pode visualizar e/ou simular dois outros projetos utilizando o buzzer para reproduzir som nos seguintes links:

Música Cai Cai Balão: www.blogdarobotica.com/tinkercad-BuzzerCaiCaiBalao

Sirene: www.blogdarobotica.com/tinkercad-BuzzerSirene

PROJETO PISCAR O LED RGB – VERMELHO, VERDE E AZUL

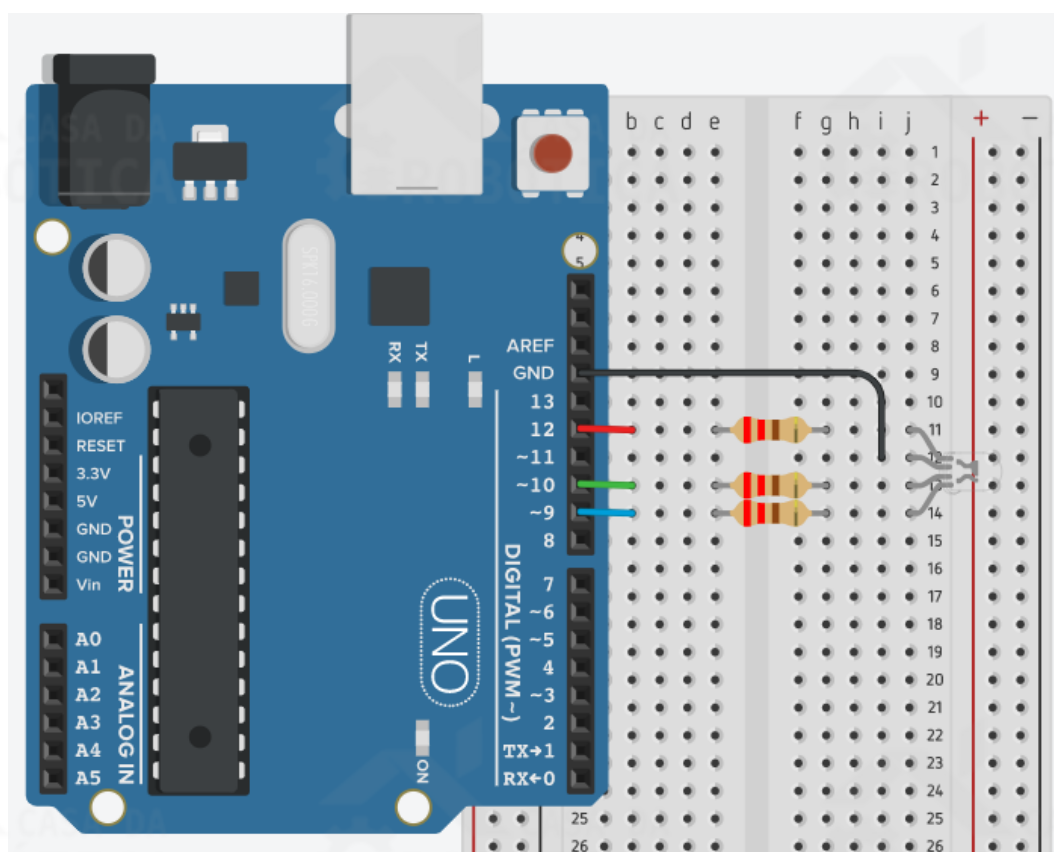
A proposta deste projeto é programar a placa UNO para acender o LED RGB alternando entre as cores vermelha, verde e azul em intervalos de 1 segundo. Diferente dos LEDs comuns, o LED RGB cátodo comum possui quatro terminais, um para o terra (GND), um para o LED interno vermelho, um para o LED interno verde e outro para o LED interno azul. Desta forma, para seu acionamento utilizaremos três portas digitais e o pino GND da placa UNO.

MATERIAIS NECESSÁRIOS

Para construção deste projeto vamos precisar dos seguintes materiais:

- ## ESQUEMÁTICO DE LIGAÇÃO DOS COMPONENTES

Figura 49 - Circuito para o projeto Piscar o LED RGB.



- O primeiro terminal do LED RGB corresponde ao LED interno vermelho;
- O segundo terminal, de maior tamanho, do LED RGB é o cátodo (negativo) e

deve ser conectado ao terra (GND);

- O terceiro terminal do LED RGB corresponde ao LED interno verde;
- O quarto terminal do LED RGB corresponde ao LED interno azul.

ELABORANDO O CÓDIGO

Com a montagem elétrica pronta, vamos a programação. Mas antes, faz-se necessária a configuração da placa e da porta de comunicação da placa UNO, conforme vimos anteriormente.

Inicialmente, vamos entender a lógica de programação a partir dos seguintes passos:

1- Definir as variáveis

A variável `vermelho` será utilizada para representar a porta 12, que está conectada ao LED interno vermelho.

A variável `verde` será utilizada para representar a porta 10, que está conectada ao LED interno verde.

A variável `azul` será utilizada para representar a porta 9, que está conectada ao LED interno azul.

2- Definir portas de saída

As variáveis `vermelho` (porta 12), `verde` (porta 10) e `azul` (porta 9) devem ser definidas como saída, ou seja, `OUTPUT`;

3- Acionar o LED RGB alternando entre as cores vermelha, verde e azul em intervalos de 1 segundo

Inicializaremos o loop ligando o LED interno vermelho. Para isso, estabelecemos nível alto para a variável `vermelho`. Em seguida, incluímos um `delay` de 1 segundo e retornaremos `vermelho` para o nível baixo (`LOW`), fazendo com que o LED da cor vermelha desligue.

Para acionar o LED interno verde, estabelecemos nível alto para a variável `verde`. Em seguida, incluímos um `delay` de 1 segundo e retornaremos `verde` para o nível baixo (`LOW`), fazendo com que o LED da cor verde desligue.

Por fim, estabelecemos nível alto para a variável azul. Em seguida, incluímos um `delay` de 1 segundo e retornaremos azul para o nível baixo (`LOW`), fazendo com que o LED da cor azul desligue.

Com isso, o Sketch deste projeto ficará da seguinte maneira:

```
int azul = 9; //Atribui o valor 9 a variável azul
int verde = 10; //Atribui o valor 10 a variável verde
int vermelho = 12; //Atribui o valor 11 a variável vermelho

void setup() {
  pinMode(azul, OUTPUT); //Define a variável azul (pino 9) como saída
  pinMode(verde, OUTPUT); //Define a variável verde (pino 10) como saída
  pinMode(vermelho, OUTPUT); //Define a variável vermelho (pino 11) como saída
}

void loop() {
  //Vermelho
  digitalWrite(vermelho, HIGH); //Coloca vermelho (pino 11) em nível alto, ligando-o
  delay(1000); //Espera 1000 milissegundos (1 segundo)
  digitalWrite(vermelho, LOW); //Coloca vermelho (pino 11) em nível baixo novamente, desligando-o
  delay(1000); //Espera 1000 milissegundos (1 segundo)
  //Verde
  digitalWrite(verde, HIGH); //Coloca verde (pino 10) em nível alto, ligando-o
  delay(1000); //Espera 1000 milissegundos (1 segundo)
  digitalWrite(verde, LOW); //Coloca verde (pino 10) em nível baixo novamente
  delay(1000); //Espera 1000 milissegundos (1 segundo)
  //Azul
  digitalWrite(azul, HIGH); //Coloca azul (pino 9) em nível alto, ligando-o
  delay(1000); //Espera 1000 milissegundos (1 segundo)
  digitalWrite(azul, LOW); //Coloca azul (pino 9) em nível baixo novamente
  delay(1000); //Espera 1000 milissegundos (1 segundo)
}
```

TINKERCAD

Deixamos disponível no Tinkercad o projeto Piscar LED RGB – Vermelho, Verde e Azul para que você possa simulá-lo. Mas preste atenção, pois os terminais do LED RGB disponível no Tinkercad seguem a sequência Red (vermelho), Blue (azul) e Green (verde). Desta forma, tanto o código quanto o circuito possuem pequenas diferenças. Para simulá-lo acesse o link: www.blogdarobotica.com/tinkercad-ledrgb.

PROJETO PISCAR O LED RGB – COMBINAÇÃO DE CORES

Este projeto tem como proposta realizar a combinação de cores para acender o LED RGB para produzir as cores branca, magenta (violeta-púrpura), amarelo e ciano. Assim como no projeto anterior, deve-se alternar as cores em intervalos de 1 segundo.

MATERIAIS NECESÁRIOS E ESQUEMÁTICO DE LIGAÇÃO DOS COMPONENTES

Para construção deste projeto vamos utilizar os mesmos componentes e esquemático de ligação dos componentes da Figura 49.

ELABORANDO O CÓDIGO

A lógica de programação deste projeto será semelhante à do projeto anterior. No entanto, os pinos deverão ser acionados simultaneamente para haver combinação entre duas ou três cores. A combinação aditiva entre as cores deve ser a seguinte:

- Branco: Deve-se acionar as três cores, simultaneamente. Então, é necessário acionar em nível lógico alto as portas 12 (vermelho), 10 (verde) e 9 (azul);
- Magenta (violeta-púrpura): Deve-se ligar as cores azul e vermelho ao mesmo tempo. Desta forma, é preciso acionar em nível lógico alto as portas 12 (vermelho) e 9 (azul);
- Amarelo: Deve-se ligar as cores verde e vermelho, concomitantemente. Para isso, é necessário acionar em nível lógico alto as portas 12 (vermelho) e 10 (verde);
- Ciano: Deve-se ligar as cores verde e azul ao mesmo tempo. Logo, é preciso acionar em nível lógico baixo as portas 10 (verde) e 9 (azul).

Com isso, o código deste projeto fica da seguinte forma:

```
int azul = 9; //Atribui o valor 9 a variável azul
int verde = 10; //Atribui o valor 10 a variável verde
int vermelho = 12; //Atribui o valor 12 a variável vermelho
```

```

void setup() {
  pinMode(azul, OUTPUT); //Define a variável azul como saída
  pinMode(verde, OUTPUT); //Define a variável verde como saída
  pinMode(vermelho, OUTPUT); //Define a variável vermelho como saída
}

void loop() {
  //Branco
  digitalWrite(azul, HIGH); //Coloca azul em nível alto, ligando-o
  digitalWrite(vermelho, HIGH); //Coloca vermelho em nível alto, ligando-o
  digitalWrite(verde, HIGH); //Coloca verde em nível alto, ligando-o
  delay(1000); //Intervalo de 1 segundo
  digitalWrite(azul, LOW); //Coloca azul em nível baixo novamente
  digitalWrite(vermelho, LOW); //Coloca vermelho em nível baixo novamente
  digitalWrite(verde, LOW); //Coloca verde em nível baixo novamente
  delay(1000); //Intervalo de 1 segundo
  //Magenta (violeta-púrpura)
  digitalWrite(azul, HIGH); //Coloca azul em nível alto
  digitalWrite(vermelho, HIGH); //Coloca vermelho em nível alto
  delay(1000); //Intervalo de 1 segundo
  digitalWrite(azul, LOW); //Coloca azul em nível baixo
  digitalWrite(vermelho, LOW); //Coloca vermelho em nível baixo
  delay(1000); //Intervalo de 1 segundo
  //Amarelo
  digitalWrite(verde, HIGH); //Coloca verde em nível alto
  digitalWrite(vermelho, HIGH); //Coloca vermelho em nível alto
  delay(1000); //Intervalo de 1 segundo
  digitalWrite(verde, LOW); //Coloca verde em nível baixo
  digitalWrite(vermelho, LOW); //Coloca vermelho em nível baixo
  delay(1000); //Intervalo de 1 segundo
  //Ciano
  digitalWrite(verde, HIGH); //Coloca verde em nível alto
  digitalWrite(azul, HIGH); //Coloca azul em nível baixo alto
  delay(1000); //Intervalo de 1 segundo
  digitalWrite(verde, LOW); //Coloca verde em nível baixo
  digitalWrite(azul, LOW); //Coloca azul em nível baixo
  delay(1000); //Intervalo de 1 segundo
}

```

TINKERCAD

Deixamos disponível no Tinkercad o projeto Piscar LED RGB – Combinação de Cores para que você possa simulá-lo. Mas lembre-se, os terminais do LED RGB disponível no Tinkercad seguem a sequência Red (vermelho), Blue (azul) e Green (verde). Desta forma, tanto o código quanto o circuito possuem pequenas diferenças. Para simulá-lo acesse o link: www.blogdarobotica.com/tinkercad-ledrbgcombinacor.

PROJETO PISCAR O LED RGB – TODAS AS CORES

A proposta deste projeto é programar a placa UNO para acender o LED RGB alternando entre todas as cores em intervalos de 1 segundo.

MATERIAIS NECESÁRIOS E ESQUEMÁTICO DE LIGAÇÃO DOS COMPONENTES

Para construção deste projeto vamos utilizar os mesmos componentes e esquemático de ligação dos componentes da Figura 49.

ELABORANDO O CÓDIGO

A programação desse projeto será a junção dos dois projetos anteriores. Desta forma, o Sketch ficará da seguinte forma:

```
int azul = 9; //Atribui o valor 9 a variável azul
int verde = 10; //Atribui o valor 10 a variável verde
int vermelho = 12; //Atribui o valor 12 a variável vermelho

void setup() {
  pinMode(azul, OUTPUT); //Define a variável azul como saída
  pinMode(verde, OUTPUT); //Define a variável verde como saída
  pinMode(vermelho, OUTPUT); //Define a variável vermelho como saída
}

void loop() {
  //Vermelho
  digitalWrite(vermelho, HIGH); //Coloca vermelho (pino 11) em nível alto, ligando-o
  delay(1000); //Espera 1000 milissegundos (1 segundo)
  digitalWrite(vermelho, LOW); //Coloca vermelho (pino 11) em nível baixo novamente, desligando-o
  delay(1000); //Espera 1000 milissegundos (1 segundo)
  //Verde
  digitalWrite(verde, HIGH); //Coloca verde (pino 10) em nível alto, ligando-o
  delay(1000); //Espera 1000 milissegundos (1 segundo)
  digitalWrite(verde, LOW); //Coloca verde (pino 10) em nível baixo novamente
  delay(1000); //Espera 1000 milissegundos (1 segundo)
  //Azul
  digitalWrite(azul, HIGH); //Coloca azul (pino 9) em nível alto, ligando-o
  delay(1000); //Espera 1000 milissegundos (1 segundo)
```



```

digitalWrite(azul, LOW); //Coloca azul (pino 9) em nível baixo novamente
delay(1000); //Espera 1000 milissegundos (1 segundo)
//Branco
digitalWrite(azul, HIGH); //Coloca azul em nível alto, ligando-o
digitalWrite(vermelho, HIGH); //Coloca vermelho em nível alto, ligando-o
digitalWrite(verde, HIGH); //Coloca verde em nível alto, ligando-o
delay(1000); //Intervalo de 1 segundo
digitalWrite(azul, LOW); //Coloca azul em nível baixo novamente
digitalWrite(vermelho, LOW); //Coloca vermelho em nível baixo novamente
digitalWrite(verde, LOW); //Coloca verde em nível baixo novamente
delay(1000); //Intervalo de 1 segundo
//Magenta (violeta-púrpura)
digitalWrite(azul, HIGH); //Coloca azul em nível alto
digitalWrite(vermelho, HIGH); //Coloca vermelho em nível alto
delay(1000); //Intervalo de 1 segundo
digitalWrite(azul, LOW); //Coloca azul em nível baixo
digitalWrite(vermelho, LOW); //Coloca vermelho em nível baixo
delay(1000); //Intervalo de 1 segundo
//Amarelo
digitalWrite(verde, HIGH); //Coloca verde em nível alto
digitalWrite(vermelho, HIGH); //Coloca vermelho em nível alto
delay(1000); //Intervalo de 1 segundo
digitalWrite(verde, LOW); //Coloca verde em nível baixo
digitalWrite(vermelho, LOW); //Coloca vermelho em nível baixo
delay(1000); //Intervalo de 1 segundo
//Ciano
digitalWrite(verde, HIGH); //Coloca verde em nível alto
digitalWrite(azul, HIGH); //Coloca azul em nível baixo alto
delay(1000); //Intervalo de 1 segundo
digitalWrite(verde, LOW); //Coloca verde em nível baixo
digitalWrite(azul, LOW); //Coloca azul em nível baixo
delay(1000); //Intervalo de 1 segundo
}

```

TINKERCAD

Disponibilizamos no Tinkercad o projeto Piscar LED RGB – Combinação de Cores para que você possa simulá-lo. Mas lembre-se, os terminais do LED RGB disponível no Tinkercad seguem a sequência Red (vermelho), Blue (azul) e Green (verde). Desta forma, tanto o código quanto o circuito possuem pequenas diferenças. Para simulá-lo acesse o link: <http://blogdarobotica.com/tinkercad-ledrgbtodascors>.

PROJETO PISCAR O LED RGB – TODAS AS CORES USANDO FUNÇÕES

A proposta deste projeto é programar a placa UNO para acender o LED RGB alternando entre todas as cores em intervalos de 1 segundo, tal como no projeto anterior. No entanto, desta vez deve-se utilizar funções.

A função é um conjunto de comandos que realiza uma tarefa ou ação definida. O uso de funções permite o reaproveitamento de código já construído, evita que um trecho do código seja repetido várias vezes e facilita a leitura do programa.

Neste projeto, utilizaremos funções com o objetivo de facilitar a leitura lógica e melhorar organização do programa.

MATERIAIS NECESÁRIOS E ESQUEMÁTICO DE LIGAÇÃO DOS COMPONENTES

Para construção deste projeto vamos utilizar os mesmos componentes e esquemático de ligação dos componentes da Figura 49.

ELABORANDO O CÓDIGO

A lógica de programação deste projeto será semelhante à do projeto anterior. No entanto, modularemos os trechos do código que definem a cor de acionamento do LED RGB em funções.

A declaração das variáveis e a função `setup()` serão idênticas aos outros projetos utilizando o LED RGB. No entanto, a função `loop()` ao invés de ter todas aquelas instruções para acionamento do LED RGB para cada uma das cores, conterà apenas o nome da função correspondente a cada uma dessas cores. Da seguinte maneira:

```
void loop() {  
  Vermelho(); //Função para acionamento na cor vermelha  
  Verde(); //Função para acionamento na cor verde  
  Azul(); //Função para acionamento na cor azul  
  Branco(); //Função para acionamento na cor branca  
  Magenta(); //Função para acionamento na cor magenta  
  Amarelo(); //Função para acionamento na cor amarela  
  Ciano(); //Função para acionamento na cor ciano  
}
```

Em seguida, cada uma das funções escritas no `loop()` deverão ser criadas, com nome idêntico declarado no loop. A primeira função que declaramos foi `Vermelho()` e sua estrutura deve ser a seguinte:

```
void Vermelho() {  
    digitalWrite(vermelho, HIGH);  
    delay(1000);  
    digitalWrite(vermelho, LOW);  
    delay(1000);  
}
```

Na função `Vermelho()` contém todas as instruções necessárias para acionamento do LED RGB na cor vermelha por 1 segundo e para o desligamento do mesmo por 1 segundo.

Para cumprir a proposta do projeto devemos escrever uma função para cada cor. Desta forma, o Sketch ficará da seguinte forma:

```
int azul = 9; //Atribui o valor 9 a variável azul  
int verde = 10; //Atribui o valor 10 a variável verde  
int vermelho = 12; //Atribui o valor 12 a variável vermelho  
  
void setup() {  
    pinMode(azul, OUTPUT); //Define a variável azul como saída  
    pinMode(verde, OUTPUT); //Define a variável verde como saída  
    pinMode(vermelho, OUTPUT); //Define a variável vermelho como saída  
}  
  
void loop() {  
    Vermelho(); //Função para acionamento na cor vermelha  
    Verde(); //Função para acionamento na cor verde  
    Azul(); //Função para acionamento na cor azul  
    Branco(); //Função para acionamento na cor branca  
    Magenta(); //Função para acionamento na cor magenta  
    Amarelo(); //Função para acionamento na cor amarela  
    Ciano(); //Função para acionamento na cor ciano  
}  
  
void Vermelho() {  
    digitalWrite(vermelho, HIGH); //Coloca vermelho em nível alto, ligando-o  
    delay(1000); //Intervalo de 1 segundo  
    digitalWrite(vermelho, LOW); //Coloca vermelho em nível baixo  
    delay(1000); //Intervalo de 1 segundo  
}  
  
void Verde() {  
    digitalWrite(verde, HIGH); //Coloca verde em nível alto  
    delay(1000); //Intervalo de 1 segundo  
    digitalWrite(verde, LOW); //Coloca verde em nível baixo  
    delay(1000); //Intervalo de 1 segundo  
}  
  
void Azul() {  
    digitalWrite(azul, HIGH); //Coloca azul em nível alto  
    delay(1000); //Intervalo de 1 segundo  
    digitalWrite(azul, LOW); //Coloca azul em nível baixo  
    delay(1000); //Intervalo de 1 segundo  
}
```

```

}
void Branco() {
    digitalWrite(azul, HIGH); //Coloca azul em nível alto
    digitalWrite(vermelho, HIGH); //Coloca vermelho em nível alto
    digitalWrite(verde, HIGH); //Coloca verde em nível alto, ligando-o
    delay(1000); //Intervalo de 1 segundo
    digitalWrite(azul, LOW); //Coloca azul em nível baixo
    digitalWrite(vermelho, LOW); //Coloca vermelho em nível baixo
    digitalWrite(verde, LOW); //Coloca verde em nível baixo
    delay(1000); //Intervalo de 1 segundo
}
void Magenta() {
    digitalWrite(azul, HIGH); //Coloca azul em nível alto
    digitalWrite(vermelho, HIGH); //Coloca vermelho em nível alto
    delay(1000); //Intervalo de 1 segundo
    digitalWrite(azul, LOW); //Coloca azul em nível baixo
    digitalWrite(vermelho, LOW); //Coloca vermelho em nível baixo
    delay(1000); //Intervalo de 1 segundo
}
void Amarelo() {
    digitalWrite(verde, HIGH); //Coloca verde em nível alto
    digitalWrite(vermelho, HIGH); //Coloca vermelho em nível alto
    delay(1000); //Intervalo de 1 segundo
    digitalWrite(verde, LOW); //Coloca verde em nível baixo
    digitalWrite(vermelho, LOW); //Coloca vermelho em nível baixo
    delay(1000); //Intervalo de 1 segundo
}
void Ciano() {
    digitalWrite(verde, HIGH); //Coloca verde em nível alto
    digitalWrite(azul, HIGH); //Coloca azul em nível baixo alto
    delay(1000); //Intervalo de 1 segundo
    digitalWrite(verde, LOW); //Coloca verde em nível baixo
    digitalWrite(azul, LOW); //Coloca azul em nível baixo
    delay(1000); //Intervalo de 1 segundo
}
}

```



OBSERVAÇÃO:

- As funções podem ser escritas antes ou depois do loop().

TINKERCAD

Disponibilizamos no Tinkercad o projeto Piscar LED RGB – Combinação de Cores Usando Funções para que você possa simulá-lo. Mas lembre-se, os terminais do LED RGB disponível no Tinkercad seguem a sequência Red (vermelho), Blue (azul) e Green (verde). Desta forma, tanto o código quanto o circuito possuem pequenas diferenças. Para simulá-lo acesse o link: www.blogdarobotica.com/tinkercad-ledrgbfuncao.

PROJETO ESCOLHER A COR DO LED RGB PELO MONITOR SERIAL

Assim como podemos receber dados e visualizá-los através do monitor serial do Arduino IDE, também podemos enviar dados e comandos por meio desta ferramenta, de modo a controlar dispositivos conectados a placa UNO. A proposta desse projeto é utilizar o monitor serial para enviar dados que permitam a escolha da cor do LED RGB que deve ser acionada. Em outras palavras, o usuário poderá escolher a cor que deseja acionar.

O intuito desse projeto é ensinar como utilizar o monitor serial para enviar dados do computador para a placa UNO. Neste projeto também será ensinado como usar a estrutura de controle de fluxo de seleção *switch...case*.

MATERIAIS NECESÁRIOS E ESQUEMÁTICO DE LIGAÇÃO DOS COMPONENTES

Para construção deste projeto vamos utilizar os mesmos componentes e esquemático de ligação dos componentes da Figura 49.

ELABORANDO O CÓDIGO

Após a montagem do circuito vamos a programação do código. O objetivo desse projeto é utilizar o monitor serial para comandar a cor do LED RGB. Para isso, vamos entender a lógica de programação a partir dos seguintes passos:

1- Declarar variáveis

Além das variáveis atribuídas as portas da placa UNO que se encontram conectadas aos terminais do LED RGB (`vermelho` = porta 12; `verde` = porta 10; `azul` = porta 9), deve ser declarada a variável `cor`, do tipo `char`, que será utilizada para armazenar o caractere que for escrito pelo usuário no monitor serial.

2- Definir os pinos de entrada e saída

As variáveis `vermelho`, `verde` e `azul` devem ser definidas como saída, ou seja, `OUTPUT`.

3- Iniciar a comunicação serial

Inicializamos a comunicação serial por meio da instrução `Serial.begin(9600);`;

4- Criar as funções para acionamento do LED RGB

Assim como no projeto anterior devemos escrever as funções que serão responsáveis por comandar os terminais do LED RGB. Criaremos uma função para cada cor e outra função para o LED desligado.

5- Verificar, receber e armazenar um caractere pelo monitor serial

Inicializaremos o loop testando se algum caractere foi enviado ao monitor serial por meio da instrução `if (Serial.available())`. Uma vez satisfeita essa condição, o caractere será lido e armazenado na variável `cor` através da instrução `cor = Serial.read();`.

6- Definir caractere correspondente a cor

Como estamos trabalhando com caracteres, vamos definir um caractere correspondente a cada cor. Nesse projeto, utilizaremos os numerais de 1 a 7 para representar as cores, conforme demonstra a Tabela 9, mas você pode utilizar qualquer outro tipo de caractere.

Tabela 9 - Caracteres e suas cores correspondentes.

Caractere	Cor correspondente
1	Vermelho
2	Verde
3	Azul
4	Branco
5	Magenta
6	Amarelo
7	Ciano

7- Verificação do caractere da variável `cor`

Utilizaremos a estrutura de controle de fluxo de seleção `switch...case` para comparar o caractere armazenado na variável `cor` aos caracteres especificados nos comandos `cases`. Quando o caractere armazenado for igual ao caractere especificado no `case`, o código para acionamento do LED RGB nesta cor deve ser executado. Caso não seja encontrado nenhum caractere correspondente ao `case` o `default` será executado.

Os caracteres especificados nos `cases` devem ser incluídos entre aspas simples (' ').

Para melhor compreensão da estrutura switch... case observe atentamente as instruções e os comentários presentes no código a seguir:

```
int azul = 9;//Atribui o valor 9 a variável azul
int verde = 10;//Atribui o valor 10 a variável verde
int vermelho = 12;//Atribui o valor 12 a variável vermelho
char cor;

void setup() {
    pinMode(azul, OUTPUT);//Define a variável azul como saída
    pinMode(verde, OUTPUT);//Define a variável verde como saída
    pinMode(vermelho, OUTPUT);//Define a variável vermelho como saída
    Serial.begin(9600);
}

//Função para acionamento da cor vermelho
void Vermelho() {
    digitalWrite(verde, LOW);
    digitalWrite(azul, LOW);
    digitalWrite(vermelho, HIGH);
    delay(1000);
}

//Função para acionamento da cor verde
void Verde() {
    digitalWrite(verde, HIGH);
    digitalWrite(azul, LOW);
    digitalWrite(vermelho, LOW);
    delay(1000);
}

//Função para acionamento da cor azul
void Azul() {
    digitalWrite(verde, LOW);
    digitalWrite(azul, HIGH);
    digitalWrite(vermelho, LOW);
    delay(1000);
}

//Função para acionamento da cor branca
void Branco() {
    digitalWrite(azul, HIGH);
    digitalWrite(vermelho, HIGH);
    digitalWrite(verde, HIGH);
    delay(1000);
}

//Função para acionamento da cor magenta
void Magenta() {
    digitalWrite(azul, HIGH);
    digitalWrite(vermelho, HIGH);
    digitalWrite(verde, LOW);
    delay(1000);
}

//Função para acionamento da cor amarela
void Amarelo() {
    digitalWrite(verde, HIGH);
    digitalWrite(vermelho, HIGH);
    digitalWrite(azul, LOW);
    delay(1000);
}

//Função para acionamento da cor ciano
void Ciano() {
    digitalWrite(verde, HIGH);
    digitalWrite(azul, HIGH);
}
```

```

digitalWrite(vermelho, LOW);
delay(1000);
}
//Função desligado
void Desligado() {
    digitalWrite(verde, LOW);
    digitalWrite(vermelho, LOW);
    digitalWrite(azul, LOW);
}

void loop() {

    if (Serial.available()) { //Se a serial receber algum caractere
        cor = Serial.read(); //Lê os dados recebidos na porta serial e guarda
na variável cor
        Serial.println(cor); //Imprime na porta serial o dado digitado pelo
usuário
    }
    switch (cor) {
        case '1':
            Vermelho(); //Executa a função Vermelho
            break;

        case '2':
            Verde(); //Executa a função Verde
            break;

        case '3':
            Azul(); //Executa a função Azul
            break;

        case '4':
            Branco(); //Executa a função Branco
            break;

        case '5':
            Magenta(); //Executa a função Magenta
            break;

        case '6':
            Amarelo();
            break;

        case '7':
            Ciano(); //Executa a função Ciano
            break;

        default:
            Desligado();
            break;
    }
}

```



OBSERVAÇÃO:

- Este projeto pode ser feito definindo a variável cor do tipo String (Utilizada para armazenar cadeias de texto). Assim a variável cor poderá armazenar uma palavra.

- No switch será realizada a comparação da palavra armazenada na variável cor às palavras especificados nos comandos cases;
- As palavras especificadas nos cases devem ser incluídos entre aspas duplas (“ ”).

TINKERCAD

Disponibilizamos no Tinkercad o projeto Escolher a Cor do LED RGB pelo Monitor Serial para que você possa simulá-lo. Mas lembre-se, os terminais do LED RGB disponível no Tinkercad seguem a sequência Red (vermelho), Blue (azul) e Green (verde). Desta forma, tanto o código quanto o circuito possuem pequenas diferenças. Para simulá-lo acesse o link: <http://blogdarobotica.com/tinkercad-ledrbgserial>.

PROJETO PISCAR LED COM INTERVALO DEFINIDO PELO POTENCIÔMETRO

O propósito deste projeto é utilizar um potenciômetro para controlar o tempo de pisca de um LED. Apesar de bastante simples, este projeto ensinará como utilizar o potenciômetro.

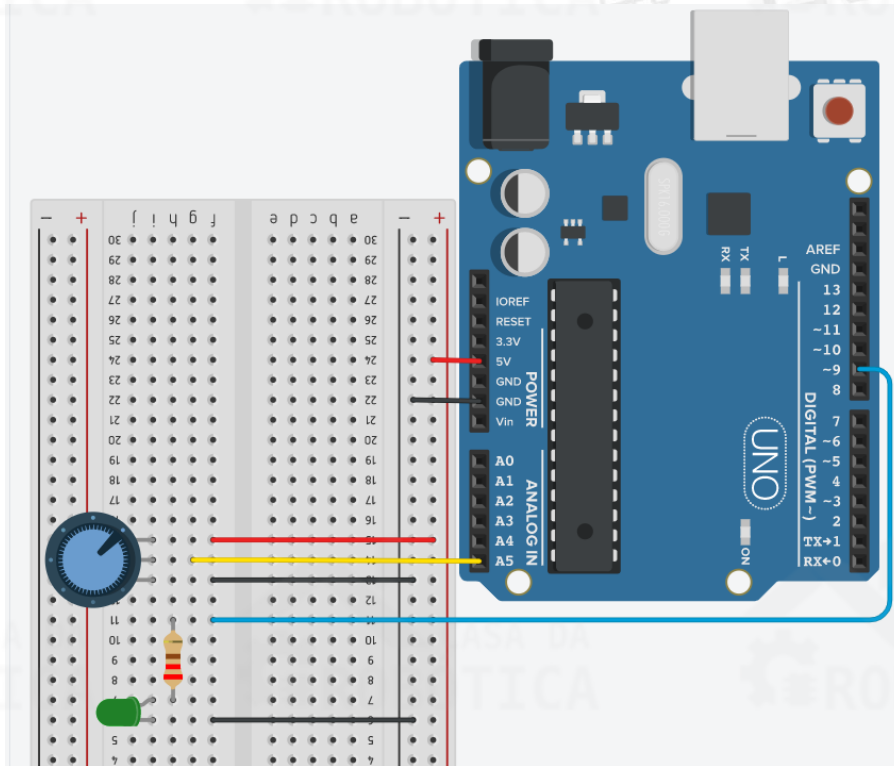
MATERIAIS NECESÁRIOS

- 1 x Placa UNO SMD R3 Atmega328 compatível com Arduino UNO;
- 1 x Cabo USB;
- 1 x Protoboard;
- 1 x LED difuso;
- 1 x Resistor de 220 Ω ;
- 1 x Potenciômetro de 10 k Ω ;
- Fios de jumper macho-macho.

ESQUEMÁTICO DE LIGAÇÃO DOS COMPONENTES

Inicialmente, certifique-se que a sua placa UNO esteja desligada. Monte o circuito da Figura 50 utilizando a protoboard, o potenciômetro, o LED, o resistor e os fios.

Figura 50 – Circuito para projeto controlar tempo de pisca do LED com potenciômetro.



Ao montar seu circuito na protoboard preste atenção nos seguintes pontos:

- Lembre-se que o LED tem polaridade. O terminal maior tem polaridade positiva, que deve ser conectado ao pino 9, e o lado chanfrado possui polaridade negativa;
- O primeiro terminal do potenciômetro deve ser conectado ao terra (GND), o segundo ao pino analógico A5 e o terceiro ao 5 V.

ELABORANDO O CÓDIGO

Com o circuito montado, vamos a programação do Sketch. O intuito deste projeto é controlar o tempo em que o LED permanece ligado ou desligado com um potenciômetro, de modo que quanto maior o valor lido pela porta analógica conectada ao potenciômetro, maior será o intervalo de pisca.

Vamos entender melhor a lógica de programação a partir dos seguintes passos:

1- Declarar as variáveis

Inicializaremos o programa declarando três variáveis: `ledPin` atribuída a porta digital 9, em que o LED está conectado; `potPin` atribuída a porta analógica A5, em que o potenciômetro está conectado; `tempo` variável do tipo inteiro responsável por armazenar a leitura analógica do potenciômetro.

2- Configurar as portas de entrada e saída

A variável `ledPin` deve ser configurada como saída (`OUTPUT`) e `potPin` como entrada (`INPUT`);

3- Iniciar a comunicação serial

Inicializamos a comunicação serial por meio da instrução `Serial.begin(9600)`;

4- Realizar a leitura da porta analógica e imprimir o valor lido

Iniciamos o `loop()` realizando a leitura da porta analógica A5 (`potPin`), para isso utilizaremos a função `analogRead(potPin)`, e armazenaremos este valor na variável `tempo`. Em seguida, imprimiremos o valor no monitor serial por meio da instrução `Serial.println(tempo)`;

5- Piscar LED com intervalo definido pelo potenciômetro

Para controlar o intervalo de pisca do LED pelo potenciômetro utilizaremos a leitura analógica armazenada na variável `tempo` como `delay`, através da instrução `delay(tempo)`; Para acionar o LED usaremos a instrução `digitalWrite(ledPin, HIGH)`; e para desligá-lo utilizaremos a instrução `digitalWrite(ledPin, LOW)`;

Ao fim, o Sketch ficará da seguinte forma:

```
int ledPin = 9; //Atribui o pino 9 a variável ledPin
int potPin = A5; //Atribui o pino analógico A5 a variável potPin
int tempo = 0; //Variável responsável pelo armazenamento da leitura bruta
do potenciômetro

void setup () {
  pinMode(ledPin, OUTPUT); //Configura ledPin como saída
  pinMode(potPin, INPUT); //Configura potPin como entrada
  Serial.begin(9600); // Inicializa a comunicação serial
}

void loop () {
  tempo = analogRead(potPin); //Efetua a leitura do potenciômetro
  Serial.println(tempo); //Imprime tempo na serial
  digitalWrite(ledPin, HIGH); //Coloca ledPin em nível alto, ligando o led
  delay(tempo); //tempo é utilizado como intervalo
```

```
digitalWrite(ledPin, LOW); //Coloca ledPin em nível baixo, desligando o  
led  
delay(tempo); //tempo é utilizado como intervalo  
}
```

TINKERCAD

Para melhor visualizar e/ou simular o circuito e a programação deste projeto basta se acessar o seguinte link: www.blogdarobotica.com/tinkercad-ledpot.

PROJETO FADE LED COM POTENCIÔMETRO

A proposta deste projeto é controlar a intensidade do brilho de um LED utilizando a placa UNO em conjunto com um potenciômetro, de modo que ao variarmos a resistência do potenciômetro ocorrerá, proporcionalmente, uma variação de tensão na entrada analógica da placa UNO. Quanto maior for esta tensão, maior será a intensidade do brilho do LED.

Para tal, vamos utilizar a técnica PWM, do inglês *Pulse Width Modulation*, que significa Modulação por largura de pulso e que pode ser utilizado para emular um sinal analógico através de pulsos digitais.

Você já deve ter percebido que algumas portas digitais do Arduino apresentam o símbolo “~”. Esta marcação indica que estes pinos são capazes de realizar PWM. O Arduino Uno possui 6 pinos para saída PWM, sendo 3, 5, 6, 9, 10 e 11.

Nesses pinos de saída digital, o Arduino envia uma onda quadrada ao ligá-los e desligá-los muito rapidamente. Esse padrão ligado/desligado pode simular uma tensão variando entre 0 V e 5 V alterando o tempo em que a saída permanece alta (ligada) e baixa (desligada). A técnica PWM consiste em manter a frequência de uma onda quadrada fixa e variar o tempo em que o sinal fica em nível alto.

Além disto, vamos usar uma função chamada `map`, que converte o valor lido da entrada analógica (entre 0 e 1023) para um valor entre 0 e 255 (8bits), que será utilizado para ajustar a intensidade do brilho do LED. Afinal, utilizaremos a leitura analógica para controlar o PWM.

Para saber mais sobre a função `map` acesse: <https://www.arduino.cc/reference/pt/language/functions/math/map/>.

MATERIAIS NECESÁRIOS E ESQUEMÁTICO DE LIGAÇÃO DOS COMPONENTES

Para construção deste projeto vamos utilizar os mesmos componentes e esquemático de ligação dos componentes da Figura 50.

ELABORANDO O CÓDIGO

Com o circuito montado, vamos a programação do Sketch. O intuito deste projeto é controlar a intensidade do brilho de um LED por meio da variação da resistência de um potenciômetro. Esta variação provocará uma variação de tensão na porta de entrada analógica (A5) da placa UNO.

Vamos entender a lógica de programação:

1- Declarar as variáveis:

Inicializaremos o programa declarando quatro variáveis: Definimos o pino digital 9 à variável `ledPin`; Definimos o pino A5 à variável `potPin`; Declaramos a variável `valorpot`, do tipo inteiro, para armazenar os valores brutos de leitura do potenciômetro. Declaramos a variável `pwm`, do tipo inteiro, para armazenar os valores convertidos pela função `map()`;

2- Configurar as portas de entrada e saída

A variável `ledPin` deve ser configurada como saída (`OUTPUT`) e `potPin` como entrada (`INPUT`);

3- Iniciar a comunicação serial

A comunicação serial foi inicializada por meio da instrução: `Serial.begin(9600)`;

4- Realizar a leitura da porta analógica

No `loop`, efetuamos a leitura da variável `potPin` (pino A5) e guardamos na variável `valorpot`, através da função `analogRead`;

5- Utilizar a função map:

A variável `pwm` recebe o valor do mapeamento da variável `valorpot`, convertendo a escala de 0 a 1023 para a escala de 0 a 255. Deste modo, se a variável `valorpot` for igual a zero, o valor de `pwm` também será zero, e se o valor da variável `valorpot`

for igual a 1023, o valor da variável `pwm` será 255. Portanto, os valores da variável `pwm` irão variar de 0 a 255, conforme o potenciômetro varia de 0 a 1023 de acordo com a posição do seu eixo.

Sintaxe da função `map()`:

Variável = `map` (valor lido, mínimo do potenciômetro (0), máximo do potenciômetro (1023), novo mínimo (0), novo máximo (255);

6- Imprimir o valor de `pwm` no monitor serial;

Imprimiremos o valor de `pwm` no monitor serial por meio da instrução `Serial.println(pwm);`.

7- Atribuir o valor de `pwm` para `ledPin`

Atribuímos o valor de `pwm` a variável `ledPin`, ou seja, enviamos um sinal analógico para saída do LED, com intensidade variável através da instrução `analogWrite(ledPin, pwm);`

Ao fim, o Sketch ficará da seguinte forma:

```
int ledPin = 9; //Atribui o pino 9 a variável ledPin
int potPin = A5; //Atribui o pino analógico A5 a variável potPin
int valorpot = 0; //Variável responsável pelo armazenamento da leitura bruta do potenciômetro
int pwm = 0; //Variável responsável pelo armazenamento do valor convertido pela função map

void setup() {
  pinMode(ledPin, OUTPUT); //Configura ledPin como saída
  pinMode(potPin, INPUT); //Configura potPin como entrada
  Serial.begin(9600); //Inicializa a serial com velocidade de comunicação de 9600.
}

void loop() {
  valorpot = analogRead(potPin); //Efetua a leitura do pino analógico A5
  pwm = map(valorpot, 0, 1023, 0, 255); //Função map() para converter a escala de 0 a 1023 para a escala de 0 a 255
  Serial.println(pwm); //Imprime valorpot na serial
  analogWrite(ledPin, pwm); //Aciona o LED proporcionalmente à leitura analógica
  delay(500); //Intervalo de 500 milissegundos
}
```

TINKERCAD

Para melhor visualizar e/ou simular o circuito e a programação deste projeto basta se acessar o seguinte link: www.blogdarobotica.com/tinkercad-fadepot.

EXTRAS

Você também pode visualizar e/ou simular outro projeto utilizando o PWM no seguinte link: www.blogdarobotica.com/tinkercad-fadeled.

Neste projeto a proposta foi utilizar a função `analogWrite()` para apagar e acender um LED, gradativamente. O `analogWrite()` usa modulação por largura de pulso (PWM), ativando e desativando um pino digital gradativamente, criando um efeito de aumento e diminuição do brilho.

PROJETO CONTADOR DE 0 A 9 COM DISPLAY DE 7 SEGMENTOS CÁTODO COMUM

Os displays de 7 segmentos são amplamente utilizados em projetos em que se faz necessária a apresentação de informações de forma visual. A proposta deste projeto é criar um contador incremental de 0 a 9 utilizando um display de 7 segmentos cátodo comum em conjunto com a placa UNO e um botão.

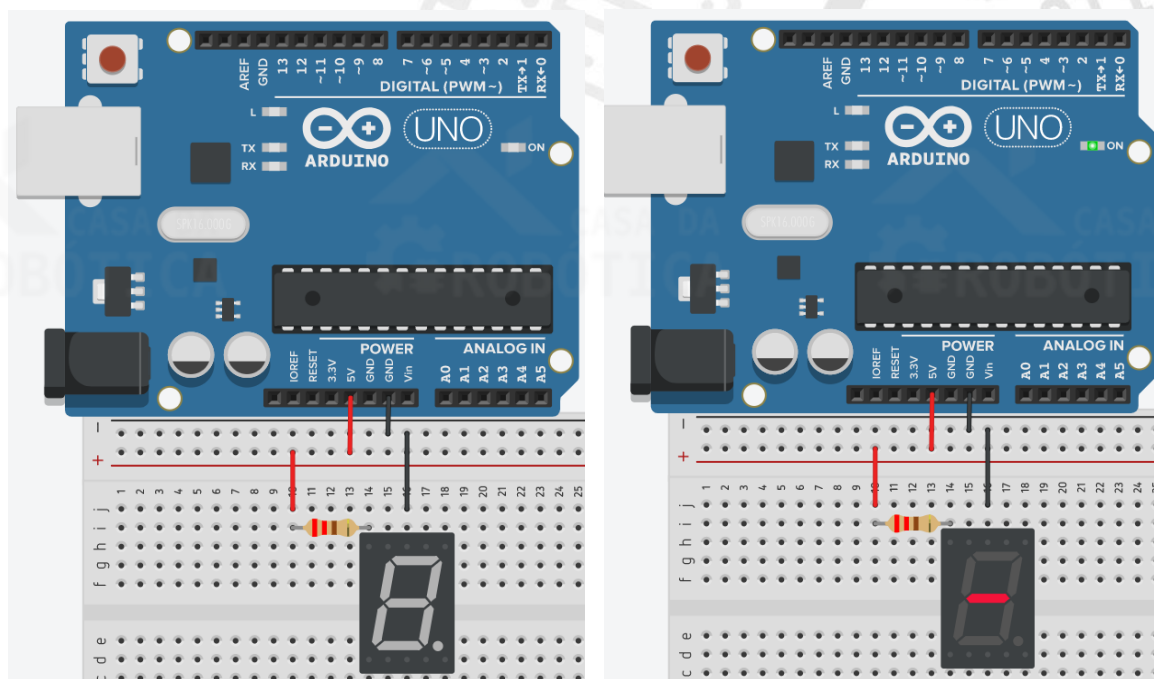
Além disso, esse projeto servirá para colocar em prática a criação de funções e o uso da estrutura de controle de fluxo *switch...case*.



OBSERVAÇÃO:

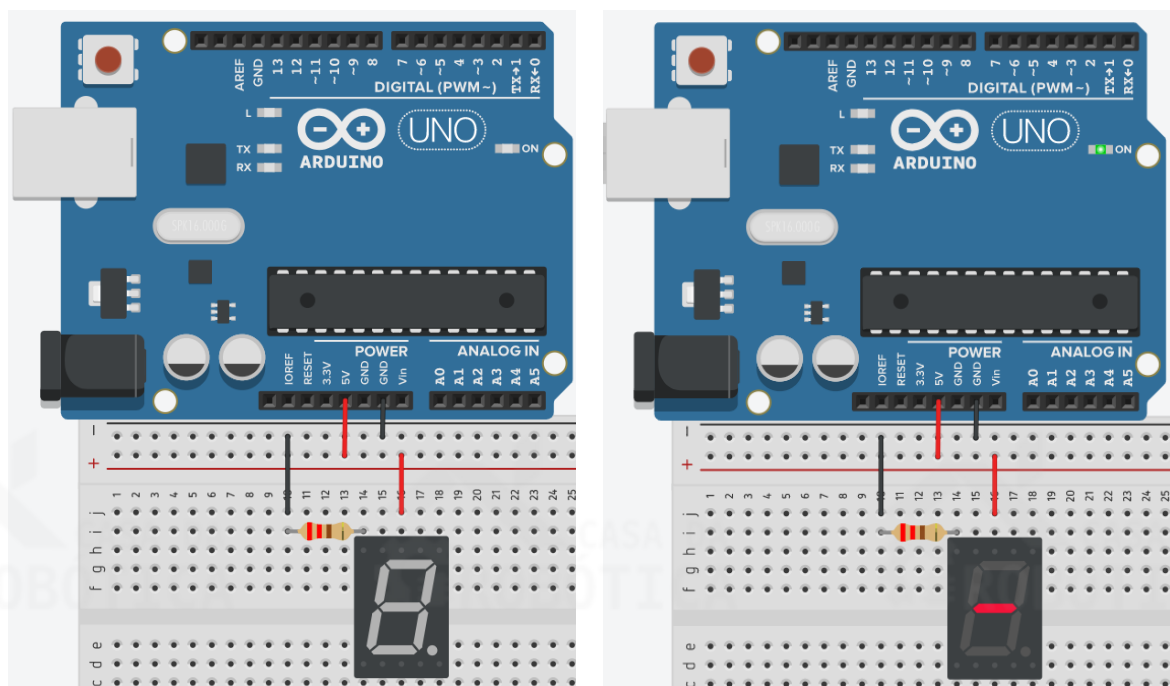
Antes de iniciar seu projeto verifique se o display de 7 segmentos do seu kit é do tipo cátodo comum ou ânodo comum. Para fazer essa identificação verifique a etiqueta do produto. Caso tenha perdido a etiqueta, a verificação do display pode ser feita montando o circuito de teste da Figura 51. Se o LED do meio (g) for acionado, seu display é do tipo **cátodo comum**.

Figura 51 - Teste do display 7 segmentos – Cátodo Comum.



Caso o LED do meio (g) não ligue seu display é do tipo **ânodo comum**. Para verificar monte o circuito da Figura 52. Desse modo, pule este exemplo e vá para o **PROJETO CONTADOR DE 0 A 9 COM DISPLAY DE 7 SEGMENTOS ÂNODO COMUM**.

Figura 52 - Teste do display 7 segmentos – Ânodo Comum.



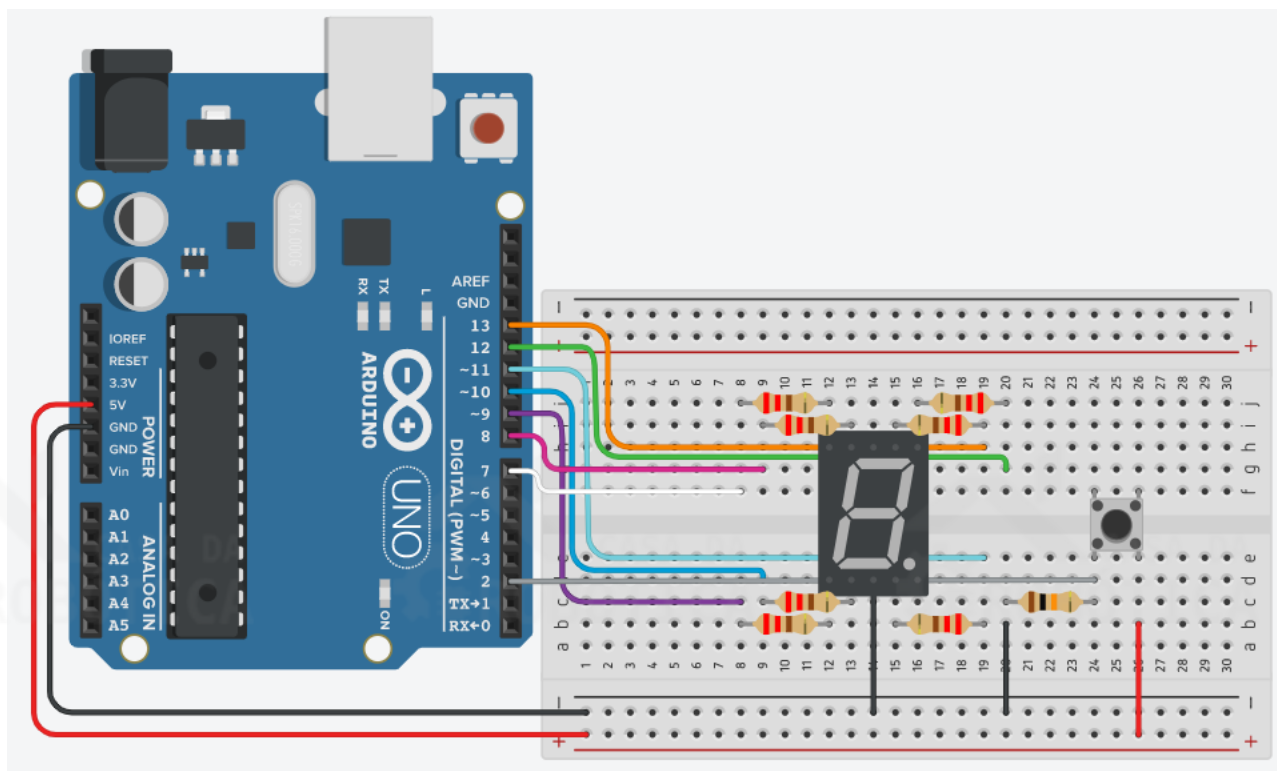
MATERIAIS NECESÁRIOS

- 1 x Placa UNO SMD R3 Atmega328 compatível com Arduino UNO;
- 1 x Cabo USB;
- 1 x Protoboard;
- 1 x Display de 7 segmentos cátodo comum;
- 7 x Resistor de 220 Ω ;
- 1 x Resistor de 10 k Ω ;
- 1 x Botão push button;
- Fios de jumper macho-macho.

ESQUEMÁTICO DE LIGAÇÃO DOS COMPONENTES

Inicialmente, certifique-se que a sua placa UNO esteja desligada. Monte o circuito da Figura 53 utilizando o display de 7 segmentos, o botão, os resistores e os fios.

Figura 53 - Circuito para projeto contador de 0 a 9 com display de 7 segmentos cátodo comum.



Ao montar seu circuito na protoboard preste atenção nos seguintes pontos:

- O display de 7 segmentos é do tipo cátodo comum, então um dos terminais comuns do mesmo deve ser conectado ao terra (GND);
- Os segmentos do display de 7 segmentos são nomeados de **a** a **g**, em sentido horário, a partir do segmento superior do componente;
- Em cada segmento do display vamos conectar um resistor de 220 Ω ;
- Os segmentos encontram-se conectado as portas digitais da seguinte maneira:
a = 13; b = 12; c = 11; d = 10; e = 9; f = 8; g = 7;
- O botão push button deve ser conectado a uma resistência pull-down de 10 k Ω e a porta digital 2.

ELABORANDO O CÓDIGO

Com o circuito montado, vamos a programação do Sketch. Vamos entender a lógica de programação para o projeto contador de 0 a 9 com display de 7 segmentos a partir dos seguintes passos:

1- Declarar as variáveis

Iniciaremos a programação declarando as variáveis correspondentes ao controle dos segmentos do display. Conforme o esquemático elétrico, atribuiremos a porta 13 a variável **a**; a porta 12 a variável **b**; a porta 11 a variável **c**; a porta 10 a variável **d**; a porta 9 a variável **e**; a porta 8 a variável **f**; a porta 7 a variável **g**.

Em seguida, devemos declarar a variável correspondente ao botão e sua leitura. A porta 2 será atribuída a variável `buttonPin`. A variável `leitura`, do tipo inteiro, será responsável por receber a leitura do estado do botão e a variável `ultleitura`, também do tipo inteiro, será responsável por receber o valor da última leitura do botão. Deve ser declarada também a variável `contador`, do tipo inteiro, que terá sua função detalhada mais adiante.

2- Configurar as portas de entrada e saída e inicializar a comunicação serial

Na função `setup` configuraremos todos as variáveis conectadas aos segmentos do display 7 segmentos como saída, ou seja, as variáveis **a**, **b**, **c**, **d**, **e**, **f** e **g** devem ser configuradas como **OUTPUT**. Por sua vez, a variável `buttonPin` deve ser configurada como entrada (**INPUT**).

A comunicação serial será inicializada por meio da instrução:

```
Serial.begin(9600);
```

3- Criar as funções dos números a serem exibidos no display 7 segmentos

Na programação deste Sketch precisamos criar funções que serão responsáveis por acionar os segmentos do display para que ele exiba números de 0 a 9.

Desta forma, vamos criar uma função para cada número. Cada função conterá a combinação de segmentos que devem ser acionados (colocados em nível alto) para que exiba o número desejado. Para formar um dígito é necessário acender os segmentos correspondentes. Na Tabela 10 estão listados os segmentos que devem ser acionados para formação dos números de 0 a 9.

Tabela 10 - Número e sequência de segmentos.

Número	Sequência
0	a, b, c, d, e, f
1	b, c
2	a, b, d, e, g
3	a, b, c, d, g
4	b, c, f, g
5	a, c, d, f, g
6	a, c, d, e, f, g
7	a, b, c
8	a, b, c, d, e, f, g
9	a, b, c, f, g

Por exemplo, a função `zero()` conterá todas as instruções necessárias para acionamento dos segmentos do display para exibir o número zero. Conforme a tabela precisamos colocar em nível alto as variáveis **a**, **b**, **c**, **d**, **e**, e **f** e colocar em nível baixo a variável **g**. Sendo assim, a função `zero()` será escrita da seguinte forma:

```
void zero() {  
    digitalWrite(a, 1); //Coloca a em nível alto  
    digitalWrite(b, 1); //Coloca b em nível alto  
    digitalWrite(c, 1); //Coloca c em nível alto  
    digitalWrite(d, 1); //Coloca d em nível alto  
    digitalWrite(e, 1); //Coloca e em nível alto  
    digitalWrite(f, 1); //Coloca f em nível alto  
    digitalWrite(g, 0); //Coloca g em nível baixo  
    delay(100); //Intervalo de 100 milissegundos  
}
```

4- Realizar a leitura do botão

Inicializaremos o loop com a leitura do botão, variável `buttonPin`, e atribuir o resultado da leitura a variável `leitura`. Isso será feito através da instrução `leitura=digitalRead(buttonPin);`

5- Comparar o valor de leitura com `ultleitura`

Neste projeto precisamos saber quantas mudanças de estado do botão incremental ocorreram, isso é chamado de detecção de mudança de estado ou detecção de borda. Para fazer essa detecção precisamos comparar o estado de leitura do botão com a leitura anterior e faremos isso através das seguintes instruções:

```
if(leitura != ultleitura) { //Compara a leitura do botão
com a leitura anterior
    if(leitura == HIGH) { //Se leitura for igual a HIGH
        contador++; //Incrementa contador em 1
    }
}
ultleitura = leitura; //Atribui a ultleitura o conteúdo de
leitura para a próxima execução do loop
```

6- Verificar o valor de `contador`

Utilizaremos a estrutura de controle de fluxo de seleção `switch...case` para comparar o valor armazenado na variável `contador` aos números especificados nos comandos `cases`. Quando o valor armazenado for igual ao caractere especificado no `case`, a função para exibição do número correspondente deve ser executado.

7- Imprimir na serial o valor de `contador`

Para ter um maior controle e comparar o valor armazenado na variável `contador` ao exibido no display 7 segmentos incluímos a instrução `Serial.println(contador);` para que seja exibido seu valor no monitor serial.

8- Comparar se o valor da variável `contador` é maior ou igual a 10

Como estamos utilizando apenas um display de 7 segmentos apenas podemos exibir números de 0 a 9, desta forma precisamos limitar o valor de `contador` para sempre que atingir um valor maior ou igual a 10 ele seja zerado. Para isso, utilizamos as seguintes instruções:

```
if(contador >= 10) { //Se contador for maior ou igual a 10
    contador = 0; //Retorna contador a zero
```

Ao fim, o Sketch ficará da seguinte forma:

```

int a = 13; //Correspondente ao LED a
int b = 12; //Correspondente ao LED b
int c = 11; //Correspondente ao LED c
int d = 10; //Correspondente ao LED d
int e = 9; //Correspondente ao LED e
int f = 8; //Correspondente ao LED f
int g = 7; //Correspondente ao LED g
int buttonPin = 2; //Correspondente ao botão
int leitura = 0; //Leitura do botão
int ultleitura = 0; //Última leitura do botão
int contador = 0; //Correspondente ao contador

void setup() {
  pinMode(a, OUTPUT); //Define a como saída
  pinMode(b, OUTPUT); //Define b como saída
  pinMode(c, OUTPUT); //Define c como saída
  pinMode(d, OUTPUT); //Define d como saída
  pinMode(e, OUTPUT); //Define e como saída
  pinMode(f, OUTPUT); //Define f como saída
  pinMode(g, OUTPUT); //Define g como saída
  pinMode(buttonPin, INPUT); //Define buttonPin como entrada
  Serial.begin(9600); //Inicia a comunicação serial
}

//Função para escrever o n° zero
void zero() {
  digitalWrite(a, 1); //coloca a em nível alto
  digitalWrite(b, 1); //coloca b em nível alto
  digitalWrite(c, 1); //coloca c em nível alto
  digitalWrite(d, 1);
  digitalWrite(e, 1);
  digitalWrite(f, 1);
  digitalWrite(g, 0);
  delay(100);
}

//Função para escrever o n° um
void um() {
  digitalWrite(a, 0);
  digitalWrite(b, 1);
  digitalWrite(c, 1);
  digitalWrite(d, 0);
  digitalWrite(e, 0);
  digitalWrite(f, 0);
  digitalWrite(g, 0);
  delay(100);
}

//Função para escrever o n° dois
void dois() {
  digitalWrite(a, 1);
  digitalWrite(b, 1);
  digitalWrite(c, 0);
  digitalWrite(d, 1);
  digitalWrite(e, 1);
  digitalWrite(f, 0);
  digitalWrite(g, 1);
  delay(100);
}

//Função para escrever o n° três
void tres() {
  digitalWrite(a, 1);
  digitalWrite(b, 1);
  digitalWrite(c, 1);
  digitalWrite(d, 1);
  digitalWrite(e, 0);
  digitalWrite(f, 0);
}

```

```

    digitalWrite(g, 1);
    delay(100);
}
//Função para escrever o n° quatro
void quatro() {
    digitalWrite(a, 0);
    digitalWrite(b, 1);
    digitalWrite(c, 1);
    digitalWrite(d, 0);
    digitalWrite(e, 0);
    digitalWrite(f, 1);
    digitalWrite(g, 1);
    delay(100);
}
//Função para escrever o n° cinco
void cinco() {
    digitalWrite(a, 1);
    digitalWrite(b, 0);
    digitalWrite(c, 1);
    digitalWrite(d, 1);
    digitalWrite(e, 0);
    digitalWrite(f, 1);
    digitalWrite(g, 1);
    delay(100);
}
//Função para escrever o n° seis
void seis() {
    digitalWrite(a, 1);
    digitalWrite(b, 0);
    digitalWrite(c, 1);
    digitalWrite(d, 1);
    digitalWrite(e, 1);
    digitalWrite(f, 1);
    digitalWrite(g, 1);
    delay(100);
}
//Função para escrever o n° sete
void sete() {
    digitalWrite(a, 1);
    digitalWrite(b, 1);
    digitalWrite(c, 1);
    digitalWrite(d, 0);
    digitalWrite(e, 0);
    digitalWrite(f, 0);
    digitalWrite(g, 0);
    delay(100);
}
//Função para escrever o n° oito
void oito() {
    digitalWrite(a, 1);
    digitalWrite(b, 1);
    digitalWrite(c, 1);
    digitalWrite(d, 1);
    digitalWrite(e, 1);
    digitalWrite(f, 1);
    digitalWrite(g, 1);
    delay(100);
}
//Função para escrever o n° nove
void nove() {
    digitalWrite(a, 1);
    digitalWrite(b, 1);
    digitalWrite(c, 1);
    digitalWrite(d, 1);

```

```

digitalWrite(e, 0);
digitalWrite(f, 1);
digitalWrite(g, 1);
delay(100);
}

void loop() {
    leitura = digitalRead(buttonPin); // Lê o estado de buttonPin e armazena
    em leitura
    if(leitura != ultleitura) { // Compara a leitura do botão com a leitura
    anterior
        if(leitura == HIGH) { // Se leitura for igual a HIGH
            contador++; // Incrementa contador em 1
        }
    }
    ultleitura = leitura; // Atribui a ultleitura o conteúdo de leitura

    switch (contador) {
        case 0:
            zero(); // Executa a função zero
            break;
        case 1:
            um(); // Executa a função um
            break;
        case 2:
            dois(); // Executa a função dois
            break;
        case 3:
            tres(); // Executa a função três
            break;
        case 4:
            quatro(); // Executa a função quatro
            break;
        case 5:
            cinco(); // Executa a função cinco
            break;
        case 6:
            seis(); // Executa a função seis
            break;
        case 7:
            sete(); // Executa a função sete
            break;
        case 8:
            oito(); // Executa a função oito
            break;
        case 9:
            nove(); // Executa a função nove
            break;
    }

    Serial.println(contador); // Imprime na serial o conteúdo de contador
    if(contador >= 10) { // Se contador for maior ou igual a 10
        contador = 0; // Retorna contador a zero
    }
}

```

TINKERCAD

Para melhor visualizar e/ou simular o circuito e a programação deste projeto basta se acessar o seguinte link: www.blogdarobotica.com/tinkercad-display7contador.

PROJETO CONTADOR DE 0 A 9 COM DISPLAY DE 7 SEGMENTOS ÂNODO COMUM

A proposta deste projeto é criar um contador incremental de 0 a 9 utilizando um display de 7 segmentos ânodo comum em conjunto com a placa UNO e um botão. Além disso, esse projeto servirá para colocar em prática a criação de funções e o uso da estrutura de controle de fluxo *switch...case*.

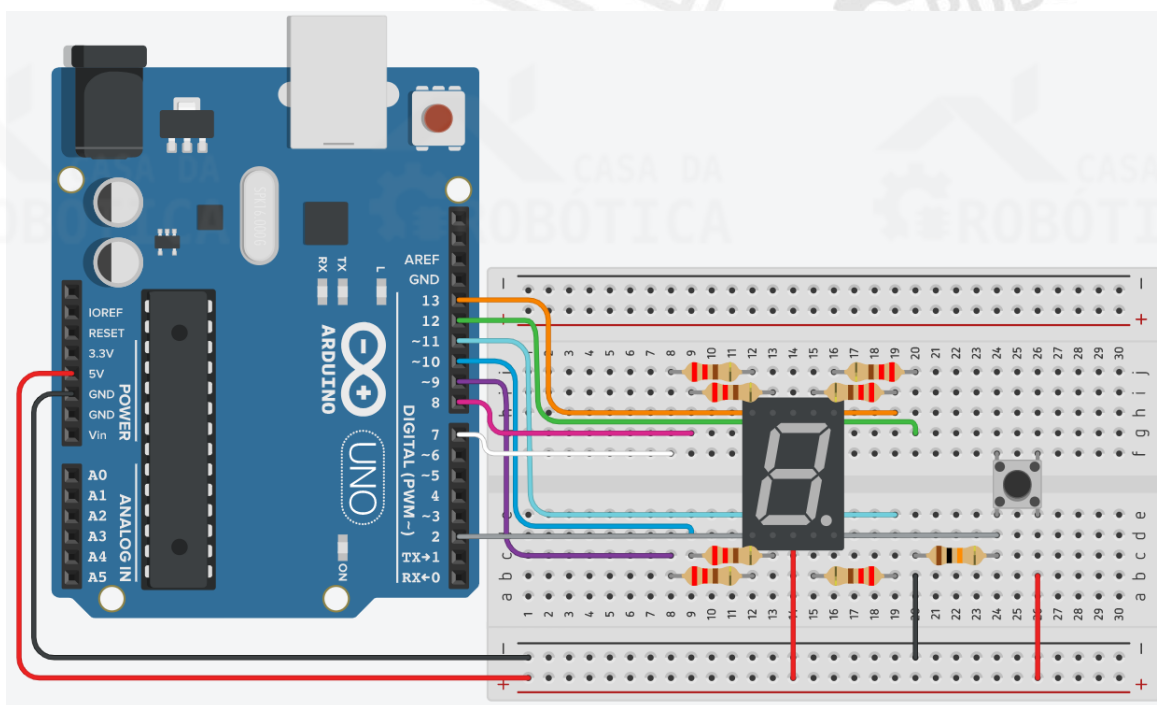
MATERIAIS NECESÁRIOS

- 1 x Placa UNO SMD R3 Atmega328 compatível com Arduino UNO;
- 1 x Cabo USB;
- 1 x Protoboard;
- 1 x Display de 7 segmentos ânodo comum;
- 7 x Resistor de 220 Ω ;
- 1 x Resistor de 10 k Ω ;
- 1 x Botão push button;
- Fios de jumper macho-macho.

ESQUEMÁTICO DE LIGAÇÃO DOS COMPONENTES

Inicialmente, certifique-se que a sua placa UNO esteja desligada. Monte o circuito da utilizando o display de 7 segmentos, o botão, os resistores e os fios.

Figura 54 - Circuito para projeto contador de 0 a 9 com display de 7 segmentos ânodo comum.



Ao montar seu circuito na protoboard preste atenção nos seguintes pontos:

- O display de 7 segmentos é do tipo ânodo comum, então um dos terminais comuns do mesmo deve ser conectado ao 5 V;
- Os segmentos do display de 7 segmentos são nomeados de **a** a **g**, em sentido horário, a partir do segmento superior do componente;
- Em cada segmento do display vamos conectar um resistor de 220 Ω ;
- Os segmentos encontram-se conectados as portas digitais da seguinte maneira:
a = 13; b = 12; c = 11; d = 10; e = 9; f = 8; g = 7;
- O botão push button deve ser conectado a uma resistência pull-down de 10 k Ω e a porta digital 2.

ELABORANDO O CÓDIGO

Com o circuito montado, vamos a programação do Sketch. Vamos entender a lógica de programação para o projeto contador de 0 a 9 com display de 7 segmentos ânodo comum a partir dos seguintes passos:

1- Declarar as variáveis

Iniciaremos a programação declarando as variáveis correspondentes ao controle dos segmentos do display. Conforme o esquemático elétrico, atribuiremos a porta 13 a variável **a**; a porta 12 a variável **b**; a porta 11 a variável **c**; a porta 10 a variável **d**; a porta 9 a variável **e**; a porta 8 a variável **f**; a porta 7 a variável **g**.

Em seguida, devemos declarar a variável correspondente ao botão e sua leitura. A porta 2 será atribuída a variável `buttonPin`. A variável `leitura`, do tipo inteiro, será responsável por receber a leitura do estado do botão e a variável `ultleitura`, também do tipo inteiro, será responsável por receber o valor da última leitura do botão. Deve ser declarada também a variável `contador`, do tipo inteiro, que terá sua função detalhada mais adiante.

2- Configurar as portas de entrada e saída e inicializar a comunicação serial
Na função `setup` configuraremos todos as variáveis conectadas aos segmentos do display 7 segmentos como saída, ou seja, as variáveis **a**, **b**, **c**, **d**, **e**, **f** e **g** devem ser configuradas como **OUTPUT**. Por sua vez, a variável `buttonPin` deve ser configurada como entrada (**INPUT**).

A comunicação serial será inicializada por meio da instrução:
`Serial.begin(9600);`

3- Criar as funções dos números a serem exibidos no display 7 segmentos
Na programação deste Sketch precisamos criar funções que serão responsáveis por acionar os segmentos do display para que ele exiba números de 0 a 9.
Desta forma, vamos criar uma função para cada número. Cada função conterá a combinação de segmentos que devem ser acionados (colocados em nível baixo) para que exiba o número desejado. Na Tabela 10 estão listados os segmentos que devem ser acionados para formação dos números de 0 a 9.

Tabela 11 - Número e sequência de segmentos.

Número	Sequência
0	a, b, c, d, e, f
1	b, c
2	a, b, d, e, g
3	a, b, c, d, g
4	b, c, f, g
5	a, c, d, f, g
6	a, c, d, e, f, g
7	a, b, c
8	a, b, c, d, e, f, g
9	a, b, c, f, g

Por exemplo, a função `zero()` conterá todas as instruções necessárias para acionamento dos segmentos do display para exibir o número zero. Conforme a tabela precisamos colocar em nível baixo as variáveis **a**, **b**, **c**, **d**, **e**, e **f** e colocar em nível alto a variável **g**. Sendo assim, a função `zero()` será escrita da seguinte forma:

```
void zero() {  
    digitalWrite(a, 0); //Coloca a em nível baixo  
    digitalWrite(b, 0); //Coloca b em nível baixo  
    digitalWrite(c, 0); //Coloca c em nível baixo  
    digitalWrite(d, 0); //Coloca d em nível baixo  
    digitalWrite(e, 0); //Coloca e em nível baixo  
    digitalWrite(f, 0); //Coloca f em nível baixo  
    digitalWrite(g, 1); //Coloca g em nível alto  
    delay(100); //Intervalo de 100 milissegundos  
}
```

4- Realizar a leitura do botão

Inicializaremos o loop com a leitura do botão, variável `buttonPin`, e atribuir o resultado da leitura a variável `leitura`. Isso será feito através da instrução `leitura=digitalRead(buttonPin);`

5- Comparar o valor de leitura com `ultleitura`

Neste projeto precisamos saber quantas mudanças de estado do botão incremental ocorreram, isso é chamado de detecção de mudança de estado ou detecção de borda. Para fazer essa detecção precisamos comparar o estado de leitura do botão com a leitura anterior e faremos isso através das seguintes instruções:

```
if(leitura != ultleitura) { //Compara a leitura do botão  
    com a leitura anterior  
    if(leitura == HIGH) { //Se leitura for igual a HIGH  
        contador++; //Incrementa contador em 1  
    }  
}  
ultleitura = leitura; //Atribui a ultleitura o conteúdo de  
leitura para a próxima execução do loop
```

6- Verificar o valor de contador

Utilizaremos a estrutura de controle de fluxo de seleção `switch...case` para comparar o valor armazenado na variável `contador` aos números especificados nos comandos cases. Quando o valor armazenado for igual ao caractere especificado no case, a função para exibição do número correspondente deve ser executado.

7- Imprimir na serial o valor de contador

Para ter um maior controle e comparar o valor armazenado na variável contador ao exibido no display 7 segmentos incluímos a instrução `Serial.println(contador);` para que seja exibido seu valor no monitor serial.

8- Comparar se o valor da variável contador é maior ou igual a 10

Como estamos utilizando apenas um display de 7 segmentos apenas podemos exibir números de 0 a 9, desta forma precisamos limitar o valor de contador para sempre que atingir um valor maior ou igual a 10 ele seja zerado. Para isso, utilizamos as seguintes instruções:

```
if(contador >= 10) { //Se contador for maior ou igual a 10
    contador = 0; //Retorna contador a zero
```

Ao fim, o Sketch ficará da seguinte forma:

```
int a = 13; //Correspondente ao LED a
int b = 12; //Correspondente ao LED b
int c = 11; //Correspondente ao LED c
int d = 10; //Correspondente ao LED d
int e = 9; //Correspondente ao LED e
int f = 8; //Correspondente ao LED f
int g = 7; //Correspondente ao LED g
int buttonPin = 2; //Correspondente ao botão
int leitura = 0; //Leitura do botão
int ultleitura = 0; //Última leitura do botão
int contador = 0; //Correspondente ao contador

void setup() {
    pinMode(a, OUTPUT); //Define a como saída
    pinMode(b, OUTPUT); //Define b como saída
    pinMode(c, OUTPUT); //Define c como saída
    pinMode(d, OUTPUT); //Define d como saída
    pinMode(e, OUTPUT); //Define e como saída
    pinMode(f, OUTPUT); //Define f como saída
    pinMode(g, OUTPUT); //Define g como saída
    pinMode(buttonPin, INPUT); //Define buttonPin como entrada
    Serial.begin(9600); //Inicia a comunicação serial
}
//Função para escrever o nº zero
void zero() {
    digitalWrite(a, 0); //coloca a em nível baixo
    digitalWrite(b, 0); //coloca b em nível baixo
    digitalWrite(c, 0); //coloca c em nível baixo
    digitalWrite(d, 0); //coloca d em nível baixo
    digitalWrite(e, 0); //coloca e em nível baixo
    digitalWrite(f, 0); //coloca f em nível baixo
    digitalWrite(g, 1); //coloca g em nível alto
    delay(100);
}
//Função para escrever o nº um
void um() {
    digitalWrite(a, 1);
    digitalWrite(b, 0);
    digitalWrite(c, 0);
    digitalWrite(d, 1);
```

```

    digitalWrite(e, 1);
    digitalWrite(f, 1);
    digitalWrite(g, 1);
    delay(100);
}
//Função para escrever o n° dois
void dois() {
    digitalWrite(a, 0);
    digitalWrite(b, 0);
    digitalWrite(c, 1);
    digitalWrite(d, 0);
    digitalWrite(e, 0);
    digitalWrite(f, 1);
    digitalWrite(g, 0);
    delay(100);
}
//Função para escrever o n° três
void tres() {
    digitalWrite(a, 0);
    digitalWrite(b, 0);
    digitalWrite(c, 0);
    digitalWrite(d, 0);
    digitalWrite(e, 1);
    digitalWrite(f, 1);
    digitalWrite(g, 0);
    delay(100);
}
//Função para escrever o n° quatro
void quatro() {
    digitalWrite(a, 1);
    digitalWrite(b, 0);
    digitalWrite(c, 0);
    digitalWrite(d, 1);
    digitalWrite(e, 1);
    digitalWrite(f, 0);
    digitalWrite(g, 0);
    delay(100);
}
//Função para escrever o n° cinco
void cinco() {
    digitalWrite(a, 0);
    digitalWrite(b, 1);
    digitalWrite(c, 0);
    digitalWrite(d, 0);
    digitalWrite(e, 1);
    digitalWrite(f, 0);
    digitalWrite(g, 0);
    delay(100);
}
//Função para escrever o n° seis
void seis() {
    digitalWrite(a, 0);
    digitalWrite(b, 1);
    digitalWrite(c, 0);
    digitalWrite(d, 0);
    digitalWrite(e, 0);
    digitalWrite(f, 0);
    digitalWrite(g, 0);
    delay(100);
}
//Função para escrever o n° sete
void sete() {
    digitalWrite(a, 0);
    digitalWrite(b, 0);

```

```

digitalWrite(c, 0);
digitalWrite(d, 1);
digitalWrite(e, 1);
digitalWrite(f, 1);
digitalWrite(g, 1);
delay(100);
}
//Função para escrever o n° oito
void oito() {
    digitalWrite(a, 0);
    digitalWrite(b, 0);
    digitalWrite(c, 0);
    digitalWrite(d, 0);
    digitalWrite(e, 0);
    digitalWrite(f, 0);
    digitalWrite(g, 0);
    delay(100);
}
//Função para escrever o n° nove
void nove() {
    digitalWrite(a, 0);
    digitalWrite(b, 0);
    digitalWrite(c, 0);
    digitalWrite(d, 0);
    digitalWrite(e, 1);
    digitalWrite(f, 0);
    digitalWrite(g, 0);
    delay(100);
}

void loop(){
    leitura = digitalRead(buttonPin); //Lê o estado de buttonPin e armazena
    em leitura
    if(leitura != ultleitura) { // Compara a leitura do botão com a leitura
    anterior
        if(leitura == HIGH) { //Se leitura for igual a HIGH
            contador++; //Incrementa contador em 1
        }
    }
    ultleitura = leitura; //Atribui a ultleitura o conteúdo de leitura

    switch (contador) {
        case 0:
            zero(); //Executa a função zero
            break;
        case 1:
            um(); //Executa a função um
            break;
        case 2:
            dois(); //Executa a função dois
            break;
        case 3:
            tres(); //Executa a função três
            break;
        case 4:
            quatro(); //Executa a função quatro
            break;
        case 5:
            cinco(); //Executa a função cinco
            break;
        case 6:
            seis(); //Executa a função seis
            break;
        case 7:

```



```

    sete();//Executa a função sete
    break;
case 8:
    oito();//Executa a função oito
    break;
case 9:
    nove();//Executa a função nove
    break;
}
Serial.println(contador);//Imprime na serial o conteúdo de contador
if(contador >= 10) { //Se contador for maior ou igual a 10
    contador = 0; //Retorna contador a zero
}
}

```

TINKERCAD

Para melhor visualizar e/ou simular o circuito e a programação deste projeto basta se acessar o seguinte link: www.blogdarobotica.com/tinkercad-display7contadoranodo.

PROJETO INCREMENTO E DECREMENTO – 0 A 9 COM DISPLAY DE 7 SEGMENTOS CÁTODO COMUM

A proposta deste projeto é criar um contador incremental e decremental utilizando um display de 7 segmentos cátodo comum em conjunto com a placa UNO e um botão.

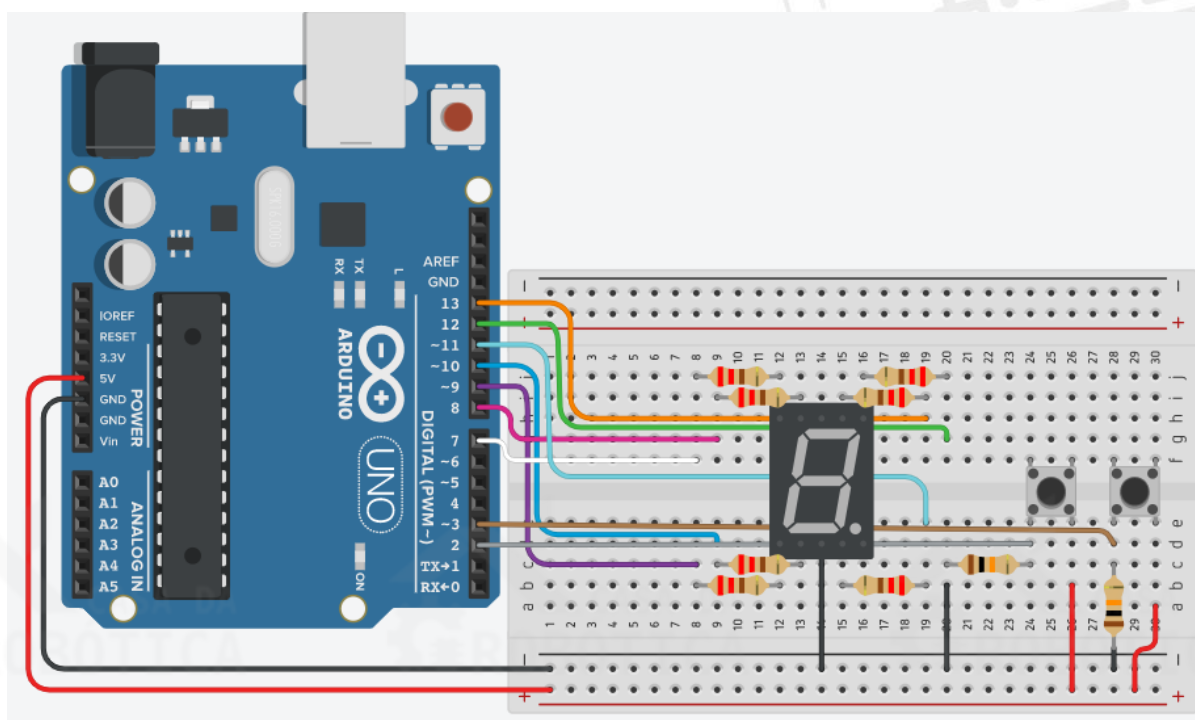
MATERIAIS NECESÁRIOS

- 1 x Placa UNO SMD R3 Atmega328 compatível com Arduino UNO;
- 1 x Cabo USB;
- 1 x Protoboard;
- 1 x Display de 7 segmentos cátodo comum;
- 7 x Resistor de 220 Ω ;
- 2 x Resistor de 10 k Ω ;
- 2 x Botão push button;
- Fios de jumper macho-macho.

ESQUEMÁTICO DE LIGAÇÃO DOS COMPONENTES

Certifique-se que a sua placa UNO esteja desligada e monte o circuito da Figura 55 utilizando o display de 7 segmentos, os botões, os resistores e os fios.

Figura 55 - Circuito para projeto incremento e decremento - 0 a 9 com display de 7 segmentos cátodo comum.



Ao montar seu circuito na protoboard preste atenção nos seguintes pontos:

- O display de 7 segmentos é do tipo cátodo comum, então um dos terminais comuns do mesmo deve ser conectado ao terra (GND);
- Os segmentos do display de 7 segmentos são nomeados de **a** a **g**, em sentido horário, a partir do segmento superior do componente;
- Em cada segmento do display vamos conectar um resistor de 220 Ω ;
- Os segmentos encontram-se conectado as portas digitais da seguinte maneira:
a = 13; b = 12; c = 11; d = 10; e = 9; f = 8; g = 7;
- O primeiro botão será usado para o incremento e deve ser conectado a uma resistência pull-down de 10 k Ω e a porta digital 2;
- O segundo botão será usado para o decremento e deve ser conectado a uma resistência pull-down de 10 k Ω e a porta digital 3.

ELABORANDO O CÓDIGO

O código para o contador incremental e decremental utilizando o display de 7 segmentos será bastante parecido ao código do projeto do contador incremental. Para melhor compreensão da lógica de programação desse código vamos aos seguintes passos:

1- Declarar as variáveis

Iniciaremos a programação declarando as variáveis correspondentes ao controle dos segmentos do display. Conforme o esquemático elétrico, atribuiremos a porta 13 a variável **a**; a porta 12 a variável **b**; a porta 11 a variável **c**; a porta 10 a variável **d**; a porta 9 a variável **e**; a porta 8 a variável **f**; a porta 7 a variável **g**.

Em seguida, devemos declarar a variável correspondente aos botões de incremento e decremento e suas leituras. A porta 2 será atribuída a variável `b1Pin` (botão de incremento). A variável `leitura1`, do tipo inteiro, será responsável por receber a leitura do estado do botão de incremento e a variável `ultleitura1`, também do tipo inteiro, será responsável por receber o valor da última leitura do botão de incremento.

A porta 3 deve ser atribuída a variável `b2Pin` (botão de decremento). A variável `leitura2`, do tipo inteiro, será responsável por receber a leitura do estado do botão de decremento e a variável `ultleitura2`, também do tipo inteiro, será responsável por receber o valor da última leitura do botão de decremento.

Deve ser declarada também a variável `contador`, do tipo inteiro.

2- Configurar as portas de entrada e saída e inicializar a comunicação serial

Na função `setup` configuraremos todas as variáveis conectadas aos segmentos do display 7 segmentos como saída, ou seja, as variáveis **a**, **b**, **c**, **d**, **e**, **f** e **g** devem ser configuradas como **OUTPUT**. Por sua vez, as variáveis `b1Pin` e `b2Pin` deve ser configurada como entrada (**INPUT**).

3- Criar as funções dos números a serem exibidos no display 7 segmentos

Assim como no projeto anterior, precisamos criar funções que serão responsáveis por acionar os segmentos do display para que ele exiba números de 0 a 9.

4- Realizar as leituras dos botões

Inicializaremos o loop com a leitura do botão incremental, variável `b1Pin`, e atribuir o resultado da leitura a variável `leitura1`. Isso será feito através da instrução `leitura1=digitalRead(b1Pin);`

Em seguida, realizaremos a leitura do botão decremental, variável `b2Pin`, e atribuir o resultado da leitura a variável `leitura2`. Isso será feito através da instrução `leitura2=digitalRead(b2Pin);`

5- Comparar o valor de `leitura1` com `ultleitura1` e de `leitura2` com `ultleitura1`

Neste projeto precisamos saber quantas mudanças de estado dos botões incremental e decremental ocorreram. Para fazer essa detecção precisamos comparar o estado de leitura do botão com a leitura anterior.

6- Verificar o valor de contador

Utilizaremos a estrutura de controle de fluxo de seleção `switch...case` para comparar o valor armazenado na variável `contador` aos números especificados nos comandos `cases`. Quando o valor armazenado for igual ao caractere especificado no `case`, o código para exibição do número correspondente deve ser executado.

7- Imprimir na serial o valor de contador

Para ter um maior controle e comparar o valor armazenado na variável `contador` ao exibido no display 7 segmentos incluímos a instrução `Serial.println(contador);` para que seja exibido seu valor no monitor serial.

8- Comparar se o valor da variável `contador` é maior ou igual a 10 e se `contador` é menor ou igual a -1

Como estamos utilizando apenas um display de 7 segmentos apenas podemos exibir números de 0 a 9, desta forma precisamos limitar o valor de `contador` para sempre que atingir um valor maior ou igual a 10 e menor ou igual a -1 ele seja zerado. Para isso, utilizamos as seguintes instruções:

```
if (contador >= 10) { //Se contador for maior ou igual a 10
    contador = 0; //Atribui 0 a contador
}
if (contador <= -1) { //Se contador for menor ou igual a -1
    contador = 9; //Atribui 9 a contador
}
```

Com isso, o código deste projeto fica da seguinte forma:

```
int a = 13; //Correspondente ao LED a
int b = 12; //Correspondente ao LED b
int c = 11; //Correspondente ao LED c
int d = 10; //Correspondente ao LED d
int e = 9; //Correspondente ao LED e
```

```

int f = 8; //Correspondente ao LED f
int g = 7; //Correspondente ao LED g
int b1Pin = 2; //Correspondente ao botão +
int b2Pin = 3; //Correspondente ao botão -
int leitura1 = 0; //Leitura do botão +
int leitura2 = 0; //Leitura do botão -
int ultleitura1 = 0; //Última leitura do botão +
int ultleitura2 = 0; //Última leitura do botão -
int contador = 0; //Correspondente ao contador

void setup() {
  pinMode(a, OUTPUT); //Define a como saída
  pinMode(b, OUTPUT); //Define b como saída
  pinMode(c, OUTPUT); //Define c como saída
  pinMode(d, OUTPUT); //Define d como saída
  pinMode(e, OUTPUT); //Define e como saída
  pinMode(f, OUTPUT); //Define f como saída
  pinMode(g, OUTPUT); //Define g como saída
  pinMode(b1Pin, INPUT); //Define b1Pin como entrada
  pinMode(b2Pin, INPUT); //Define b2Pin como entrada
  Serial.begin(9600); //Inicia a comunicação serial
}

//Função para escrever o n° zero
void zero() {
  digitalWrite(a, 1);
  digitalWrite(b, 1);
  digitalWrite(c, 1);
  digitalWrite(d, 1);
  digitalWrite(e, 1);
  digitalWrite(f, 1);
  digitalWrite(g, 0);
  delay(100);
}

//Função para escrever o n° um
void um() {
  digitalWrite(a, 0);
  digitalWrite(b, 1);
  digitalWrite(c, 1);
  digitalWrite(d, 0);
  digitalWrite(e, 0);
  digitalWrite(f, 0);
  digitalWrite(g, 0);
  delay(100);
}

//Função para escrever o n° dois
void dois() {
  digitalWrite(a, 1);
  digitalWrite(b, 1);
  digitalWrite(c, 0);
  digitalWrite(d, 1);
  digitalWrite(e, 1);
  digitalWrite(f, 0);
  digitalWrite(g, 1);
  delay(100);
}

//Função para escrever o n° três
void tres() {
  digitalWrite(a, 1);
  digitalWrite(b, 1);
  digitalWrite(c, 1);
  digitalWrite(d, 1);
  digitalWrite(e, 0);
  digitalWrite(f, 0);
  digitalWrite(g, 1);
}

```

```

    delay(100);
}
//Função para escrever o n° quatro
void quatro() {
    digitalWrite(a, 0);
    digitalWrite(b, 1);
    digitalWrite(c, 1);
    digitalWrite(d, 0);
    digitalWrite(e, 0);
    digitalWrite(f, 1);
    digitalWrite(g, 1);
    delay(100);
}
//Função para escrever o n° cinco
void cinco() {
    digitalWrite(a, 1);
    digitalWrite(b, 0);
    digitalWrite(c, 1);
    digitalWrite(d, 1);
    digitalWrite(e, 0);
    digitalWrite(f, 1);
    digitalWrite(g, 1);
    delay(100);
}
//Função para escrever o n° seis
void seis() {
    digitalWrite(a, 1);
    digitalWrite(b, 0);
    digitalWrite(c, 1);
    digitalWrite(d, 1);
    digitalWrite(e, 1);
    digitalWrite(f, 1);
    digitalWrite(g, 1);
    delay(100);
}
//Função para escrever o n° sete
void sete() {
    digitalWrite(a, 1);
    digitalWrite(b, 1);
    digitalWrite(c, 1);
    digitalWrite(d, 0);
    digitalWrite(e, 0);
    digitalWrite(f, 0);
    digitalWrite(g, 0);
    delay(100);
}
//Função para escrever o n° oito
void oito() {
    digitalWrite(a, 1);
    digitalWrite(b, 1);
    digitalWrite(c, 1);
    digitalWrite(d, 1);
    digitalWrite(e, 1);
    digitalWrite(f, 1);
    digitalWrite(g, 1);
    delay(100);
}
//Função para escrever o n° nove
void nove() {
    digitalWrite(a, 1);
    digitalWrite(b, 1);
    digitalWrite(c, 1);
    digitalWrite(d, 1);
    digitalWrite(e, 0);

```



```

digitalWrite(f, 1);
digitalWrite(g, 1);
delay(100);
}

void loop(){
    leitura1 = digitalRead(b1Pin); // Lê o estado de b1Pin e armazena em
    leitura1
    leitura2 = digitalRead(b2Pin); // Lê o estado de b2Pin e armazena em
    leitura2
    if (leitura1 != ultleitura1) { // Se leitura1 não for igual a ultleitura1
        if (leitura1 == HIGH) { // Se leitura1 for igual a HIGH
            contador++; // Incrementa contador em 1
        }
    }
    ultleitura1 = leitura1; // Atribui a ultleitura1 o conteúdo de leitura1

    if (leitura2 != ultleitura2) { // Se leitura2 não for igual a ultleitura2
        if (leitura2 == HIGH) { // Se leitura2 for igual a HIGH
            contador--; // Decrementa contador em 1
        }
    }
    ultleitura2 = leitura2; // Atribui a ultleitura2 o conteúdo de leitura2

    switch (contador) {
        case 0:
            zero(); // Executa a função zero
            break;
        case 1:
            um(); // Executa a função um
            break;
        case 2:
            dois(); // Executa a função dois
            break;
        case 3:
            tres(); // Executa a função três
            break;
        case 4:
            quatro(); // Executa a função quatro
            break;
        case 5:
            cinco(); // Executa a função cinco
            break;
        case 6:
            seis(); // Executa a função seis
            break;
        case 7:
            sete(); // Executa a função sete
            break;
        case 8:
            oito(); // Executa a função oito
            break;
        case 9:
            nove(); // Executa a função nove
            break;
    }
    Serial.println(contador); // Imprime na serial o conteúdo de contador
    if (contador >= 10) { // Se contador for maior ou igual a 10
        contador = 0; // Atribui 0 a contador
    }
    if (contador <= -1) { // Se contador for menor ou igual a -1
        contador = 9; // Atribui 9 a contador
    }
}
}

```

TINKERCAD

Para melhor visualizar e/ou simular o circuito e a programação deste projeto basta se acessar o seguinte link: www.blogdarobotica.com/tinkercad-display7incrementodecremento.

PROJETO INCREMENTO E DECREMENTO – 0 A 9 COM DISPLAY DE 7 SEGMENTOS ÂNODO COMUM

A proposta deste projeto é criar um contador incremental e decremental utilizando um display de 7 segmentos ânodo comum em conjunto com a placa UNO e um botão.

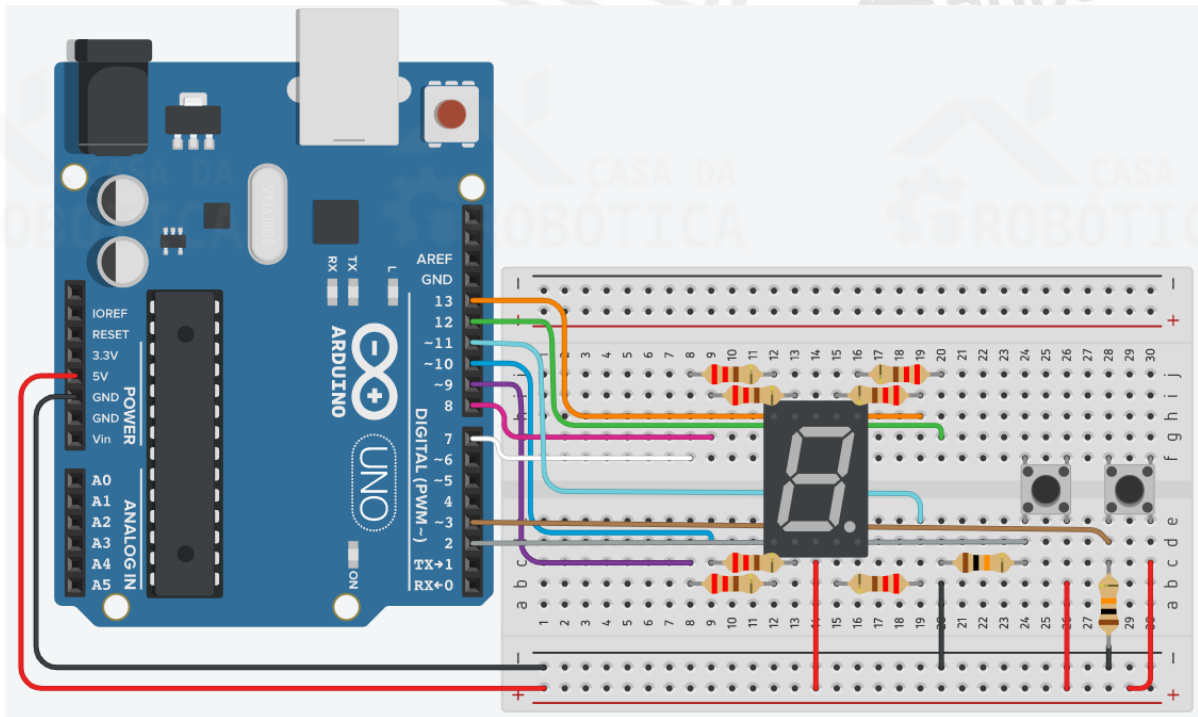
MATERIAIS NECESÁRIOS

- 1 x Placa UNO SMD R3 Atmega328 compatível com Arduino UNO;
- 1 x Cabo USB;
- 1 x Protoboard;
- 1 x Display de 7 segmentos ânodo comum;
- 7 x Resistor de 220 Ω ;
- 2 x Resistor de 10 k Ω ;
- 2 x Botão push button;
- Fios de jumper macho-macho.

ESQUEMÁTICO DE LIGAÇÃO DOS COMPONENTES

Certifique-se que a sua placa UNO esteja desligada e monte o circuito da Figura 55 utilizando o display de 7 segmentos, os botões, os resistores e os fios.

Figura 56 - Circuito para projeto incremento e decremento - 0 a 9 com display de 7 segmentos ânodo comum.



Ao montar seu circuito na protoboard preste atenção nos seguintes pontos:

- O display de 7 segmentos é do tipo ânodo comum, então um dos terminais comuns do mesmo deve ser conectado ao 5V;
- Os segmentos do display de 7 segmentos são nomeados de **a** a **g**, em sentido horário, a partir do segmento superior do componente;
- Em cada segmento do display vamos conectar um resistor de 220 Ω ;
- Os segmentos encontram-se conectado as portas digitais da seguinte maneira:
a = 13; b = 12; c = 11; d = 10; e = 9; f = 8; g = 7;
- O primeiro botão será usado para o incremento e deve ser conectado a uma resistência pull-down de 10 k Ω e a porta digital 2;
- O segundo botão será usado para o decremento e deve ser conectado a uma resistência pull-down de 10 k Ω e a porta digital 3.

ELABORANDO O CÓDIGO

O código para o contador incremental e decremental utilizando o display de 7 segmentos será bastante parecido ao código do projeto do contador incremental. Para

melhor compreensão da lógica de programação desse código, vamos aos seguintes passos:

1- Declarar as variáveis

Iniciaremos a programação declarando as variáveis correspondentes ao controle dos segmentos do display. Conforme o esquemático elétrico, atribuiremos a porta 13 a variável **a**; a porta 12 a variável **b**; a porta 11 a variável **c**; a porta 10 a variável **d**; a porta 9 a variável **e**; a porta 8 a variável **f**; a porta 7 a variável **g**.

Em seguida, devemos declarar a variável correspondente aos botões de incremento e decremento e suas leituras. A porta 2 será atribuída a variável **b1Pin** (botão de incremento). A variável **leitura1**, do tipo inteiro, será responsável por receber a leitura do estado do botão de incremento e a variável **ultleitura1**, também do tipo inteiro, será responsável por receber o valor da última leitura do botão de incremento.

A porta 3 deve ser atribuída a variável **b2Pin** (botão de decremento). A variável **leitura2**, do tipo inteiro, será responsável por receber a leitura do estado do botão de decremento e a variável **ultleitura2**, também do tipo inteiro, será responsável por receber o valor da última leitura do botão de decremento. Deve ser declarada também a variável **contador**, do tipo inteiro.

2- Configurar as portas de entrada e saída e inicializar a comunicação serial

Na função **setup** configuraremos todas as variáveis conectadas aos segmentos do display 7 segmentos como saída, ou seja, as variáveis **a**, **b**, **c**, **d**, **e**, **f** e **g** devem ser configuradas como **OUTPUT**. Por sua vez, as variáveis **b1Pin** e **b2Pin** deve ser configurada como entrada (**INPUT**).

3- Criar as funções dos números a serem exibidos no display 7 segmentos

Assim como no projeto anterior, precisamos criar funções que serão responsáveis por acionar os segmentos do display para que ele exiba números de 0 a 9.

4- Realizar as leituras dos botões

Inicializaremos o loop com a leitura do botão incremental, variável **b1Pin**, e atribuir o resultado da leitura a variável **leitura1**. Isso será feito através da instrução **leitura1=digitalRead(b1Pin)** ;

Em seguida, realizaremos a leitura do botão decremental, variável `b2Pin`, e atribuir o resultado da leitura a variável `leitura2`. Isso será feito através da instrução `leitura2=digitalRead(b2Pin);`

5- Comparar o valor de `leitura1` com `ultleitura1` e de `leitura2` com `ultleitura1`

Neste projeto precisamos saber quantas mudanças de estado dos botões incremental e decremental ocorreram. Para fazer essa detecção precisamos comparar o estado de leitura do botão com a leitura anterior.

6- Verificar o valor de contador

Utilizaremos a estrutura de controle de fluxo de seleção `switch...case` para comparar o valor armazenado na variável `contador` aos números especificados nos comandos `cases`. Quando o valor armazenado for igual ao caractere especificado no `case`, o código para exibição do número correspondente deve ser executado.

7- Imprimir na serial o valor de contador

Para ter um maior controle e comparar o valor armazenado na variável `contador` ao exibido no display 7 segmentos incluímos a instrução `Serial.println(contador);` para que seja exibido seu valor no monitor serial.

8- Comparar se o valor da variável `contador` é maior ou igual a 10 e se `contador` é menor ou igual a -1

Como estamos utilizando apenas um display de 7 segmentos apenas podemos exibir números de 0 a 9, desta forma precisamos limitar o valor de `contador` para sempre que atingir um valor maior ou igual a 10 e menor ou igual a -1 ele seja zerado.

Para isso, utilizamos as seguintes instruções:

```
if (contador >= 10) { //Se contador for maior ou igual a 10
    contador = 0; //Atribui 0 a contador
}
if (contador <= -1) { //Se contador for menor ou igual a -1
    contador = 9; //Atribui 9 a contador
}
```

Com isso, o código deste projeto fica da seguinte forma:

```
int a = 13; //Correspondente ao LED a
int b = 12; //Correspondente ao LED b
int c = 11; //Correspondente ao LED c
int d = 10; //Correspondente ao LED d
int e = 9; //Correspondente ao LED e
int f = 8; //Correspondente ao LED f
int g = 7; //Correspondente ao LED g
int b1Pin = 2; //Correspondente ao botão +
int b2Pin = 3; //Correspondente ao botão -
int leitura1 = 0; //Leitura do botão +
int leitura2 = 0; //Leitura do botão -
```

```

int ultleitura1 = 0; //Última leitura do botão +
int ultleitura2 = 0; //Última leitura do botão -
int contador = 0; //Correspondente ao contador

void setup() {
  pinMode(a, OUTPUT); //Define a como saída
  pinMode(b, OUTPUT); //Define b como saída
  pinMode(c, OUTPUT); //Define c como saída
  pinMode(d, OUTPUT); //Define d como saída
  pinMode(e, OUTPUT); //Define e como saída
  pinMode(f, OUTPUT); //Define f como saída
  pinMode(g, OUTPUT); //Define g como saída
  pinMode(b1Pin, INPUT); //Define b1Pin como entrada
  pinMode(b2Pin, INPUT); //Define b2Pin como entrada
  Serial.begin(9600); //Inicia a comunicação serial
}

//Função para escrever o n° zero
void zero() {
  digitalWrite(a, 0);
  digitalWrite(b, 0);
  digitalWrite(c, 0);
  digitalWrite(d, 0);
  digitalWrite(e, 0);
  digitalWrite(f, 0);
  digitalWrite(g, 1);
  delay(100);
}

//Função para escrever o n° um
void um() {
  digitalWrite(a, 1);
  digitalWrite(b, 0);
  digitalWrite(c, 0);
  digitalWrite(d, 1);
  digitalWrite(e, 1);
  digitalWrite(f, 1);
  digitalWrite(g, 1);
  delay(100);
}

//Função para escrever o n° dois
void dois() {
  digitalWrite(a, 0);
  digitalWrite(b, 0);
  digitalWrite(c, 1);
  digitalWrite(d, 0);
  digitalWrite(e, 0);
  digitalWrite(f, 1);
  digitalWrite(g, 0);
  delay(100);
}

//Função para escrever o n° três
void tres() {
  digitalWrite(a, 0);
  digitalWrite(b, 0);
  digitalWrite(c, 0);
  digitalWrite(d, 0);
  digitalWrite(e, 1);
  digitalWrite(f, 1);
  digitalWrite(g, 0);
  delay(100);
}

//Função para escrever o n° quatro
void quatro() {
  digitalWrite(a, 1);
  digitalWrite(b, 0);

```



```

    digitalWrite(c, 0);
    digitalWrite(d, 1);
    digitalWrite(e, 1);
    digitalWrite(f, 0);
    digitalWrite(g, 0);
    delay(100);
}
//Função para escrever o n° cinco
void cinco() {
    digitalWrite(a, 0);
    digitalWrite(b, 1);
    digitalWrite(c, 0);
    digitalWrite(d, 0);
    digitalWrite(e, 1);
    digitalWrite(f, 0);
    digitalWrite(g, 0);
    delay(100);
}
//Função para escrever o n° seis
void seis() {
    digitalWrite(a, 0);
    digitalWrite(b, 1);
    digitalWrite(c, 0);
    digitalWrite(d, 0);
    digitalWrite(e, 0);
    digitalWrite(f, 0);
    digitalWrite(g, 0);
    delay(100);
}
//Função para escrever o n° sete
void sete() {
    digitalWrite(a, 0);
    digitalWrite(b, 0);
    digitalWrite(c, 0);
    digitalWrite(d, 1);
    digitalWrite(e, 1);
    digitalWrite(f, 1);
    digitalWrite(g, 1);
    delay(100);
}
//Função para escrever o n° oito
void oito() {
    digitalWrite(a, 0);
    digitalWrite(b, 0);
    digitalWrite(c, 0);
    digitalWrite(d, 0);
    digitalWrite(e, 0);
    digitalWrite(f, 0);
    digitalWrite(g, 0);
    delay(100);
}
//Função para escrever o n° nove
void nove() {
    digitalWrite(a, 0);
    digitalWrite(b, 0);
    digitalWrite(c, 0);
    digitalWrite(d, 0);
    digitalWrite(e, 1);
    digitalWrite(f, 0);
    digitalWrite(g, 0);
    delay(100);
}
}

void loop() {

```

```

    leitura1 = digitalRead(b1Pin); // Lê o estado de b1Pin e armazena em
    leitura1
    leitura2 = digitalRead(b2Pin); // Lê o estado de b2Pin e armazena em
    leitura2
    if (leitura1 != ultleitura1) { // Se leitura1 não for igual a ultleitura1
        if (leitura1 == HIGH) { // Se leitura1 for igual a HIGH
            contador++; // Incrementa contador em 1
        }
    }
    ultleitura1 = leitura1; // Atribui a ultleitura1 o conteúdo de leitura1

    if (leitura2 != ultleitura2) { // Se leitura2 não for igual a ultleitura2
        if (leitura2 == HIGH) { // Se leitura2 for igual a HIGH
            contador--; // Decrementa contador em 1
        }
    }
    ultleitura2 = leitura2; // Atribui a ultleitura2 o conteúdo de leitura2

    switch (contador) {
        case 0:
            zero(); // Executa a função zero
            break;
        case 1:
            um(); // Executa a função um
            break;
        case 2:
            dois(); // Executa a função dois
            break;
        case 3:
            tres(); // Executa a função três
            break;
        case 4:
            quatro(); // Executa a função quatro
            break;
        case 5:
            cinco(); // Executa a função cinco
            break;
        case 6:
            seis(); // Executa a função seis
            break;
        case 7:
            sete(); // Executa a função sete
            break;
        case 8:
            oito(); // Executa a função oito
            break;
        case 9:
            nove(); // Executa a função nove
            break;
    }
    Serial.println(contador); // Imprime na serial o conteúdo de contador
    if (contador >= 10) { // Se contador for maior ou igual a 10
        contador = 0; // Atribui 0 a contador
    }
    if (contador <= -1) { // Se contador for menor ou igual a -1
        contador = 9; // Atribui 9 a contador
    }
}

```

TINKERCAD

Para melhor visualizar e/ou simular o circuito e a programação deste projeto basta se acessar o seguinte link: www.blogdarobotica.com/tinkercad-display7incrementodecrementoAnodo.

PROJETO ILUMINAÇÃO SEQUENCIAL COM LEDS

A proposta desse projeto é criar uma sequência de LEDs para realizar um efeito de iluminação sequencial. Neste projeto aprenderemos o conceito de vetor (ou arrays) e a forma de utilização da estrutura de controle de fluxo de repetição *while*.

Um vetor, ou array, é uma coleção de variáveis que são acessadas com um número índice, é uma forma de criar uma lista de valores. As variáveis que encontramos até agora tinham apenas um único valor, usando um vetor podemos criar uma variável com um conjunto de valores que podem ser acessados dando sua posição dentro da lista.

Para conhecer mais sobre esse tipo de dado, sua sintaxe e aplicação acesse: <https://www.arduino.cc/reference/pt/language/variables/data-types/array/>.

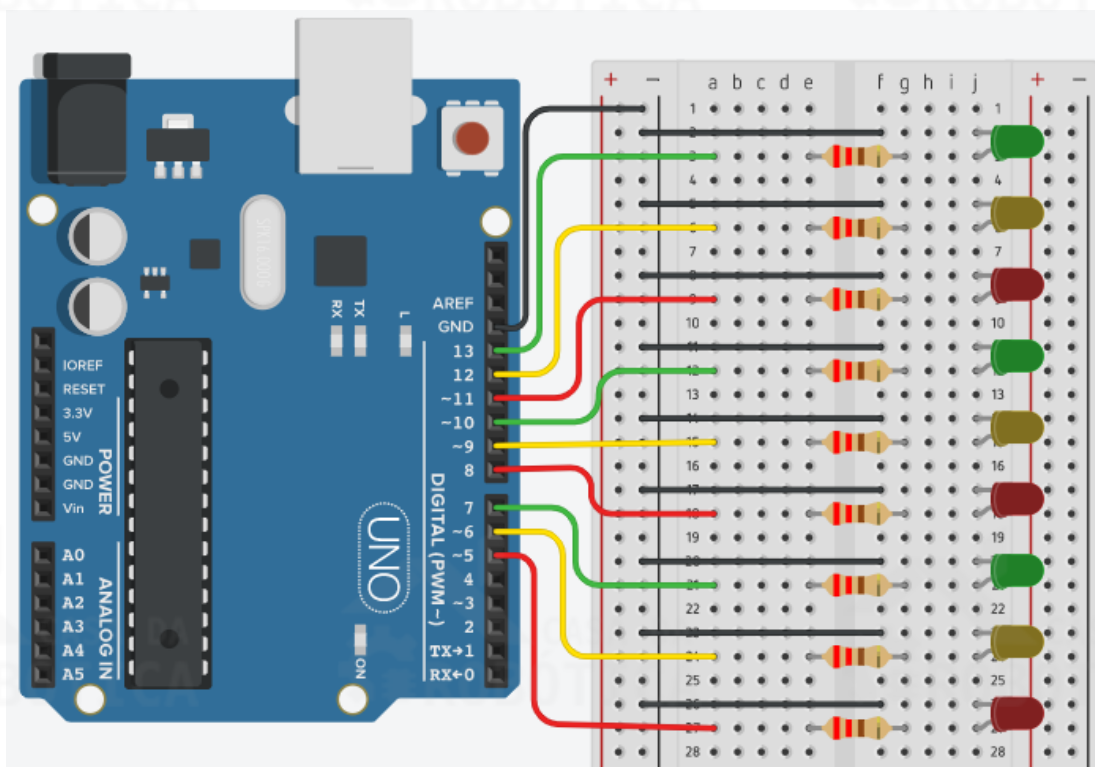
MATERIAIS NECESÁRIOS

- 1 x Placa UNO SMD R3 Atmega328 compatível com Arduino UNO;
- 1 x Cabo USB;
- 1 x Protoboard;
- 9 x LEDs difuso;
- 9 x Resistores de 220 Ω ;
- Fios de jumper macho-macho.

ESQUEMÁTICO DE LIGAÇÃO DOS COMPONENTES

Conecte os LEDs e os resistores na protoboard e na placa UNO conforme a Figura 57. Verifique cuidadosamente os cabos de ligação antes de ligar sua placa UNO. Lembre-se que a placa UNO deve estar desconectada enquanto você monta o circuito.

Figura 57 - Circuito para projeto iluminação sequencial com LEDs.



Ao montar seu circuito na protoboard preste atenção nos seguintes pontos:

- Atente-se a polaridade dos LEDs. Os terminais maiores tem polaridade positiva, que devem ser conectados aos pinos digitais da placa UNO. Os lados chanfrados dos LEDs possuem polaridade negativa e devem ser conectados ao GND;

ELABORANDO O CÓDIGO

A proposta desse projeto é criar um efeito sequencial numa fila de 9 LEDs, ou seja, os LEDs deverão ser acionados um após o outro. Para melhor compreensão da lógica de programação desse código vamos aos seguintes passos:

- 1- Declarar as variáveis

Assim como nos projetos anteriores, iniciaremos a programação deste projeto declarando as variáveis. No entanto, faremos isso de uma maneira diferente, através da instrução: `byte ledPin[] = {5, 6, 7, 8, 9, 10, 11, 12, 13};`. Essa declaração de variável é do tipo array. Conforme mencionado anteriormente, um array é uma lista de variáveis que podem ser acessadas utilizando um número de índice. O nome do array criado nesta instrução é `ledPin` do tipo `byte`.

O array criado possui 9 elementos que representarão os pinos digitais de 5 a 13. Para acessar um elemento desse array vamos nos referir ao número de índice desse elemento. Os arrays são indexados a partir de zero, ou seja, o primeiro índice é zero, e não um. Assim, em nosso array de 9 elementos os índices vão de 0 a 8. Nesta situação, o elemento 0 (`ledPin[0]`) tem em seu conteúdo o valor 5, o elemento 1 (`ledPin[1]`) tem em seu conteúdo o valor 6, e assim sucessivamente.

2- Configurar os pinos de saída

No `setup`, vamos configurar todos os pinos como saída. Faremos isso percorrendo os elementos do array e os configurando como saída (`OUTPUT`) com o auxílio da estrutura de repetição `for`.

3- Criar variável para estrutura de controle

Iniciaremos o loop criando a variável que será utilizada pela estrutura de controle de fluxo de repetição `while`, através da instrução: `int x = 0;`

4- Estrutura `while`

Para realizar o acionamento e desligamento dos LEDs de forma sequencial utilizaremos duas a estrutura de repetição `while`. Na primeira, realizaremos um teste para verificar se o valor de `x` é menor que 9. Enquanto (`while`) o valor de `x` for menor que 9, o LED na posição `x` será acionado e, após o intervalo de 150 milissegundos, será desligado novamente. Após isso, `x` é incrementado em uma unidade.

As instruções dentro do laço `while` serão repetidas 9 vezes e `x` assumirá valores de 0 a 8, que serão utilizado como índices dos elementos do array. Quando `x` for maior que 9, o programa saíra do primeiro laço de repetição `while` e seguirá para o próximo. Na segunda estrutura de repetição `while`, realizaremos um teste para verificar se o valor de `x` é maior que zero. Enquanto (`while`) o valor de `x` for maior que 0, o LED na posição `x` será acionado e, após o intervalo de 150 milissegundos, será desligado novamente. Ao fim, `x` será decrementado em uma unidade.

```

byte ledPin[] = {5, 6, 7, 8, 9, 10, 11, 12, 13}; //Array de 9 elementos
para os pinos dos LEDs

void setup() {
    for (int x = 0; x < 9; x++) { //Condicional para percorrer os elementos
do array
        pinMode(ledPin[x], OUTPUT); //Define todos os pinos como saída de
acordo com seus índices
    }
}

void loop() {
    int x = 0; //Variável que será utilizada na estrutura de controle
    while (x < 9) { //Enquanto x for menor que 9
        digitalWrite(ledPin[x], HIGH); //Aciona o LED da posição x do array
        delay(150); //Intervalo de 150 milissegundos
        digitalWrite(ledPin[x], LOW); //Desliga o LED da posição x do array
        x = ++x; //Incrementa x em um e retorna o novo valor de x
    }
    while (x > 0) { //Enquanto x for maior que 0
        digitalWrite(ledPin[x], HIGH); //Aciona o LED da posição x do array
        delay(150); //Intervalo de 150 milissegundos
        digitalWrite(ledPin[x], LOW); //Desliga o LED da posição x do array
        x = --x; //Decrementa x em um e retorna o novo valor de x
    }
}

```

TINKERCAD

Para melhor visualizar e/ou simular o circuito e a programação deste projeto basta se acessar o seguinte link: www.blogdarobotica.com/tinkercad-LEDsequencial.

PROJETO MEDIR TEMPERATURA DO AMBIENTE COM TERMISTOR NTC

O intuito deste projeto é utilizar o sensor de temperatura termistor NTC de 10 K para medir a temperatura do ambiente em conjunto com o Arduino. O valor da temperatura medida deverá ser exibido no monitor serial do software Arduino IDE.

Aproveitaremos este projeto para aprender como instalar uma biblioteca no Arduino IDE. Para isso, vamos utilizar biblioteca “Thermistor.h” que implementa as funcionalidades de um termistor NTC e suas aplicações de forma mais simples.

MATERIAIS NECESÁRIOS

- 1 x Placa UNO SMD R3 Atmega328 compatível com Arduino UNO;

Figura 58. Verifique o O. Lembre-se que a

uito.

e com o termistor

Figura 58 - Circuito para projeto medir temperatura do ambiente com o termistor NTC.

- As

- Um terminal do termistor NTC deve ser conectado ao 5 V e o outro ao pino analógico da placa UNO, neste caso usamos o pino A4. Conectamos também uma resistência de 10 kΩ entre o pino A4 e o GND da placa UNO.

ELABORANDO O CÓDIGO

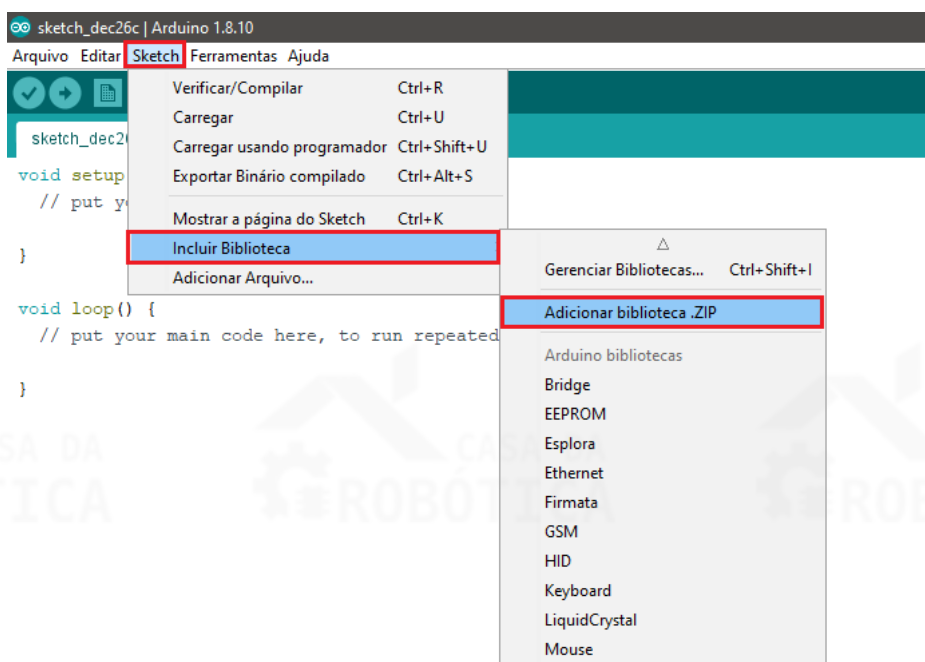
O principal intuito deste projeto é a utilização do termistor NTC de 10K para medir a temperatura do ambiente. No entanto, vamos mostrar como adicionar uma biblioteca no Arduino IDE. Uma das grandes vantagens na utilização das placas Arduino é a diversidade de bibliotecas disponíveis gratuitamente que podem ser utilizadas na construção de seus projetos.

Desta forma, vamos proceder a instalação da biblioteca específica para utilização do termistor. Esta biblioteca encontra-se disponível para download no seguinte link:

www.blogdarobotica.com/bibliotecas/thermistor

Após realizar o download da biblioteca, vamos instalá-la por meio do seguinte caminho: Toolbar > Sketch > Incluir biblioteca > Adicionar biblioteca ZIP, conforme ilustra a Figura 59.

Figura 59 - Caminho para incluir biblioteca no Arduino IDE.



Com a biblioteca instalada, feche o Arduino IDE e abra-o novamente. Feito isso, vamos a programação do nosso Sketch:

O primeiro passo é a inclusão da biblioteca do termistor no editor de texto por meio da instrução `#include <Thermistor.h>`.

Em seguida, criamos o objeto `temp` do tipo `Thermistor` e atribuímos os sinais do pino A4 a ele. Este objeto receberá os dados brutos vindos da porta analógica A4;

No `setup`, inicializamos a comunicação serial por meio da instrução: `Serial.begin(9600)`.

No `loop`, criamos a variável inteira `temperature` que será responsável por armazenar o valor da temperatura calculada pela biblioteca.

Logo após, imprimimos no monitor serial o texto "Temperatura: ", o valor da temperatura calculada e armazenada na variável `temperature` e o texto °C.

Por fim, é dado um intervalo de 1 segundo entre as leituras;

O programa do projeto proposto encontra-se detalhado a seguir:

```
#include<Thermistor.h>//Inclusão da biblioteca Thermistor

Thermistor temp(4);//Atribui o pino analógico A4, em que o termistor está
conectado, a variável temp
void setup() {
    Serial.begin(9600);//Inicializa a comunicação serial
}
void loop() {
    int temperature = temp.getTemp();//Variável do tipo inteiro que recebe o
valor da temperatura calculado pela biblioteca
    Serial.print("Temperatura: ");//Imprime na serial o texto "Temperatura"
    Serial.print(temperature);//Imprime na serial o valor da temperatura
calculada
    Serial.println("°C");//Imprime na serial "°C"
    delay(1000);//Intervalo de 1 segundo
}
```

PROJETO DETECTAR LINHA COM SENSOR INFRAVERMELHO

A proposta deste projeto é utilizar o sensor infravermelho em conjunto com a placa UNO para detectar linha. O infravermelho utilizado será o sensor seguidor de linha TCRT 5000, componente comumente aplicado em projetos de robôs seguidor de linha.

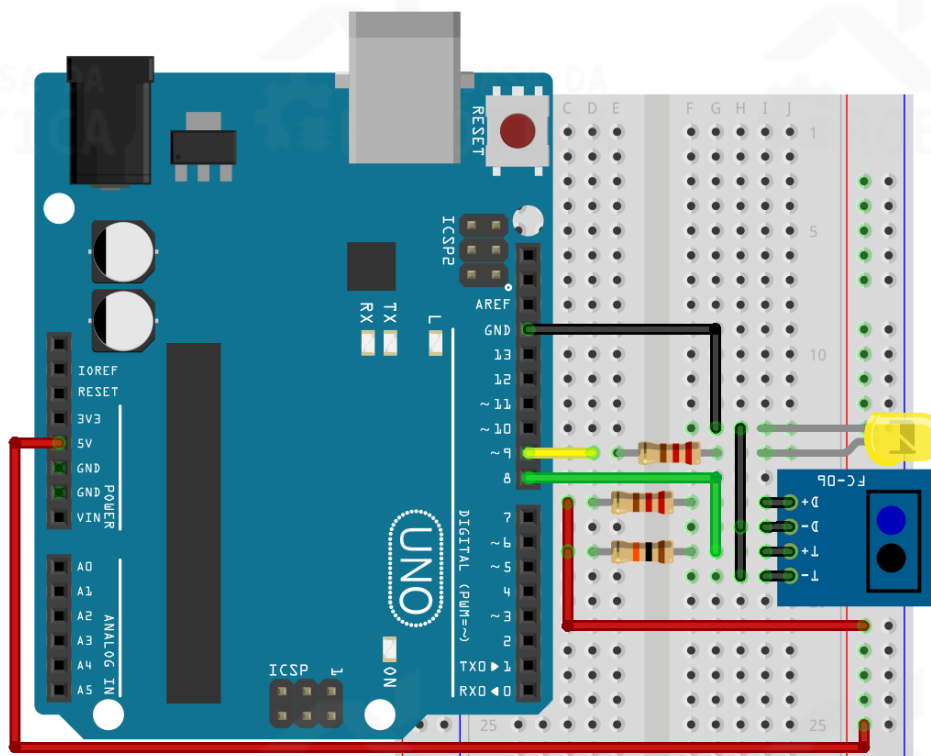
MATERIAIS NECESÁRIOS

- 1 x Placa UNO SMD R3 Atmega328 compatível com Arduino UNO;
- 1 x Cabo USB;
- 1 x Protoboard;
- 1 x TCRT 5000;
- 1 x Resistor de 10 k Ω ;
- 2 x Resistores de 220 Ω ;
- 1 x LED difuso.

ESQUEMÁTICO DE LIGAÇÃO DOS COMPONENTES

Conecte os componentes na protoboard conforme a Figura 58.

Figura 60 - Circuito para projeto detectar linha com sensor infravermelho.

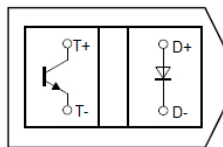


Ao montar seu circuito na protoboard preste atenção nos seguintes pontos:

- O TCRT 5000 é formado por dois componentes. O primeiro é um LED infravermelho (componente de cor azul), que emite um feixe de luz numa

frequência não visível a olho nu; O segundo é um fototransistor (componente de cor preta), que captura o feixe de luz emitido pelo infravermelho;

- Estes dois componentes funcionam em conjunto. O LED emite a luz infravermelha que é refletida por um objeto, posicionado em frente ao sensor, que é detectada pelo fototransistor;
- A cor e o material do objeto podem interferir no funcionamento do TCRT 5000. Ex. Objetos de cor preta não são bons refletores;
- O sensor TCRT 5000 possui quatro pinos, sendo dois do LED infravermelho e dois do fototransistor, como ilustra a imagem a seguir:



- Ânodo do LED infravermelho (D+);
- Cátodo do LED infravermelho (D-);
- Coletor do fototransistor (T+);
- Emissor do fototransistor (T-);

- O cátodo do LED difuso, o cátodo do LED infravermelho (D-) e o emissor do fototransistor (T-) devem ser ligados ao terra (GND) da placa UNO;
- O ânodo do LED difuso deve ser ligado a porta digital 9, o ânodo do LED infravermelho (D+) deve ser ligado ao 5V e o coletor do fototransistor deve ser ligado a porta digital 8. Conectamos também uma resistência de 10 k Ω entre a porta 8 e o 5V da placa UNO.

ELABORANDO O CÓDIGO

O propósito deste projeto é realizar a detecção de uma linha preta utilizando o sensor infravermelho TCRT 5000, de forma que ao detectar a linha preta o estado do sensor será nível baixo (LOW).

Vamos entender a lógica de programação deste projeto com os seguintes passos:

1- Definir as variáveis:

Definimos o pino digital 8, em que o pino T+ do sensor TCRT 5000 está conectado, a variável `pinoIR`, e o pino 9, em que o LED difuso está conectado, a variável `ledpin`. Definimos a variável `valorLido`, do tipo inteiro, para armazenar o estado (`HIGH/LOW`) do infravermelho;

2- Configurar as portas de entrada e saída e inicializar a comunicação serial
O pinoIR (pino 8) será definido como entrada (**INPUT**) e ledpin será definido como saída (**OUTPUT**);

A comunicação serial foi inicializada por meio da instrução: **Serial.begin**(9600);

- 3- No loop realizamos a leitura digital da variável pinoIR (pino 8) e atribuímos este valor a variável valorLido;
- 4- Utilizaremos a lógica do if...else para comparar se a variável valorLido encontra-se em nível alto ou baixo (linha detectada ou linha não detectada);
- 5- Se a variável valorLido estiver em nível lógico baixo (LOW) será exibida no monitor serial a mensagem “Linha detectada” e o LED difuso será acionado;
- 6- Senão, será exibida no monitor serial a mensagem “Linha NÃO detectada” e o LED difuso será desligado.

O programa do projeto proposto encontra-se detalhado a seguir:

```
int IRpin = 8; //Atribui o pino 8 a variável IRpin
int ledpin = 9; //Atribui o pino 9 a variável ledpin
int valorLido = 0; //Variável responsável por armazenar o estado do
infravermelho (LOW/HIGH)

void setup() {
    Serial.begin(9600); //Inicializa a comunicação serial, com velocidade de
    comunicação de 9600
    pinMode(IRpin, INPUT); //IRpin definido como entrada
    pinMode(ledpin, OUTPUT); //ledpin definido como saída
}

void loop() {
    valorLido = digitalRead(IRpin); //Armazena o valor digital de IRpin em
    valorLido
    if (valorLido == LOW) { //Se valor lido for igual a LOW
        Serial.println("Linha Detectada"); //Escreve na serial "Linha
        Detectada"
        digitalWrite(ledpin, HIGH); //Liga o LED
    }
    else { //Senão
        Serial.println("Linha NAO Detectada"); //Escreve na serial "Linha não
        detectada"
        digitalWrite(ledpin, LOW); //Desliga o LED
    }
    delay(100); //Intervalo de 100 milissegundos
}
```


PROJETO DETECTAR CAMPO MAGNÉTICO COM REED SWITCH

Conforme explicamos anteriormente, o reed switch é componente elétrico que possui duas lâminas, que em condições normais encontram-se separadas e não conduzem corrente elétrica, operando como uma chave aberta. No entanto, quando submetidas a um campo magnético essas lâminas se aproximam, permitindo a passagem da corrente elétrica, operando como um circuito fechado.

Sabendo dessa propriedade, a proposta deste projeto é utilizar um reed switch em conjunto com a placa uso para realizar a detecção de um campo magnético.

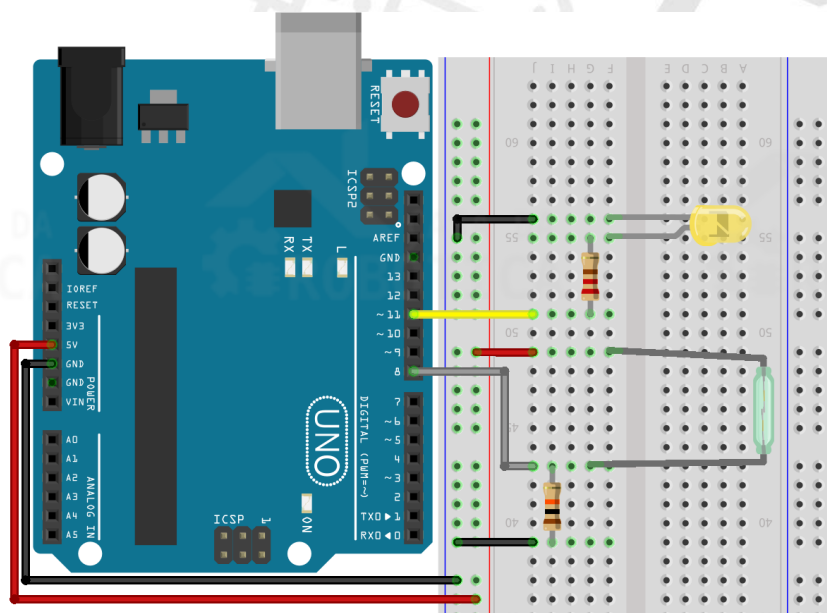
MATERIAIS NECESÁRIOS

- 1 x Placa UNO SMD R3 Atmega328 compatível com Arduino UNO;
- 1 x Cabo USB;
- 1 x Protoboard;
- 1 x Reed switch;
- 1 x Resistor de 10 k Ω ;
- 1 x Resistores de 220 Ω ;
- 1 x LED difuso.

ESQUEMÁTICO DE LIGAÇÃO DOS COMPONENTES

Conecte os componentes na protoboard como ilustra a Figura 61. Verifique cuidadosamente os cabos de ligação antes de ligar sua placa UNO. Lembre-se que a placa UNO deve estar desconectada enquanto você monta o circuito.

Figura 61 – Circuito para projeto detectar campo magnético com reed switch.



Ao montar seu circuito na protoboard preste atenção nos seguintes pontos:

- Por possuir um invólucro de vidro, o reed switch se torna bastante frágil e deve ser manuseado com bastante cuidado;
- Assim como os resistores o reed switch não possui polaridade;
- Um terminal do reed switch deve ser ligado ao 5 V e o outro terminal deve ser conectado ao pino digital 8. Conectamos também uma resistência de 10 kΩ entre a porta 8 e o GND da placa UNO.

ELABORANDO O CÓDIGO

A proposta desse projeto é criar um detector de campo magnético utilizando o reed switch em conjunto com a placa UNO. Esse projeto deve funcionar da seguinte maneira: Se o reed switch detectar um campo magnético uma mensagem de alerta deve ser exibida no monitor serial e o LED de alerta será acionado. Por sua vez, quando submetido a condições normais, o LED de alerta deve ser desligado.

Sabendo disso, vamos a programação do Sketch. Para melhor compreensão do código acompanhe os seguintes passos:

1- Declarar as variáveis

Iniciaremos a programação declarando três variáveis. Atribuímos a porta 8, que o reed switch encontra-se conectado, a variável reedPin. Definimos o pino 11 a variável

ledPin. Declaramos a variável valorLido, do tipo inteiro, para armazenar o estado do reed switch (ligado ou desligado).

2- Configurar os pinos de entrada e saída e inicializar a comunicação serial

A variável reedPin (pino 8) será definido como entrada (INPUT) e ledpin será definido como saída (OUTPUT);

A comunicação serial deve ser inicializada por meio da instrução: `Serial.begin(9600);`

3- No loop realizamos a leitura digital da variável reedPin (pino 8) e atribuímos este valor a variável valorLido;

4- Utilizaremos a lógica do `if...else` para comparar se a variável valorLido encontra-se em nível alto ou baixo (campo magnético detectado ou não detectado);

5- Se a variável valorLido estiver em nível lógico baixo (LOW) o LED difuso será desligado;

6- Senão, será exibida no monitor serial a mensagem "ALERTA: Campo magnético detectado" e o LED será ligado.

Com isso, o código deste projeto fica da seguinte forma:

```
int reedPin = 8; //Atribui o pino 8 a variável reedPin
int ledpin = 11; //Atribui o pino 11 a variável ledpin
int valorLido = 0; //Variável responsável por armazenar o estado do reed
Switch (LOW/HIGH)
void setup() {
    Serial.begin(9600); //Inicializa a comunicação serial, com velocidade de
    comunicação de 9600
    pinMode(reedPin, INPUT); //reedPin definido como entrada
    pinMode(ledpin, OUTPUT); //ledpin definido como saída
}
void loop() {
    valorLido = digitalRead(reedPin); //Armazena o valor digital de reedPin
    em valorLido
    if(valorLido == LOW) { //Se valorLido for igual a LOW
        digitalWrite(ledpin, LOW); //Coloca ledPin em nível baixo
    }
    else { //Senão
        Serial.println("ALERTA: Campo magnético detectado"); //Escreve na
        serial "ALERTA: Campo magnético detectado"
        digitalWrite(ledpin, HIGH); //Coloca ledPin em nível alto
    }
    delay(100); //Intervalo de 100 milissegundos
}
```

5. JOGO DA MEMÓRIA

O jogo eletrônico Genius foi um brinquedo muito popular na década de 1980. Esse jogo busca estimular a memorização através de sequências de cores e sons. Para vencer, o jogador deve reproduzir essas sequências na mesma ordem em que lhe é apresentada.

A proposta desse projeto é criar um jogo de memória similar ao game Genius utilizando componentes eletrônicos em conjunto com a placa UNO. Neste projeto, reuniremos todos os conceitos básicos aprendidos até aqui para nos divertir. Além disso, vamos aprender como produzir uma sequência de números randômicos.

Para criar as sequências de cores e sons do jogo utilizaremos as funções `randomSeed()` e `random()`. A função `randomSeed()` será utilizada para iniciar o gerador de números aleatórios utilizando como entrada aleatória a leitura de um pino analógico desconectado. Por sua vez, a função `random()` será usada para gerar um número aleatório entre 0 e 2. Para saber mais sobre essas funções acesse: <https://www.arduino.cc/reference/pt/language/functions/random-numbers/randomseed/>.

JOGO DA MEMÓRIA COM DISPLAY CÁTODO COMUM

MATERIAIS NECESÁRIOS

- 1 x Placa UNO SMD R3 Atmega328 compatível com Arduino UNO;
- 1 x Cabo USB;
- 1 x Protoboard;
- 3 x LEDs difusos;
- 1 x Display de 7 segmentos cátodo comum;
- 9 x Resistores de 220 Ω ;
- 3 x Resistores de 10 k Ω ;
- 1 x Buzzer ativo;
- 3 x Botões do tipo push button com capa;



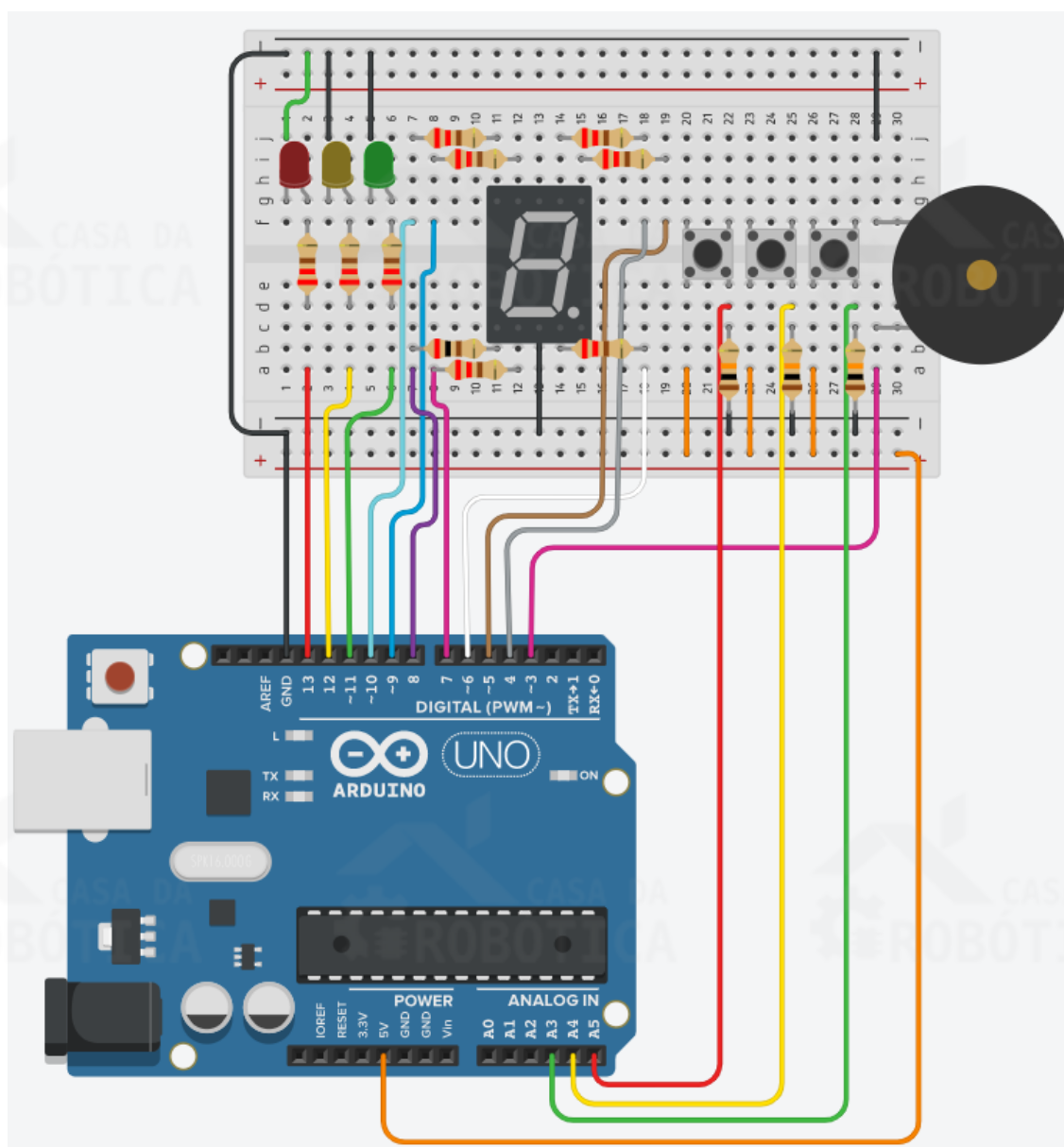
IDEIA:

Sugerimos que você utilize três LEDs de cores diferentes e que as capas dos botões correspondam a estas cores

ESQUEMÁTICO DE LIGAÇÃO DOS COMPONENTES

Conecte os componentes na protoboard como ilustra a Figura 62. Verifique cuidadosamente os cabos de ligação antes de ligar sua placa UNO. Lembre-se que a placa UNO deve estar desconectada enquanto você monta o circuito.

Figura 62 - Circuito para projeto Jogo da Memória – Display cátodo comum.



Ao montar seu circuito na protoboard preste atenção nos seguintes pontos:

- Construiremos o Game Genius utilizando apenas três sequências de cores e sons. Desta forma, utilizaremos três LEDs (vermelho, amarelo e verde) e três botões que representaram as cores dos LEDs e que devem ser pressionados pelo usuário para reproduzir a sequência;
- O LED vermelho deve ser conectado à porta digital 13, o amarelo à porta digital 12 e o verde à porta digital 11;
- Utilizaremos o display de 7 segmentos para informar ao jogador em qual nível do jogo ele está. O display de 7 segmentos é do tipo cátodo comum, então um dos terminais comuns do mesmo deve ser conectado ao terra (GND);
- Os segmentos encontram-se conectado as portas digitais da seguinte maneira: a = 4; b = 5; c = 6; d = 7; e = 8; f = 9; g = 10;
- O buzzer ativo deve ser conectado à porta digital 3;
- O primeiro botão, correspondente ao botão vermelho, será utilizado para acionar o LED vermelho e deve ser conectado a uma resistência pull-down e a porta analógica A5;
- O segundo botão, correspondente ao botão amarelo, será utilizado para acionar o LED amarelo e deve ser conectado a uma resistência pull-down e a porta analógica A4;
- O último botão, correspondente ao botão verde, será utilizado para acionar o LED verde e deve ser conectado a uma resistência pull-down e a porta analógica A3.

ELABORANDO O CÓDIGO

Com o circuito montado vamos a programação do Sketch do Jogo da Memória. Para melhor compreensão o código será explicado passo-a-passo a seguir. Neste momento, observe o código abaixo e aproveite para analisar sua estrutura.

```
//ENTRADAS
int butRed = A5;//Atribui a porta A5 a variável butRed - botão vermelho
int butYel = A4;//Atribui a porta A4 a variável butYel - botão amarelo
int butGre = A3;//Atribui a porta A3 a variável butGre - botão verde
```



```

//SAÍDAS
//Led
int ledRed = 13;//Atribui a porta 13 a variável ledRed - LED vermelho
int ledYel = 12;//Atribui a porta 12 a variável ledYel - LED amarelo
int ledGre = 11;//Atribui a porta 11 a variável ledGre - LED verde
//Display
int a = 4;//Atribui a porta 4 a variável "a" - segmento a do display
int b = 5;//Atribui a porta 5 a variável "b" - segmento b do display
int c = 6;//Atribui a porta 6 a variável "c" - segmento c do display
int d = 7;//Atribui a porta 7 a variável "d" - segmento d do display
int e = 8;//Atribui a porta 8 a variável "e" - segmento e do display
int f = 9;//Atribui a porta 9 a variável "f" - segmento f do display
int g = 10;//Atribui a porta 10 a variável "g" - segmento g do display
//Buzzer
int buzzerPin = 3;//Atribui a porta 3 a variável buzzerPin - buzzer

//Tons botões
#define tomRed 294//Tom para LED vermelho
#define tomYel 330//Tom para LED amarelo
#define tomGre 350//Tom para LED verde

//VARIÁVEIS
int rodada = 0;//Variável que armazenará o número de rodadas
int led = 0;//Variável que armazenará o valor randômico que representará o
LED que estará ligado
int temp_led_ligado = 600;//Tempo em que o LED deve estar ligado
int temp_led_desligado = 100;//Tempo em que o LED deve estar desligado
int mat[10];//Vetor de 10 posições que armazenará a sequência
int posi_mat = 0;//Posição do vetor

void setup(){
    //Configuração dos pinos de entrada
    pinMode(butRed, INPUT);
    pinMode(butYel, INPUT);
    pinMode(butGre, INPUT);
    //Configuração dos pinos de saída
    pinMode(ledRed, OUTPUT);
    pinMode(ledYel, OUTPUT);
    pinMode(ledGre, OUTPUT);
    pinMode(a, OUTPUT);
    pinMode(b, OUTPUT);
    pinMode(c, OUTPUT);
    pinMode(d, OUTPUT);
    pinMode(e, OUTPUT);
    pinMode(f, OUTPUT);
    pinMode(g, OUTPUT);
    pinMode(buzzerPin, OUTPUT);
    //Inicialização da comunicação serial
    Serial.begin(9600);
    //Função randômica
    randomSeed(analogRead(A0));//Pega como referência um valor aleatório do
canal analógico para gerar os valores randômicos
    inicio();//Chama a função início
}
void loop(){
    led = random(0, 3);//Gera o valor aleatório entre 0 e 2
    mat[posi_mat] = led;//Guarda o valor gerado no vetor

    switch (posi_mat) { //Liga o display conforme posição do vetor (nível do
jogo)
        case 0:

```

```

    zero();//Executa a função zero
    break;
case 1:
    um();//Executa a função um
    break;
case 2:
    dois();//Executa a função dois
    break;
case 3:
    tres();//Executa a função três
    break;
case 4:
    quatro();//Executa a função quatro
    break;
case 5:
    cinco();//Executa a função cinco
    break;
case 6:
    seis();//Executa a função seis
    break;
case 7:
    sete();//Executa a função sete
    break;
case 8:
    oito();//Executa a função oito
    break;
case 9:
    nove();//Executa a função nove
    break;
}

for (int j = 0; j <= posi_mat; j++) { //Mostra a sequência dos LEDs sempre
que passar o nível

    switch (mat[j]) { //Liga o LED correspondente a valor que está no vetor
na posição j
        case 0:
            digitalWrite(ledGre, HIGH); //Liga LED verde
            analogWrite(buzzerPin, tomGre); //Emite tom para LED verde
            delay(200); //Intervalo de 200 milissegundos
            digitalWrite(buzzerPin, LOW); //Desliga buzzer
            delay(temp_led_ligado);
            digitalWrite(ledGre, LOW); //Desliga LED verde
            delay (temp_led_desligado);
            break;
        case 1:
            digitalWrite(ledYel, HIGH); //Liga LED amarelo
            analogWrite(buzzerPin, tomYel); //Emite tom para LED amarelo
            delay(200); //Intervalo de 200 milissegundos
            digitalWrite(buzzerPin, LOW); //Desliga buzzer
            delay(temp_led_ligado);
            digitalWrite(ledYel, LOW); //Desliga LED amarelo
            delay (temp_led_desligado);
            break;
        case 2:
            digitalWrite(ledRed, HIGH); //Liga LED vermelho
            analogWrite(buzzerPin, tomRed); //Emite tom para LED vermelho
            delay(200); //Intervalo de 200 milissegundos
            digitalWrite(buzzerPin, LOW); //Desliga buzzer
            delay(temp_led_ligado);
            digitalWrite(ledRed, LOW); //Desliga LED vermelho
            delay (temp_led_desligado);

```

```

        break;
    default:
        break;
    }
}
int temp = 0; //Variável temporária utilizada para aguardar o jogador
digitar a sequência fornecida pelo jogo
while (temp <= posi_mat){
    if (digitalRead(butGre) == HIGH) { //Se butGre for nível alto - botão
verde pressionado
        digitalWrite(ledGre, HIGH); //Liga LED verde
        analogWrite(buzzerPin, tomGre); //Emite tom para LED verde
        delay(100); //Intervalo de 100 milissegundos
        digitalWrite(buzzerPin, LOW); //Desliga buzzer
        while (digitalRead(butGre) == HIGH); //Enquanto o botão verde
estiver pressionado não faz nada

        digitalWrite(ledGre, LOW); //Desliga o LED verde
        delay(200); //Intervalo de 200 milissegundos
        if (mat[temp] == 0) { //Se na posição do vetor é verdadeiro (igual a
zero)
            temp = temp + 1; //Incrementa a variável temp para dar continuidade
na sequência do jogo
        } else //Se não
        {
            Serial.println("ERRO!!!!"); //Imprime na serial Erro
            posi_mat = 11; //Atribui 11 na variável posi_mat
            break;
        }
    }
    if (digitalRead(butYel) == HIGH) { //Se butYel for nível alto - botão
amarelo pressionado
        digitalWrite(ledYel, HIGH); //Liga led amarelo
        analogWrite(buzzerPin, tomYel); //Emite tom para LED amarelo
        delay(100); //Intervalo de 100 milissegundos
        digitalWrite(buzzerPin, LOW); //Desliga buzzer
        while (digitalRead(butYel) == HIGH); //Enquanto o botão amarelo
estiver pressionado não faz nada
        digitalWrite(ledYel, LOW);
        delay(200);
        if (mat[temp] == 1) { //Se na posição do vetor é verdadeiro (igual a
zero)
            temp = temp + 1;
        } else
        {
            Serial.println("ERRO!!!!");
            posi_mat = 11; //Atribui 11 na variável posi_mat
            break;
        }
    }
    if (digitalRead(butRed) == HIGH) { //Se butRed for nível alto - botão
vermelho pressionado
        digitalWrite(ledRed, HIGH); //Liga led vermelho
        analogWrite(buzzerPin, tomRed); //Emite tom para led vermelho
        delay(100); //Intervalo de 100 milissegundos
        digitalWrite(buzzerPin, LOW); //Desliga buzzer
        while (digitalRead(butRed) == HIGH);
        digitalWrite(ledRed, LOW);
        delay(200);
        if (mat[temp] == 2) {
            temp = temp + 1;
        } else

```

```

    {
        Serial.println("ERRO!!!!"); //Imprime na serial "ERRO!!!!"
        posi_mat = 11; //Atribui 11 na variável posi_mat
        break;
    }
}
posi_mat = posi_mat + 1;

if (posi_mat == 10) { //Se a posi_mat for igual a 10
    ganhou(); //Chama a função ganhou()
    Serial.println("Parabens voce venceu "); //Imprime no serial "Parabens
voce venceu"
    posi_mat = 0; //Atribui 0 a variável posi_mat para reiniciar o jogo
    Serial.println("INICIO DE JOGO"); //Imprime no serial "INICIO DE JOGO"
}

if (posi_mat >= 11) { //Se posi_mat for maior ou igual a 11
    Serial.println(" ---- >>> FIM DE JOGO <<< ----"); //Imprime no serial "
---- >>> FIM DE JOGO <<< ----"
    perdeu(); //Chama a função perdeu
    posi_mat = 0; //Atribui 0 a variável posi_mat para reiniciar o jogo
    Serial.println("INICIO DE JOGO"); //Imprime no serial "INICIO DE JOGO"
    delay(3000); //Intervalo de 3 segundos
}
}

//Função para escrever os números no display
void zero() {
    digitalWrite(a, 1);
    digitalWrite(b, 1);
    digitalWrite(c, 1);
    digitalWrite(d, 1);
    digitalWrite(e, 1);
    digitalWrite(f, 1);
    digitalWrite(g, 0);
    delay(100);
}
void um() {
    digitalWrite(a, 0);
    digitalWrite(b, 1);
    digitalWrite(c, 1);
    digitalWrite(d, 0);
    digitalWrite(e, 0);
    digitalWrite(f, 0);
    digitalWrite(g, 0);
    delay(100);
}
void dois() {
    digitalWrite(a, 1);
    digitalWrite(b, 1);
    digitalWrite(c, 0);
    digitalWrite(d, 1);
    digitalWrite(e, 1);
    digitalWrite(f, 0);
    digitalWrite(g, 1);
    delay(100);
}
void tres() {
    digitalWrite(a, 1);
    digitalWrite(b, 1);
    digitalWrite(c, 1);

```

```

digitalWrite(d, 1);
digitalWrite(e, 0);
digitalWrite(f, 0);
digitalWrite(g, 1);
delay(100);
}
void quatro() {
digitalWrite(a, 0);
digitalWrite(b, 1);
digitalWrite(c, 1);
digitalWrite(d, 0);
digitalWrite(e, 0);
digitalWrite(f, 1);
digitalWrite(g, 1);
delay(100);
}
void cinco() {
digitalWrite(a, 1);
digitalWrite(b, 0);
digitalWrite(c, 1);
digitalWrite(d, 1);
digitalWrite(e, 0);
digitalWrite(f, 1);
digitalWrite(g, 1);
delay(100);
}
void seis() {
digitalWrite(a, 1);
digitalWrite(b, 0);
digitalWrite(c, 1);
digitalWrite(d, 1);
digitalWrite(e, 1);
digitalWrite(f, 1);
digitalWrite(g, 1);
delay(100);
}
void sete() {
digitalWrite(a, 1);
digitalWrite(b, 1);
digitalWrite(c, 1);
digitalWrite(d, 0);
digitalWrite(e, 0);
digitalWrite(f, 0);
digitalWrite(g, 0);
delay(100);
}
void oito() {
digitalWrite(a, 1);
digitalWrite(b, 1);
digitalWrite(c, 1);
digitalWrite(d, 1);
digitalWrite(e, 1);
digitalWrite(f, 1);
digitalWrite(g, 1);
delay(100);
}
void nove() {
digitalWrite(a, 1);
digitalWrite(b, 1);
digitalWrite(c, 1);
digitalWrite(d, 1);
digitalWrite(e, 0);

```

```

digitalWrite(f, 1);
digitalWrite(g, 1);
delay(100);
}

void inicio() {
  for (int y = 0; y < 2; y++) { //Repetir duas vezes
    tone(buzzerPin, 330, 200); //Frequência e duração da nota Mi
    digitalWrite(ledRed, HIGH); //Liga o LED vermelho
    delay(100); //Intervalo de 200 milissegundos
    tone(buzzerPin, 330, 200); //Frequência e duração da nota Mi
    digitalWrite(ledYel, HIGH); //Liga o LED amarelo
    delay(100); //Intervalo de 200 milissegundos
    tone(buzzerPin, 330, 200); //Frequência e duração da nota Mi
    digitalWrite(ledGre, HIGH); //Liga o LED verde
    delay(100); //Intervalo de 300 milissegundos
    tone(buzzerPin, 262, 300); //Frequência e duração da nota Dó
    delay(100); //Intervalo de 200 milissegundos
    digitalWrite(ledRed, LOW); //Desliga o LED vermelho
    digitalWrite(ledYel, LOW); //Desliga o LED amarelo
    digitalWrite(ledGre, LOW); //Desliga o LED verde
    delay(500); //Intervalo de 0,5 segundos
  }
}

void perdeu() {
  for (int x = 0; x < 3; x++) { //Repetir 3 vezes
    for (int z = 0; z < 3; z++) { //Repetir 2 vezes
      tone(buzzerPin, 262, 200); //Frequência e duração da nota Dó
      digitalWrite(ledRed, HIGH); //Liga LED vermelho
      digitalWrite(ledYel, HIGH); //Liga LED amarelo
      digitalWrite(ledGre, HIGH); //Liga LED verde
      delay(250); //Intervalo de 250 milissegundos
      digitalWrite(ledRed, LOW); //Desliga o LED vermelho
      digitalWrite(ledYel, LOW); //Desliga o LED amarelo
      digitalWrite(ledGre, LOW); //Desliga o LED verde
    }
    tone(buzzerPin, 528, 300); //Frequência e duração da nota Dó
    delay(100); //Intervalo de 100 milissegundos
  }
}

void ganhou() {
  for (int w = 0; w < 2; w++) { //Repetir 2 vezes
    tone(buzzerPin, 440, 200); //Frequência e duração da nota Lá
    digitalWrite(ledRed, HIGH); //Liga led vermelho
    digitalWrite(ledYel, HIGH); //Liga led amarelo
    digitalWrite(ledGre, HIGH); //Liga led verde
    delay(250); //Intervalo de 250 milissegundos
    digitalWrite(ledRed, LOW); //Desliga led vermelho
    digitalWrite(ledYel, LOW); //Desliga led amarelo
    digitalWrite(ledGre, LOW); //Desliga led verde
    tone(buzzerPin, 495, 200); //Frequência e duração da nota Si
    digitalWrite(ledRed, HIGH); //Liga led vermelho
    digitalWrite(ledYel, HIGH); //Liga led amarelo
    digitalWrite(ledGre, HIGH); //Liga led verde
    delay(250); //Intervalo de 250 milissegundos
    digitalWrite(ledRed, LOW); //Desliga led vermelho
    digitalWrite(ledYel, LOW); //Desliga led amarelo
    digitalWrite(ledGre, LOW); //Desliga led verde
    tone(buzzerPin, 528, 200); //Frequência e duração da nota Dó
    digitalWrite(ledRed, HIGH); //Liga led vermelho
  }
}

```



```

digitalWrite(ledYel, HIGH); //Liga led amarelo
digitalWrite(ledGre, HIGH); //Liga led verde
delay(250); //Intervalo de 250 milissegundos
digitalWrite(ledRed, LOW); //Desliga led vermelho
digitalWrite(ledYel, LOW); //Desliga led amarelo
digitalWrite(ledGre, LOW); //Desliga led verde
tone(buzzerPin, 528, 200); //Frequência e duração da nota Dó
digitalWrite(ledRed, HIGH); //Liga led vermelho
digitalWrite(ledYel, HIGH); //Liga led amarelo
digitalWrite(ledGre, HIGH); //Liga led verde
delay(250); //Intervalo de 250 milissegundos
digitalWrite(ledRed, LOW); //Desliga led vermelho
digitalWrite(ledYel, LOW); //Desliga led amarelo
digitalWrite(ledGre, LOW); //Desliga led verde
delay(500); //Intervalo de 500 milissegundos
}
}

```

Para melhor compreensão do código acompanhe os seguintes passos:

1- Declarar as variáveis

Assim como nos projetos anteriores, iniciamos a programação declarando as variáveis. Utilizamos as variáveis `butRed`, `butYel` e `butGre` para representar os botões vermelho, amarelo e verde, e atribuímos a elas os pinos analógicos A5, A4 e A3, respectivamente.

Em seguida, declaramos as variáveis `ledRed`, `ledYel` e `ledGre` atribuindo as portas digitais em que os LEDs se encontram conectados, ou seja, as portas 13, 12 e 11, respectivamente.

De modo semelhante ao projeto Contador de 0 a 9 com display de 7 segmentos, declaramos uma variável para cada segmento do display. Conforme o esquemático elétrico, atribuiremos a porta 4 a variável `a`; a porta 5 a variável `b`; a porta 6 a variável `c`; a porta 7 a variável `d`; a porta 8 a variável `e`; a porta 9 a variável `f`; a porta 10 a variável `g`.

Logo após, atribuímos a porta 3, em que o buzzer está conectado, a variável `buzzerPin`. Para que o buzzer reproduza sons diferentes para cada cor de LED declaramos as variáveis `tomRed`, `tomYel`, e `tomGre` atribuindo a elas o valor da frequência da nota musical desejada. Observe que para declarar estas variáveis utilizamos a diretiva `#define`, que permite dar um nome a um valor constante antes de o programa ser compilado. Nesse caso, as constantes definidas no Arduino não ocupam espaço na memória de programa do chip. Para saber mais sobre essa diretiva acesse: <https://www.arduino.cc/reference/pt/language/structure/further-syntax/define/>.

Declaramos também a variável `rodada` para armazenar o número de rodadas do jogo, `led` para guardar o valor randômico que representará qual LED deve ser acionado, `temp_led_ligado` para armazenar o tempo em que o LED deve permanecer ligado (600 milissegundos), e `temp_led_desligado` para armazenar o tempo em que o LED deve permanecer desligado (100 milissegundos).

Além destas variáveis, declaramos também o vetor `mat` com 10 posições para armazenar as sequências do jogo e a variável `posi_mat` para armazenar a posição do vetor.

2- Configurar os pinos de entrada e saída e inicializar a comunicação serial

As variáveis `butRed`, `butYel` e `butGre` foram configuradas como entrada (`INPUT`). Por sua vez, as variáveis `ledRed`, `ledYel`, `ledGre`, `a`, `b`, `c`, `d`, `e`, `f`, `g` e `buzzerPin` foram configuradas como saída (`OUTPUT`).

A comunicação serial foi inicializada por meio da instrução: `Serial.begin(9600);`.

3- Inicializar o gerador de números aleatórios e chamar função `inicio()`

O gerador de números aleatórios foi inicializado através da função `randomSeed(analogRead(A0))` que utiliza como entrada aleatória a leitura analógica do pino A0.

Ainda no setup, chamamos a função `inicio()` que tocará uma melodia e acionará os LEDs para representar o início do jogo.

4- Gerar um valor aleatório e armazenar no vetor

Através da instrução `led = random(0, 3);` geramos um valor aleatório (0, 1 ou 2) e armazenamos na variável `led`. O valor de `led` será armazenado no vetor `mat` na posição `posi_mat`.

Por exemplo, ao iniciar o jogo `posi_mat` será igual a 0 (zero), supondo que o número aleatório gerado pela função `random` tenha sido 1, esse valor será salvo na variável `led`. Desta forma, o valor de `led` (1) será armazenado no vetor `mat` na posição 0.

5- Verificar o valor de `posi_mat`

Toda vez que o jogador acertar a sequência de cores e sons a variável `posi_mat` será incrementada em 1. Utilizamos o valor armazenado nessa variável como parâmetro da estrutura de controle de fluxo de seleção `switch...case`, comparando esse valor aos números especificados nos cases. Quando o valor

armazenado em `posi_mat` for igual ao caractere especificado no case, a função para exibição do número correspondente no display deve ser executada.

6- Criar estrutura que mostre a sequência dos LEDs sempre que passar o nível

A cada nível acertado pelo jogador, o jogo deverá repetir a sequência anterior e incluir mais um comando de LED aceso para ser memorizado. Por exemplo, no nível 0 apenas o LED verde foi acionado, no nível 1 o LED verde deve ser acionado novamente e mais um LED deverá ser acionado (verde, amarelo ou vermelho, dependendo do valor aleatório gerado), e assim sucessivamente.

Desta forma, criamos uma estrutura de repetição utilizando o *for* para percorrer o vetor de forma que mostre toda a sequência gerada.

7- Criar estrutura que selecione os LEDs na sequência gerada

Para selecionar os LEDs na sequência gerada utilizamos a estrutura *switch... case*, que verificará o valor armazenado no vetor `mat` na posição `j`. Caso o conteúdo de `mat` na posição `j` seja igual a 0 o LED verde e seu tom serão acionados, caso `j` seja igual a 1 o LED amarelo e seu tom serão acionados, e sendo `j` igual a 2 o LED vermelho e seu tom serão acionados.

8- Declarar variável temporária para aguardar o jogador digitar a sequência fornecida

Criamos a variável do tipo `int` para aguardar o jogador digitar a sequência fornecida, através da instrução: `int temp = 0;`

9- Criar estrutura de controle para aguardar o usuário pressionar o botão com a sequência

No momento em que o usuário pressiona o botão será feita a comparação se a sequência pressionada foi igual a gerada pelo jogo. Isso é feito através das estruturas *if* de verificação que estão dentro do *while*.

A cada acerto da sequência a variável `temp` é incrementada com o objetivo de efetuar a próxima comparação, se houver. Caso o jogador pressione a sequência incorreta será impresso uma mensagem de erro no monitor serial, a variável `posi_mat` será igualada a 11 e um `break` será executado para parar o laço de repetição *while*.

Para identificar se o jogador ganhou verificamos o valor da variável `posi_mat`. Se `posi_mat` for igual a 10 indica a vitória do jogador e a função `ganhou()` será chamada, tocando uma melodia.

Se o valor de `posi_mat` for maior ou igual a 11 indica que o usuário errou a sequência e a função `perdeu()` será chamada. Além disso, será exibida na serial a mensagem: " ---- >>> FIM DE JOGO <<< ----".

Em ambas as situações, o jogo será reiniciado atribuindo zero a variável `posi_mat`.

10-Criar as funções dos números a serem exibidos no display 7 segmentos

Assim como no projeto Contador de 0 a 9 com display de 7 segmentos, criamos as funções que serão responsáveis por acionar os segmentos do display para que ele exiba números de 0 a 9. O display será utilizado para mostrar ao jogador o nível em que ele está, sendo assim, a cada sequência acertada o display incrementará em 1 o valor mostrado.

11-Criar as melodias nas funções `inicio()`, `perdeu()` e `ganhou()`.



OBSERVAÇÃO:

- Utilize o cabo para bateria 9 V com plug P4 para alimentar sua placa UNO. Desse modo, você poderá jogar o Jogo da Memória sem a necessidade do computador e poderá leva-lo a qualquer lugar.

TINKERCAD

Para melhor visualizar e/ou simular o circuito e a programação deste projeto basta se acessar o seguinte link: www.blogdarobotica.com/memoria_catodo.

JOGO DA MEMÓRIA COM DISPLAY ÂNODO COMUM

MATERIAIS NECESÁRIOS

- 1 x Placa UNO SMD R3 Atmega328 compatível com Arduino UNO;
- 1 x Cabo USB;
- 1 x Protoboard;
- 3 x LEDs difusos;
- 1 x Display de 7 segmentos ânodo comum;
- 9 x Resistores de 220 Ω ;
- 3 x Resistores de 10 k Ω ;
- 1 x Buzzer ativo;
- 3 x Botões do tipo push button com capa;



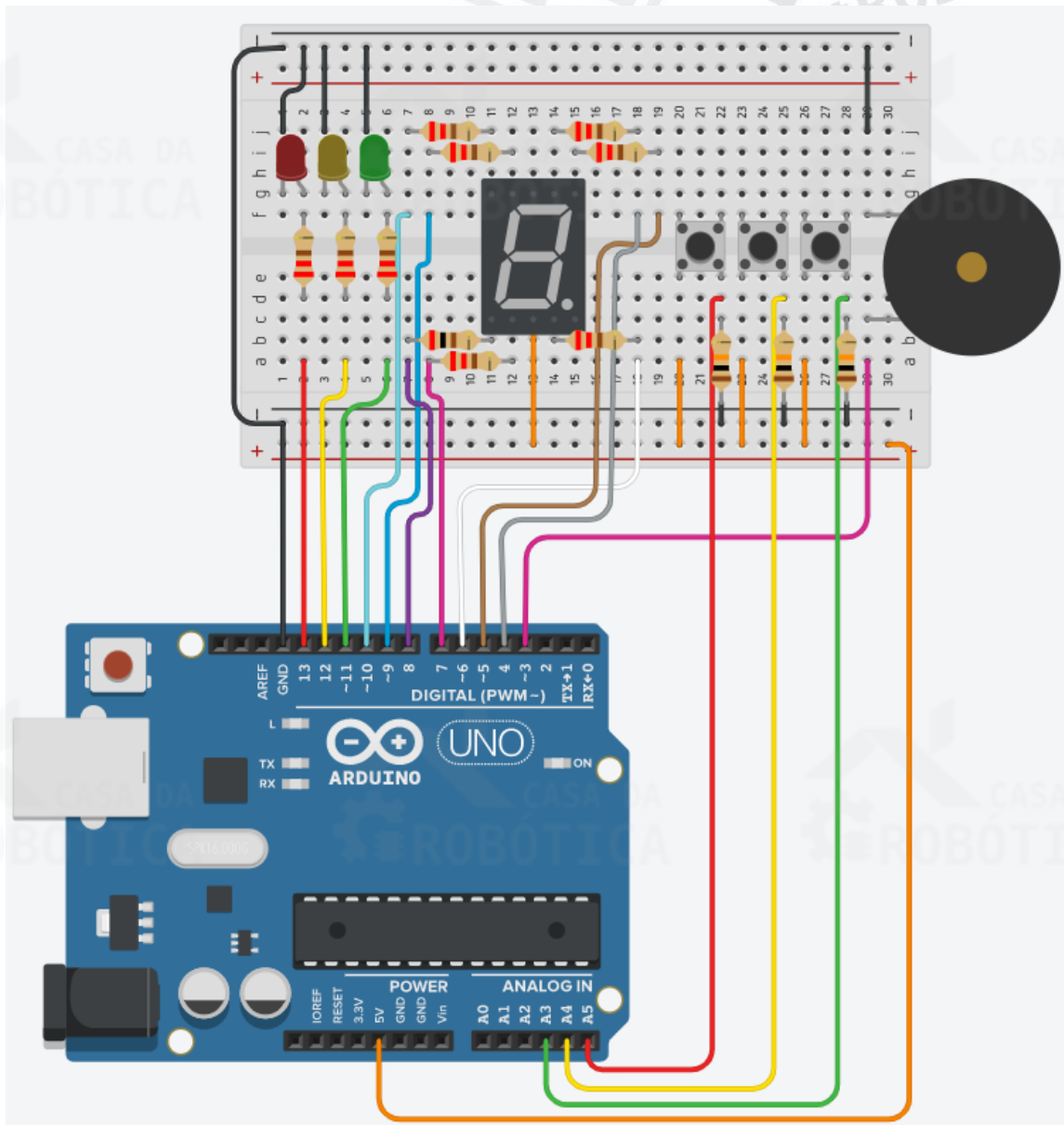
IDEIA:

Sugerimos que você utilize três LEDs de cores diferentes e que as capas dos botões correspondam a estas cores

ESQUEMÁTICO DE LIGAÇÃO DOS COMPONENTES

Conecte os componentes na protoboard como ilustra a Figura 63. Verifique cuidadosamente os cabos de ligação antes de ligar sua placa UNO. Lembre-se que a placa UNO deve estar desconectada enquanto você monta o circuito.

Figura 63 - Circuito para projeto Jogo da Memória – Display ânodo comum.



Ao montar seu circuito na protoboard preste atenção nos seguintes pontos:

- Construiremos o Game Genius utilizando apenas três sequências de cores e sons. Desta forma, utilizaremos três LEDs (vermelho, amarelo e verde) e três botões que representaram as cores dos LEDs e que devem ser pressionados pelo usuário para reproduzir a sequência;
- O LED vermelho deve ser conectado à porta digital 13, o amarelo à porta digital 12 e o verde à porta digital 11;
- Utilizaremos o display de 7 segmentos para informar ao jogador em qual nível do jogo ele está. O display de 7 segmentos é do tipo ânodo comum, então um dos terminais comuns do mesmo deve ser conectado ao 5 V;

- Os segmentos encontram-se conectado as portas digitais da seguinte maneira:
a = 4; b = 5; c = 6; d = 7; e = 8; f = 9; g = 10;
- O buzzer ativo deve ser conectado à porta digital 3;
- O primeiro botão, correspondente ao botão vermelho, será utilizado para acionar o LED vermelho e deve ser conectado a uma resistência pull-down e a porta analógica A5;
- O segundo botão, correspondente ao botão amarelo, será utilizado para acionar o LED amarelo e deve ser conectado a uma resistência pull-down e a porta analógica A4;
- O último botão, correspondente ao botão verde, será utilizado para acionar o LED verde e deve ser conectado a uma resistência pull-down e a porta analógica A3.

ELABORANDO O CÓDIGO

Com o circuito montado vamos a programação do Sketch do Jogo da Memória. Para melhor compreensão o código será explicado passo-a-passo a seguir. Neste momento, observe o código abaixo e aproveite para analisar sua estrutura.

```
//ENTRADAS
int butRed = A5;//Atribui a porta A5 a variável butRed - botão vermelho
int butYel = A4;//Atribui a porta A4 a variável butYel - botão amarelo
int butGre = A3;//Atribui a porta A3 a variável butGre - botão verde

//SAÍDAS
//Led
int ledRed = 13;//Atribui a porta 13 a variável ledRed - LED vermelho
int ledYel = 12;//Atribui a porta 12 a variável ledYel - LED amarelo
int ledGre = 11;//Atribui a porta 11 a variável ledGre - LED verde
//Display
int a = 4;//Atribui a porta 4 a variável "a" - segmento a do display
int b = 5;//Atribui a porta 5 a variável "b" - segmento b do display
int c = 6;//Atribui a porta 6 a variável "c" - segmento c do display
int d = 7;//Atribui a porta 7 a variável "d" - segmento d do display
int e = 8;//Atribui a porta 8 a variável "e" - segmento e do display
int f = 9;//Atribui a porta 9 a variável "f" - segmento f do display
int g = 10;//Atribui a porta 10 a variável "g" - segmento g do display
//Buzzer
int buzzerPin = 3;//Atribui a porta 3 a variável buzzerPin - buzzer

//Tons botões
#define tomRed 294//Tom para LED vermelho
```

```

#define tomYel 330//Tom para LED amarelo
#define tomGre 350//Tom para LED verde

//VARIÁVEIS
int rodada = 0;//Variável que armazenará o número de rodadas
int led = 0;//Variável que armazenará o valor randômico que representará o
LED que estará ligado
int temp_led_ligado = 600;//Tempo em que o LED deve estar ligado
int temp_led_desligado = 100;//Tempo em que o LED deve estar desligado
int mat[10];//Vetor de 10 posições que armazenará a sequência
int posi_mat = 0;//Posição do vetor

void setup(){
  //Configuração dos pinos de entrada
  pinMode(butRed, INPUT);
  pinMode(butYel, INPUT);
  pinMode(butGre, INPUT);
  //Configuração dos pinos de saída
  pinMode(ledRed, OUTPUT);
  pinMode(ledYel, OUTPUT);
  pinMode(ledGre, OUTPUT);
  pinMode(a, OUTPUT);
  pinMode(b, OUTPUT);
  pinMode(c, OUTPUT);
  pinMode(d, OUTPUT);
  pinMode(e, OUTPUT);
  pinMode(f, OUTPUT);
  pinMode(g, OUTPUT);
  pinMode(buzzerPin, OUTPUT);
  //Inicialização da comunicação serial
  Serial.begin(9600);
  //Função randômica
  randomSeed(analogRead(A0)); //Pega como referência um valor aleatório do
canal analógico para gerar os valores randômicos
  inicio();//Chama a função início
}

void loop(){
  led = random(0, 3);//Gera o valor aleatório entre 0 e 2
  mat[posi_mat] = led;//Guarda o valor gerado no vetor

  switch (posi_mat) { //Liga o display conforme posição do vetor (nível do
jogo)
    case 0:
      zero();//Executa a função zero
      break;
    case 1:
      um();//Executa a função um
      break;
    case 2:
      dois();//Executa a função dois
      break;
    case 3:
      tres();//Executa a função três
      break;
    case 4:
      quatro();//Executa a função quatro
      break;
    case 5:
      cinco();//Executa a função cinco
      break;
    case 6:
      seis();//Executa a função seis

```

```

        break;
    case 7:
        sete();//Executa a função sete
        break;
    case 8:
        oito();//Executa a função oito
        break;
    case 9:
        nove();//Executa a função nove
        break;
}

for (int j = 0; j <= posi_mat; j++) { //Mostra a sequência dos LEDs sempre
    que passar o nível

    switch (mat[j]) { //Liga o LED correspondente a valor que está no vetor
        na posição j
        case 0:
            digitalWrite(ledGre, HIGH); //Liga LED verde
            analogWrite(buzzerPin, tomGre); //Emite tom para LED verde
            delay(200); //Intervalo de 200 milissegundos
            digitalWrite(buzzerPin, LOW); //Desliga buzzer
            delay(temp_led_ligado);
            digitalWrite(ledGre, LOW); //Desliga LED verde
            delay (temp_led_desligado);
            break;
        case 1:
            digitalWrite(ledYel, HIGH); //Liga LED amarelo
            analogWrite(buzzerPin, tomYel); //Emite tom para LED amarelo
            delay(200); //Intervalo de 200 milissegundos
            digitalWrite(buzzerPin, LOW); //Desliga buzzer
            delay(temp_led_ligado);
            digitalWrite(ledYel, LOW); //Desliga LED amarelo
            delay (temp_led_desligado);
            break;
        case 2:
            digitalWrite(ledRed, HIGH); //Liga LED vermelho
            analogWrite(buzzerPin, tomRed); //Emite tom para LED vermelho
            delay(200); //Intervalo de 200 milissegundos
            digitalWrite(buzzerPin, LOW); //Desliga buzzer
            delay(temp_led_ligado);
            digitalWrite(ledRed, LOW); //Desliga LED vermelho
            delay (temp_led_desligado);
            break;
        default:
            break;
    }
}

int temp = 0; //Variável temporária utilizada para aguardar o jogador
digitar a sequência fornecida pelo jogo
while (temp <= posi_mat){
    if (digitalRead(butGre) == HIGH) { //Se butGre for nível alto - botão
    verde pressionado
        digitalWrite(ledGre, HIGH); //Liga LED verde
        analogWrite(buzzerPin, tomGre); //Emite tom para LED verde
        delay(100); //Intervalo de 100 milissegundos
        digitalWrite(buzzerPin, LOW); //Desliga buzzer
        while (digitalRead(butGre) == HIGH); //Enquanto o botão verde estiver
        pressionado não faz nada

        digitalWrite(ledGre, LOW); //Desliga o LED verde
        delay(200); //Intervalo de 200 milissegundos
    }
}

```

```

    if ( mat[temp] == 0) { //Se na posição do vetor é verdadeiro (igual a
zero)
        temp = temp + 1; //Incrementa a variável temp para dar continuidade
na sequência do jogo
    } else //Se não
    {
        Serial.println("ERRO!!!!"); //Imprime na serial Erro
        posi_mat = 11; //Atribui 11 na variável posi_mat
        break;
    }
}
if (digitalRead(butYel) == HIGH) { //Se butYel for nível alto - botão
amarelo pressionado
    digitalWrite(ledYel, HIGH); //Liga led amarelo
    analogWrite(buzzerPin, tomYel); //Emite tom para LED amarelo
    delay(100); //Intervalo de 100 milissegundos
    digitalWrite(buzzerPin, LOW); //Desliga buzzer
    while (digitalRead(butYel) == HIGH); //Enquanto o botão amarelo
estiver pressionado não faz nada
    digitalWrite(ledYel, LOW);
    delay(200);
    if ( mat[temp] == 1) { //Se na posição do vetor é verdadeiro (igual a
zero)
        temp = temp + 1;
    } else
    {
        Serial.println("ERRO!!!!");
        posi_mat = 11; //Atribui 11 na variável posi_mat
        break;
    }
}
if (digitalRead(butRed) == HIGH) { //Se butRed for nível alto - botão
vermelho pressionado
    digitalWrite(ledRed, HIGH); //Liga led vermelho
    analogWrite(buzzerPin, tomRed); //Emite tom para led vermelho
    delay(100); //Intervalo de 100 milissegundos
    digitalWrite(buzzerPin, LOW); //Desliga buzzer
    while (digitalRead(butRed) == HIGH);
    digitalWrite(ledRed, LOW);
    delay(200);
    if ( mat[temp] == 2) {
        temp = temp + 1;
    } else
    {
        Serial.println("ERRO!!!!"); //Imprime na serial "ERRO!!!!"
        posi_mat = 11; //Atribui 11 na variável posi_mat
        break;
    }
}
}
posi_mat = posi_mat + 1;

if (posi_mat == 10) { //Se a posi_mat for igual a 10
    ganhou(); //Chama a função ganhou()
    Serial.println("Parabens voce venceu "); //Imprime no serial "Parabens
voce venceu"
    posi_mat = 0; //Atribui 0 a variável posi_mat para reiniciar o jogo
    Serial.println("INICIO DE JOGO"); //Imprime no serial "INICIO DE JOGO"
}

if (posi_mat >= 11) { //Se posi_mat for maior ou igual a 11

```

```

Serial.println(" ---- >>> FIM DE JOGO <<< ----"); //Imprime no serial "
---- >>> FIM DE JOGO <<< ----"
perdeu(); //Chama a função perdeu
posi_mat = 0; //Atribui 0 a variável posi_mat para reiniciar o jogo
Serial.println("INICIO DE JOGO"); //Imprime no serial "INICIO DE JOGO"
delay(3000); //Intervalo de 3 segundos
}
}

//Função para escrever os números no display
void zero() {
    digitalWrite(a, 0);
    digitalWrite(b, 0);
    digitalWrite(c, 0);
    digitalWrite(d, 0);
    digitalWrite(e, 0);
    digitalWrite(f, 0);
    digitalWrite(g, 1);
    delay(100);
}
void um() {
    digitalWrite(a, 1);
    digitalWrite(b, 0);
    digitalWrite(c, 0);
    digitalWrite(d, 1);
    digitalWrite(e, 1);
    digitalWrite(f, 1);
    digitalWrite(g, 1);
    delay(100);
}
void dois() {
    digitalWrite(a, 0);
    digitalWrite(b, 0);
    digitalWrite(c, 1);
    digitalWrite(d, 0);
    digitalWrite(e, 0);
    digitalWrite(f, 1);
    digitalWrite(g, 0);
    delay(100);
}
void tres() {
    digitalWrite(a, 0);
    digitalWrite(b, 0);
    digitalWrite(c, 0);
    digitalWrite(d, 0);
    digitalWrite(e, 1);
    digitalWrite(f, 1);
    digitalWrite(g, 0);
    delay(100);
}
void quatro() {
    digitalWrite(a, 1);
    digitalWrite(b, 0);
    digitalWrite(c, 0);
    digitalWrite(d, 1);
    digitalWrite(e, 1);
    digitalWrite(f, 0);
    digitalWrite(g, 0);
    delay(100);
}
void cinco() {
    digitalWrite(a, 0);

```

```

digitalWrite(b, 1);
digitalWrite(c, 0);
digitalWrite(d, 0);
digitalWrite(e, 1);
digitalWrite(f, 0);
digitalWrite(g, 0);
delay(100);
}
void seis() {
digitalWrite(a, 0);
digitalWrite(b, 1);
digitalWrite(c, 0);
digitalWrite(d, 0);
digitalWrite(e, 0);
digitalWrite(f, 0);
digitalWrite(g, 0);
delay(100);
}
void sete() {
digitalWrite(a, 0);
digitalWrite(b, 0);
digitalWrite(c, 0);
digitalWrite(d, 1);
digitalWrite(e, 1);
digitalWrite(f, 1);
digitalWrite(g, 1);
delay(100);
}
void oito() {
digitalWrite(a, 0);
digitalWrite(b, 0);
digitalWrite(c, 0);
digitalWrite(d, 0);
digitalWrite(e, 0);
digitalWrite(f, 0);
digitalWrite(g, 0);
delay(100);
}
void nove() {
digitalWrite(a, 0);
digitalWrite(b, 0);
digitalWrite(c, 0);
digitalWrite(d, 0);
digitalWrite(e, 1);
digitalWrite(f, 0);
digitalWrite(g, 0);
delay(100);
}

void inicio() {
for (int y = 0; y < 2; y++) { //Repetir duas vezes
tone(buzzerPin, 330, 200); //Frequência e duração da nota Mi
digitalWrite(ledRed, HIGH); //Liga o LED vermelho
delay(100); //Intervalo de 200 milissegundos
tone(buzzerPin, 330, 200); //Frequência e duração da nota Mi
digitalWrite(ledYel, HIGH); //Liga o LED amarelo
delay(100); //Intervalo de 200 milissegundos
tone(buzzerPin, 330, 200); //Frequência e duração da nota Mi
digitalWrite(ledGre, HIGH); //Liga o LED verde
delay(100); //Intervalo de 300 milissegundos
tone(buzzerPin, 262, 300); //Frequência e duração da nota Dó
delay(100); //Intervalo de 200 milissegundos
}
}

```



```

digitalWrite(ledRed, LOW); //Desliga o LED vermelho
digitalWrite(ledYel, LOW); //Desliga o LED amarelo
digitalWrite(ledGre, LOW); //Desliga o LED verde
delay(500); //Intervalo de 0,5 segundos
}
}

void perdeu() {
  for (int x = 0; x < 3; x++) { //Repetir 3 vezes
    for (int z = 0; z < 3; z++) { //Repetir 2 vezes
      tone(buzzerPin, 262, 200); //Frequência e duração da nota Dó
      digitalWrite(ledRed, HIGH); //Liga LED vermelho
      digitalWrite(ledYel, HIGH); //Liga LED amarelo
      digitalWrite(ledGre, HIGH); //Liga LED verde
      delay(250); //Intervalo de 250 milissegundos
      digitalWrite(ledRed, LOW); //Desliga o LED vermelho
      digitalWrite(ledYel, LOW); //Desliga o LED amarelo
      digitalWrite(ledGre, LOW); //Desliga o LED verde
    }
    tone(buzzerPin, 528, 300); //Frequência e duração da nota Dó
    delay(100); //Intervalo de 100 milissegundos
  }
}

void ganhou() {
  for (int w = 0; w < 2; w++) { //Repetir 2 vezes
    tone(buzzerPin, 440, 200); //Frequência e duração da nota Lá
    digitalWrite(ledRed, HIGH); //Liga led vermelho
    digitalWrite(ledYel, HIGH); //Liga led amarelo
    digitalWrite(ledGre, HIGH); //Liga led verde
    delay(250); //Intervalo de 250 milissegundos
    digitalWrite(ledRed, LOW); //Desliga led vermelho
    digitalWrite(ledYel, LOW); //Desliga led amarelo
    digitalWrite(ledGre, LOW); //Desliga led verde
    tone(buzzerPin, 495, 200); //Frequência e duração da nota Si
    digitalWrite(ledRed, HIGH); //Liga led vermelho
    digitalWrite(ledYel, HIGH); //Liga led amarelo
    digitalWrite(ledGre, HIGH); //Liga led verde
    delay(250); //Intervalo de 250 milissegundos
    digitalWrite(ledRed, LOW); //Desliga led vermelho
    digitalWrite(ledYel, LOW); //Desliga led amarelo
    digitalWrite(ledGre, LOW); //Desliga led verde
    tone(buzzerPin, 528, 200); //Frequência e duração da nota Dó
    digitalWrite(ledRed, HIGH); //Liga led vermelho
    digitalWrite(ledYel, HIGH); //Liga led amarelo
    digitalWrite(ledGre, HIGH); //Liga led verde
    delay(250); //Intervalo de 250 milissegundos
    digitalWrite(ledRed, LOW); //Desliga led vermelho
    digitalWrite(ledYel, LOW); //Desliga led amarelo
    digitalWrite(ledGre, LOW); //Desliga led verde
    tone(buzzerPin, 528, 200); //Frequência e duração da nota Dó
    digitalWrite(ledRed, HIGH); //Liga led vermelho
    digitalWrite(ledYel, HIGH); //Liga led amarelo
    digitalWrite(ledGre, HIGH); //Liga led verde
    delay(250); //Intervalo de 250 milissegundos
    digitalWrite(ledRed, LOW); //Desliga led vermelho
    digitalWrite(ledYel, LOW); //Desliga led amarelo
    digitalWrite(ledGre, LOW); //Desliga led verde
    delay(500); //Intervalo de 500 milissegundos
  }
}
}

```

Para melhor compreensão do código acompanhe os seguintes passos:

1- Declarar as variáveis

Assim como nos projetos anteriores, iniciamos a programação declarando as variáveis. Utilizamos as variáveis `butRed`, `butYel` e `butGre` para representar os botões vermelho, amarelo e verde, e atribuímos a elas os pinos analógicos A5, A4 e A3, respectivamente.

Em seguida, declaramos as variáveis `ledRed`, `ledYel` e `ledGre` atribuindo as portas digitais em que os LEDs se encontram conectados, ou seja, as portas 13, 12 e 11, respectivamente.

De modo semelhante ao projeto Contador de 0 a 9 com display de 7 segmentos, declaramos uma variável para cada segmento do display. Conforme o esquemático elétrico, atribuiremos a porta 4 a variável `a`; a porta 5 a variável `b`; a porta 6 a variável `c`; a porta 7 a variável `d`; a porta 8 a variável `e`; a porta 9 a variável `f`; a porta 10 a variável `g`.

Logo após, atribuímos a porta 3, em que o buzzer está conectado, a variável `buzzerPin`. Para que o buzzer reproduza sons diferentes para cada cor de LED declaramos as variáveis `tomRed`, `tomYel`, e `tomGre` atribuindo a elas o valor da frequência da nota musical desejada. Observe que para declarar estas variáveis utilizamos a diretiva `#define`, que permite dar um nome a um valor constante antes de o programa ser compilado. Nesse caso, as constantes definidas no Arduino não ocupam espaço na memória de programa do chip. Para saber mais sobre essa diretiva acesse: <https://www.arduino.cc/reference/pt/language/structure/further-syntax/define/>.

Declaramos também a variável `rodada` para armazenar o número de rodadas do jogo, `led` para guardar o valor randômico que representará qual LED deve ser acionado, `temp_led_ligado` para armazenar o tempo em que o LED deve permanecer ligado (600 milissegundos), e `temp_led_desligado` para armazenar o tempo em que o LED deve permanecer desligado (100 milissegundos).

Além destas variáveis, declaramos também o vetor `mat` com 10 posições para armazenar as sequências do jogo e a variável `posi_mat` para armazenar a posição do vetor.

2- Configurar os pinos de entrada e saída e inicializar a comunicação serial

As variáveis `butRed`, `butYel` e `butGre` foram configuradas como entrada (`INPUT`). Por sua vez, as variáveis `ledRed`, `ledYel`, `ledGre`, `a`, `b`, `c`, `d`, `e`, `f`, `g` e `buzzerPin` foram configuradas como saída (`OUTPUT`).

A comunicação serial foi inicializada por meio da instrução:
`Serial.begin(9600);`.

3- Inicializar o gerador de números aleatórios e chamar função `inicio()`

O gerador de números aleatórios foi inicializado através da função `randomSeed(analogRead(A0))` que utiliza como entrada aleatória a leitura analógica do pino A0.

Ainda no `setup`, chamamos a função `inicio()` que tocará uma melodia e acionará os LEDs para representar o início do jogo.

4- Gerar um valor aleatório e armazenar no vetor

Através da instrução `led = random(0, 3);` geramos um valor aleatório (0, 1 ou 2) e armazenamos na variável `led`. O valor de `led` será armazenado no vetor `mat` na posição `posi_mat`.

Por exemplo, ao iniciar o jogo `posi_mat` será igual a 0 (zero), supondo que o número aleatório gerado pela função `random` tenha sido 1, esse valor será salvo na variável `led`. Desta forma, o valor de `led` (1) será armazenado no vetor `mat` na posição 0.

5- Verificar o valor de `posi_mat`

Toda vez que o jogador acertar a sequência de cores e sons a variável `posi_mat` será incrementada em 1. Utilizamos o valor armazenado nessa variável como parâmetro da estrutura de controle de fluxo de seleção `switch...case`, comparando esse valor aos números especificados nos `cases`. Quando o valor armazenado em `posi_mat` for igual ao caractere especificado no `case`, a função para exibição do número correspondente no display deve ser executada.

6- Criar estrutura que mostre a sequência dos LEDs sempre que passar o nível

A cada nível acertado pelo jogador, o jogo deverá repetir a sequência anterior e incluir mais um comando de LED aceso para ser memorizado. Por exemplo, no nível 0 apenas o LED verde foi acionado, no nível 1 o LED verde deve ser acionado novamente e mais um LED deverá ser acionado (verde, amarelo ou vermelho, dependendo do valor aleatório gerado), e assim sucessivamente.

Desta forma, criamos uma estrutura de repetição utilizando o *for* para percorrer o vetor de forma que mostre toda a sequência gerada.

7- Criar estrutura que selecione os LEDs na sequência gerada

Para selecionar os LEDs na sequência gerada utilizamos a estrutura *switch...case*, que verificará o valor armazenado no vetor `mat` na posição `j`. Caso o conteúdo de `mat` na posição `j` seja igual a 0 o LED verde e seu tom serão acionados, caso `j` seja igual a 1 o LED amarelo e seu tom serão acionados, e sendo `j` igual a 2 o LED vermelho e seu tom serão acionados.

8- Declarar variável temporária para aguardar o jogador digitar a sequência fornecida

Criamos a variável do tipo `int` para aguardar o jogador digitar a sequência fornecida, através da instrução: `int temp = 0;`

9- Criar estrutura de controle para aguardar o usuário pressionar o botão com a sequência

No momento em que o usuário pressiona o botão será feita a comparação se a sequência pressionada foi igual a gerada pelo jogo. Isso é feito através das estruturas *if* de verificação que estão dentro do *while*.

A cada acerto da sequência a variável `temp` é incrementada com o objetivo de efetuar a próxima comparação, se houver. Caso o jogador pressione a sequência incorreta será impresso uma mensagem de erro no monitor serial, a variável `posi_mat` será igualada a 11 e um `break` será executado para parar o laço de repetição *while*.

Para identificar se o jogador ganhou verificamos o valor da variável `posi_mat`. Se `posi_mat` for igual a 10 indica a vitória do jogador e a função `ganhou()` será chamada, tocando uma melodia.

Se o valor de `posi_mat` for maior ou igual a 11 indica que o usuário errou a sequência e a função `perdeu()` será chamada. Além disso, será exibida na serial a mensagem: " ---- >>> FIM DE JOGO <<< ----".

Em ambas as situações, o jogo será reiniciado atribuindo zero a variável `posi_mat`.

10-Criar as funções dos números a serem exibidos no display 7 segmentos

Assim como no projeto Contador de 0 a 9 com display de 7 segmentos, criamos as funções que serão responsáveis por acionar os segmentos do display para que ele

exiba números de 0 a 9. O display será utilizado para mostrar ao jogador o nível em que ele está, sendo assim, a cada sequência acertada o display incrementará em 1 o valor mostrado.

11-Criar as melodias nas funções `inicio()`, `perdeu()` e `ganhou()`.



OBSERVAÇÃO:

- Utilize o cabo para bateria 9 V com plug P4 para alimentar sua placa UNO. Desse modo, você poderá jogar o Jogo da Memória sem a necessidade do computador e poderá leva-lo a qualquer lugar.

Para melhor visualizar e/ou simular o circuito e a programação deste projeto basta se acessar o seguinte link: www.blogdarobotica.com/memoria_anodo.

6. BÔNUS: PLATAFORMA BLYNK

O Blynk é uma plataforma desenvolvida para aplicações em Internet das Coisas (*Internet of Things* – IoT) que permite que o usuário crie interfaces para controlar e monitorar projetos de hardware a partir de dispositivos móveis Android e iOS.

Com o Blynk é possível controlar microcontroladores, coletar dados de sensores, criar painéis visuais personalizados e salvar dados automaticamente na nuvem Blynk Cloud. Além disso, esta plataforma permite enviar notificações, e-mails ou tweets.

O Blynk é perfeito para realizar a interação com projetos simples, como monitorar a temperatura de um ambiente, ligar e desligar as luzes, controlar as cores de um LED RGB de uma sala de estar, entre outras. Sua interface de arrastar e soltar composta por vários widgets o torna bastante fácil e intuitivo. O Blynk é, basicamente, formado por três componentes principais, que são:

- Blynk App: Aplicativo gratuito disponível para Android e iOS que permite ao usuário criar interfaces de projetos de forma bastante simples, em que é necessário apenas arrastar os Widgets (botões, chaves, displays, joysticks) e realizar a configuração deles;
- Servidor Blynk: é o servidor responsável por todas as comunicações entre o smartphone e o hardware. Vale ressaltar que você pode usar o servidor Blynk Cloud ou executar o servidor Blynk em sua máquina local;
- Bibliotecas Blynk: Do lado do hardware, o Blynk disponibiliza bibliotecas para as plataformas de desenvolvimento mais populares, que possibilitam a comunicação com o servidor e processam todos os comandos de entrada e saída.

Para utilizar o Blynk faz-se necessário os seguintes requisitos:

- Hardware: Um Arduino, Raspberry Pi, ESP 8266 ou um kit de desenvolvimento semelhante;
- Internet: Para funcionamento o Blynk necessita de acesso à internet;
- Smartphone: O Blynk App funciona tanto em smartphones Android ou iOS;
- Cadastro de usuário: Após a instalação do Blynk App é necessário realizar um cadastro para poder utilizar a ferramenta;
- Biblioteca Blynk: É necessário instalar a biblioteca Blynk para utilizá-lo.

PRIMEIROS PASSOS

Download e instalação do Blynk App

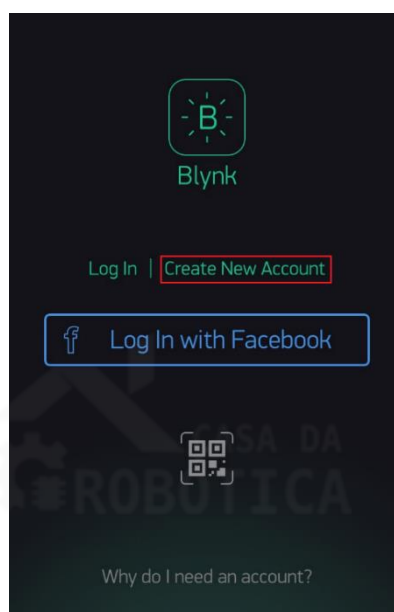
Conforme mencionado anteriormente, o Blynk App encontra-se disponível para Android e iOS. Desta forma, para realizar o download em seu dispositivo será necessário acessar a loja de aplicativos, Play Store ou App Store. Se achar melhor, você também pode encontrar o Blynk App por meio dos seguintes links:

Link Android: <https://play.google.com/store/apps/details?id=cc.blynk>

Link iOS: <https://apps.apple.com/us/app/blynk-control-arduino-raspberry/id808760481?ls=1>

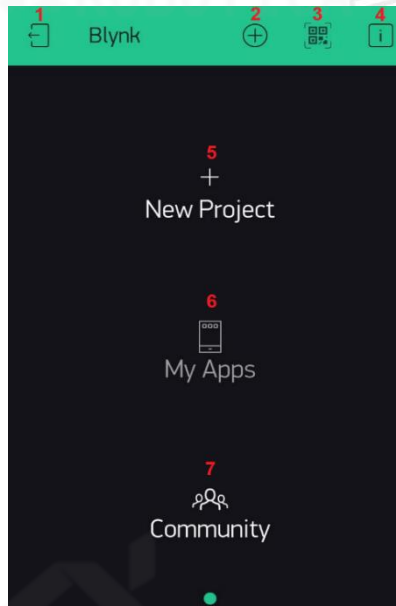
Após a instalação do aplicativo em seu dispositivo um ícone será criado na tela inicial e para iniciar o aplicativo basta clicar neste ícone. Ao abrir o Blynk App pela primeira vez faz-se necessário a criação de um cadastro de acesso, que deve ser feito selecionando a opção Create New Account, inserindo um e-mail válido e uma senha. A Figura 64 demonstra a tela inicial do Blynk App.

Figura 64 - Tela inicial do Blynk App.



No momento do cadastro insira um endereço de e-mail válido, pois algumas informações importantes para o funcionamento de seus projetos serão encaminhadas por meio dele. Após a realização do cadastro a tela da Figura 65 será exibida.

Figura 65 - Tela de criação do Blynk App.

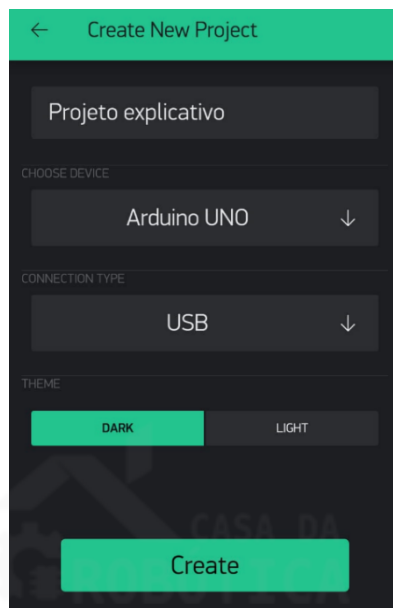


Os ícones numerados em vermelho na Figura 65 são detalhados a seguir:

1. Log Out: Ao selecionar este ícone, você saíra da conta logada e retornará a tela inicial (Figura 64);
2. New Project: Possui a mesma função do ícone de número 5, que é criar um novo projeto;
3. QRcode: Esta opção permite que você leia, por meio do QRcode, e copie um projeto já existente que foi compartilhado por outro usuário do Blynk;
4. About: Traz algumas informações sobre o aplicativo, como versão, usuário e servidos.
5. New Project: Possui a mesma função do ícone de número 2, que é criar um novo projeto;
6. My Apps: Ao selecionar este item seu projeto poderá ser exportado para seu dispositivo móvel e ser publicado no Play Store ou App Store.
7. Community: Ao selecionar esta opção você será direcionado a comunidade Blynk, site destinado a solicitação e votação de novos recursos, perguntas, comentários e compartilhamento de ideias.

Para criar o seu aplicativo você deve selecionar a opção New Project (item 2 ou 6 da Figura 65). Em seguida será exibida a tela Create New Project, onde será possível definir um nome para o projeto, a plataforma utilizada, o tipo de conexão entre o aplicativo e o microcontrolador, e o tema do aplicativo. A Figura 66 ilustra a tela Create New Project do Blynk App.

Figura 66 - Tela Create New Project.



Na Figura 66, para fim explicativo, definimos o nome do projeto de “Projeto explicativo”, escolhemos o microcontrolador Arduino UNO e o tipo de conexão será por USB. O tema do aplicativo pode ser “Dark” (escuro), que é a cor padrão do Blynk, ou “Light” (claro). Ao optar pelo tema Light o aplicativo ficará branco. Após a definição destas configurações, clique no botão “Create”.

Em seguida, será exibida uma mensagem informando que um “Auth Token” do projeto foi enviado ao seu e-mail cadastrado. O “Auth Token” é um código de 32 caracteres utilizado para que o servidor Blynk direcione as informações entre o aplicativo e a plataforma. Clique em OK para fechar esta mensagem. Logo após, a tela de projeto será exibida, conforme Figura 67.

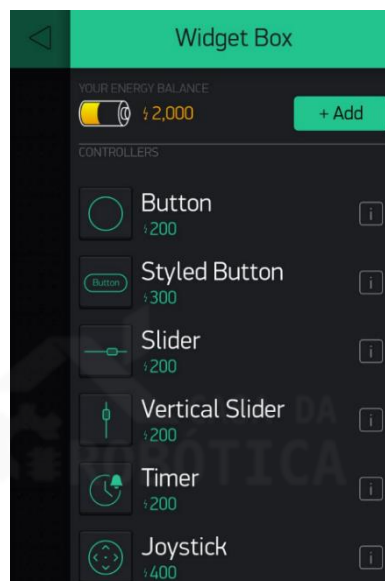
Figura 67 - Tela de projeto do Blynk App.



Os ícones da parte superior da tela de projeto do Blynk App encontram-se detalhados a seguir:

1. Log Out: Ao selecionar este ícone, você sairá da tela de projetos e retornará a tela de criação (Figura 65);
2. Project Settings: Ao selecionar esta opção você terá acesso a tela de configurações do projeto;
3. Widget Box: O ícone representado pelo símbolo + permite a adição de widgets a tela do projeto. Ao selecionar esta opção uma lista de widgets será aberta, conforme Figura 68.
4. Play: Ao clicar neste ícone a comunicação entre o aplicativo e a plataforma será iniciada.

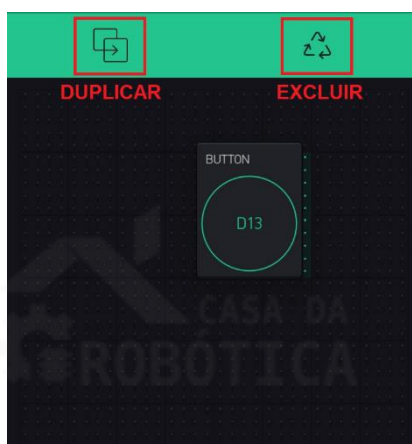
Figura 68 - Lista de widgets.



Apesar do Blynk App, Servidor Blynk e as bibliotecas Blynk serem gratuitas, os Widgets são pagos. Cada Widget custa uma quantidade de Energy – espécie de moeda virtual. Iniciamos o Blynk App com 2000 Energy disponível para ser utilizada em nossos projetos. Para o desenvolvimento de projetos mais complexos mais Energy pode ser comprada.

Lembrando que cada Energy utilizado ao incluir um Widget é retornado à carteira quando o excluímos. Para excluir um Widget basta pressionar por um período de tempo e aguardar a exibição da opção excluir na parte superior da tela do aplicativo, conforma a Figura 69.

Figura 69 - Ícones Duplicar e Excluir Widgets.



Instalação do pacote de bibliotecas no Arduino IDE

Para que o Blynk funcione em sua plataforma de hardware é necessário instalar uma biblioteca. A biblioteca Blynk é uma extensão que irá rodar em seu hardware, sendo responsável pela conectividade, autenticação de dispositivo na nuvem e processos de comandos entre o Blynk App, a nuvem e o hardware. Esta biblioteca encontra-se disponível para download no seguinte link:

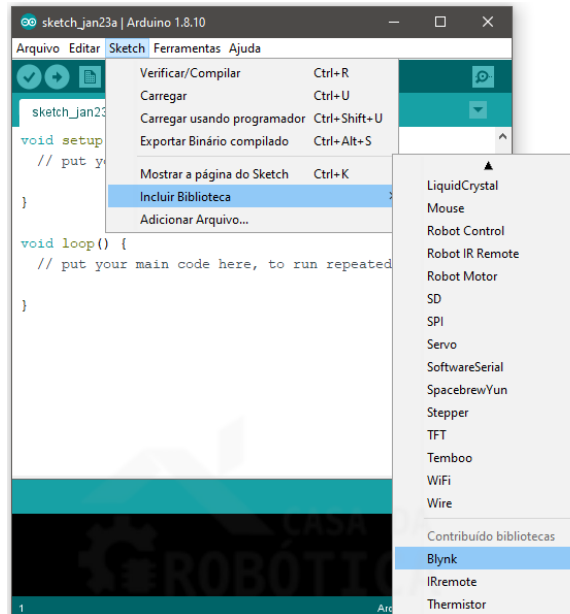
https://github.com/blynkkk/blynk-library/releases/download/v0.6.1/Blynk_Release_v0.6.1.zip

Após realizar o download da biblioteca, vamos instalá-la por meio do seguinte caminho: Toolbar > Sketch > Incluir biblioteca > Adicionar biblioteca ZIP, conforme

ilustra a Figura 59. Com a biblioteca instalada, feche o Arduino IDE e abra-o novamente.

Em seguida, vamos verificar se a biblioteca foi instalada corretamente por meio do seguinte caminho: Toolbar > Sketch > Incluir Biblioteca, onde buscaremos a biblioteca Blynk, conforme a Figura 70.

Figura 70 - Caminho para verificar se a biblioteca Blynk foi instalada.



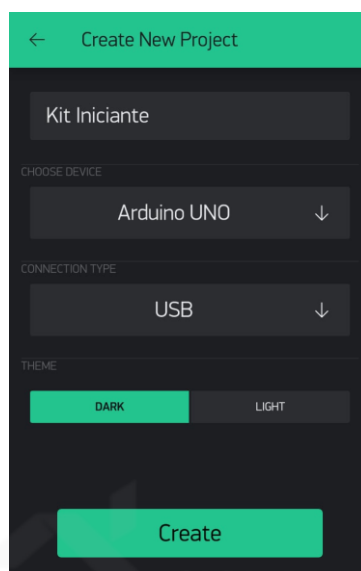
CRIANDO O PRIMEIRO PROJETO NO BLYNK

Agora que já conhecemos a plataforma Blynk, temos seu aplicativo instalado e sua biblioteca inclusa no Arduino IDE, vamos a criação de um projeto. A proposta desse projeto, intitulado projeto Kit Iniciante, é criar um aplicativo no Blynk que realize a interface entre o sistema de hardware construído em conjunto com a placa UNO capaz de:

- Controlar a cor do LED RGB;
- Controlar a intensidade do brilho de um LED difuso;
- Medir a temperatura e nível de luminosidade e exibir estas informações no aplicativo;
- Controlar o acionamento do buzzer.

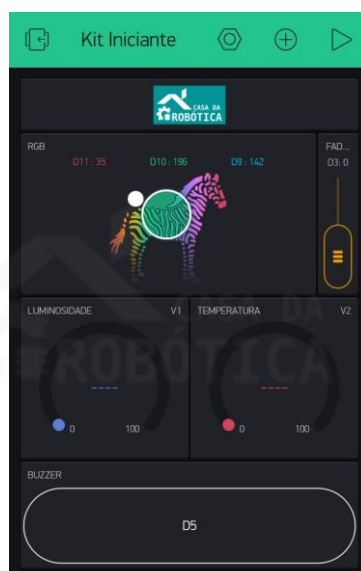
Iniciaremos nosso projeto com a criação do aplicativo no Blynk app. Para isso, abriremos a tela de criação do Blynk (Figura 65) e clicamos na opção New Project. Em seguida, na tela Create New Project configuraremos o projeto conforme a Figura 71.

Figura 71 - Configuração do aplicativo Kit Iniciante.



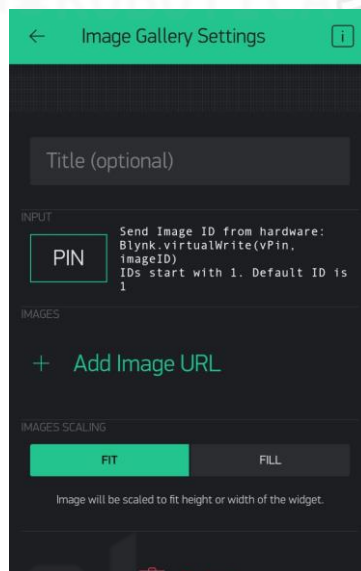
Em seguida, você receberá uma mensagem informando que o Auth Token do projeto foi enviado ao seu e-mail de cadastro. Clique em OK para fechar esta mensagem. Na tela de projeto (Figura 67) incluiremos os Widgets necessários para construção do aplicativo, conforme a Figura 72.

Figura 72 - Widgets do projeto Kit Iniciante.



Iniciaremos incluindo o Widget Image Gallery (Figura 73) que permite que você exiba qualquer imagem em seu projeto. Para isso, você precisa apenas informar a URL da imagem desejada no campo Add Image URL.

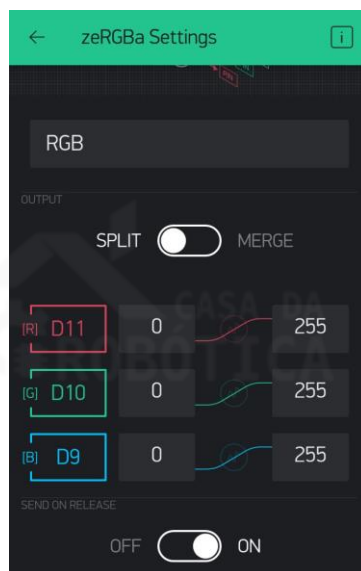
Figura 73 - Widget Image Gallery.



Em nosso exemplo utilizamos como imagem a logo da Casa da Robótica, que encontra-se disponível na URL: www.blogdarobotica.com/logocasadarobotica.

Em seguida, acrescentamos o Widget zeRGBa que é um seletor de cor e brilho do LED RGB. Configuraremos o zeRGBa no modo Split para enviar parâmetros diretamente ao pino da placa UNO.

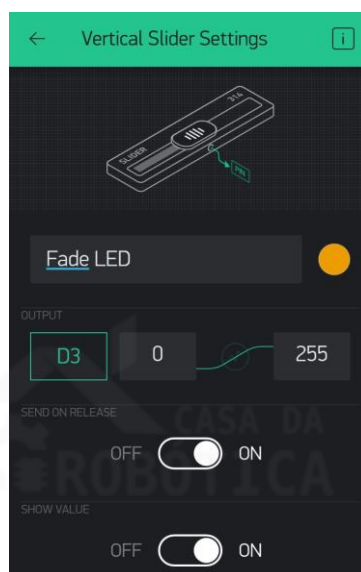
Figura 74 - Widget zeRGBa.



Os pinos também devem ser configurados, tendo como referência o circuito que montaremos posteriormente, atribuindo o pino digital 11 ao terminal vermelho, o pino 10 ao terminal verde e o pino 9 ao terminal azul do LED RGB, conforme ilustrada a Figura 74.

Logo após, incluiremos o Widget Vertical Slider para realizar o controle da intensidade de brilho do LED. Este Widget funciona de forma semelhante a um potenciômetro, permitindo o envio de valores em um determinado intervalo.

Figura 75 - Widget Vertical Slider.



A configuração do Vertical Slider deve ser feita conforme a Figura 75, incluindo como saída (OUTPUT) o pino digital 3 (D3) com uma faixa de valores de 0 a 255.

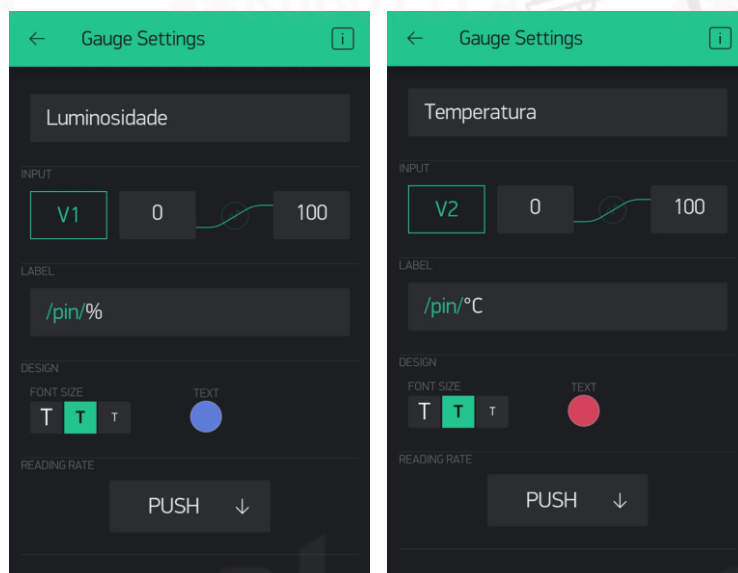
Em seguida, incluiremos dois widgets do tipo Gauge, um para exibir a luminosidade em porcentagem (%) e outro para mostrar a temperatura em graus Celsius (°C). O Gauge é uma ótima maneira de apresentar os valores numéricos recebidos de forma visual.

Configuraremos estes Gauges conforme a Figura 76. Neste caso, iremos utilizar dois pinos virtuais. Os pinos virtuais foram projetados para enviar quaisquer dados do microcontrolador para o aplicativo Blynk e vice-versa.

No Gauge Luminosidade configuraremos o pino virtual 1 (V1) como entrada (INPUT) para ler do LDR convertido em porcentagem. Desta forma, a faixa de valor necessário para exibir a luminosidade deve ser configurado de 0 a 100.

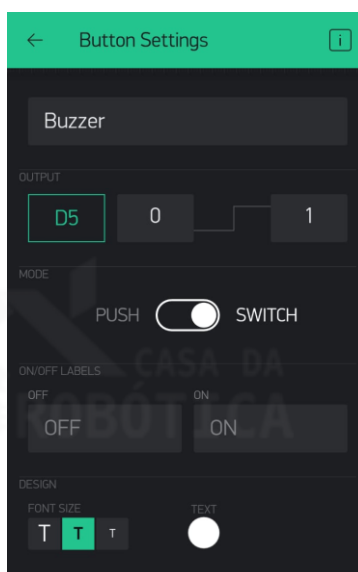
No Gauge Temperatura configuraremos o pino virtual 2 (V2) como entrada (INPUT) para ler o termistor NTC em graus Celsius. A faixa de valores também deve ser configurada de 0 a 100.

Figura 76 - Widgets Gauge para exibir luminosidade e temperatura.



O último ítem que vamos adicionar ao aplicativo é o Widget Button, que funciona como um botão (push button ou switch) permitindo o envio de valores ON e OFF (HIGH e LOW) para o microcontrolador. Utilizaremos esse Widget para acionar e desligar o buzzer. A Figura 77 ilustra como o Button deve ser configurado.

Figura 77 - Widget Button.



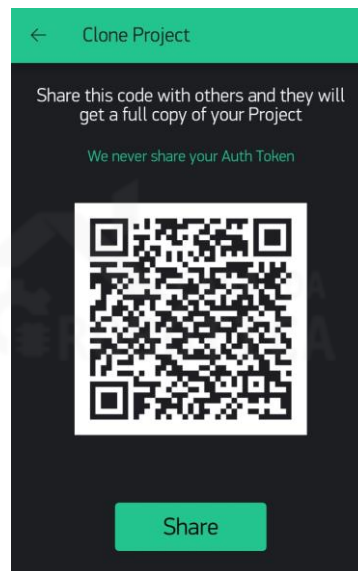
Utilizaremos o pino digital 5 (D5) como saída (OUTPUT), com faixa de 0 a 1 (desligado/ligado) e modo switch, ou seja, como um interruptor.



OBSERVAÇÕES:

- O aplicativo Blynk permite o compartilhamento de projetos via QR-code. Para fazer a cópia do projeto Kit Iniciante selecione a opção QR-code na tela de criação (item 3 da Figura 65) e utilize o código da Figura 78. Ao copiar o projeto lembre-se de alterar o Auth Token no código.
- É necessário ter 2000 de energia para copiar esse projeto.

Figura 78 - QR-code para cópia do projeto Kit Iniciante.



Com o aplicativo criado, vamos proceder a montagem do circuito elétrico do projeto Kit Iniciante. Para tal, utilizaremos os seguintes materiais:

- 1 x Placa UNO SMD R3 Atmega328 compatível com Arduino UNO;
- 1 x Cabo USB;
- 1 x LED RGB;
- 1 x Buzzer ativo;
- 1 x LED difuso;
- 1 x LDR;
- 1 x Termistor NTC;
- 2 x Resistor de 10 k Ω ;
- 4 x Resistores de 220 Ω ;

Monte o circuito da Figura 79. Verifique cuidadosamente os fios de ligação antes de ligar sua placa UNO, conforme detalha a Tabela 12. Lembre-se que a placa UNO deve estar desconectada enquanto você monta o circuito.

Figura 79 - Circuito para projeto Kit Iniciante.

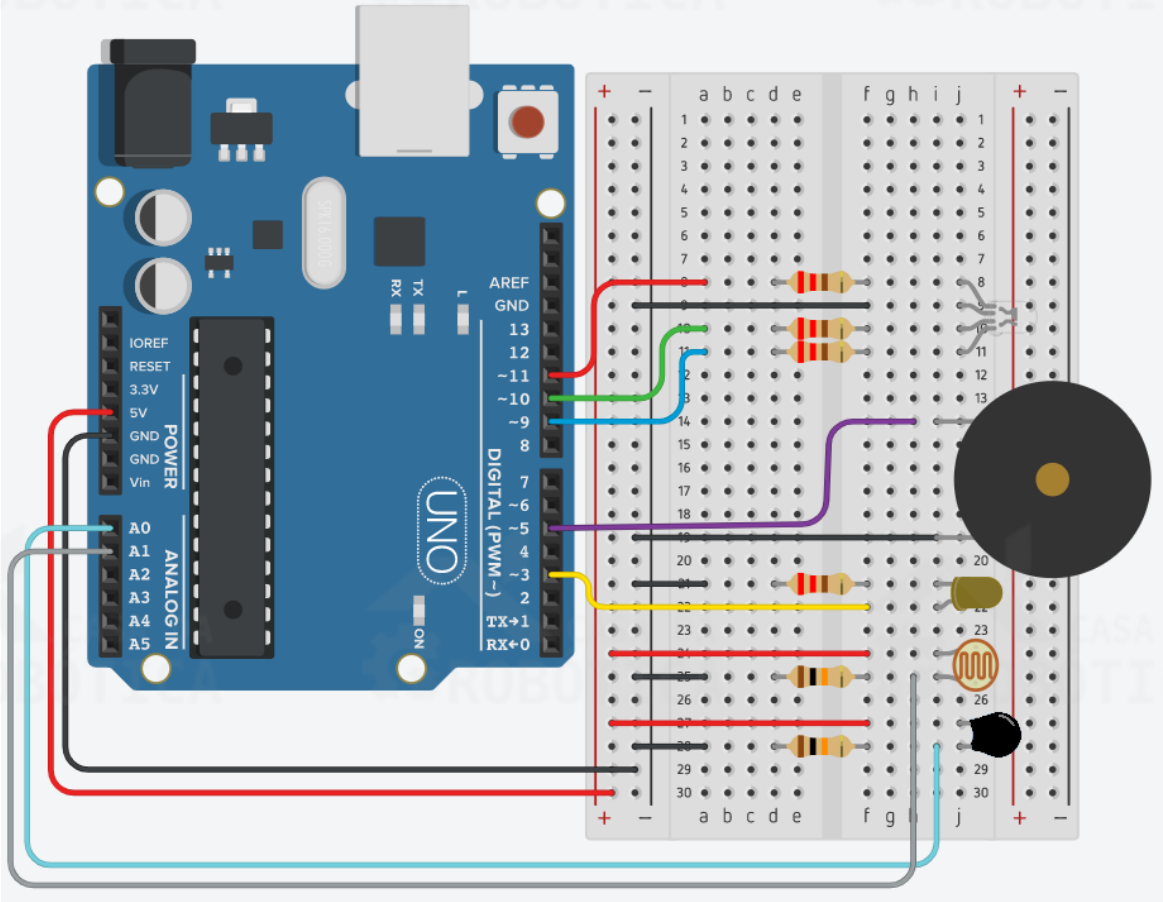


Tabela 12 - Conexões do projeto Kit Iniciante.

Componente	Porta da Placa UNO
LED RGB	R - D11 - Digital
	G - D10 - Digital
	B - D9 - Digital
BUZZER	D5 - Digital
LED DIFUSO	D3 - Digital
LDR	A1 - Analógico
TERMISTOR NTC	A0 - Analógico

Com o circuito montado, vamos a programação do nosso Sketch. Para isso, vamos precisar das bibliotecas Blynk e Thermistor que foram disponibilizadas para download anteriormente.

O programa do projeto Kit Iniciante deve conter a seguinte estrutura:

```
#include <BlynkSimpleStream.h> //Biblioteca do Blynk
#include <Thermistor.h> //Inclusão da biblioteca Thermistor

//Variáveis para termistor
Thermistor temp(0); //Atribui o pino A0, em que o termistor está conectado,
a variável temp
int valorntc = 0; //Variável que armazenará a temperatura lida

//Variáveis para LDR
int ldr = A1; //Atribui o pino A0 a variável ldr
int valorldr = 0; //Variável que armazenará o valor lido do ldr
int converter1 = 0; //Variável que armazenará o valor lido do ldr convertido
(de 0 a 1023 para 0 a 100%)

char auth[] = "oP3R_NE4DOxY9mRUyoillvTyuM8czesg"; //Código Auth Token

void setup()
{
  Serial.begin(9600); //Inicializa a comunicação serial
  Blynk.begin(Serial, auth); //Inicializa a comunicação serial do Blynk
  passando como parâmetro o Auth Token
}

void loop()
{
  Blynk.run(); //Chama a função Blynk.run
  luminosidade(); //Chama a função luminosidade
  temperatura(); //Chama a função temperatura
}

void luminosidade() {
  valorldr = analogRead(ldr); //Lê o valor do sensor ldr e armazena na
variável valorldr
  converter1 = map(valorldr, 0, 1023, 0, 100); //Converte a escala de 0 a
1023 para a escala de 0 a 100
  Blynk.virtualWrite(V1, converter1); //Escreve no pino virtual V1 o valor
de converter1
}

void temperatura() {
  valorntc = temp.getTemp(); //Variável do tipo inteiro que recebe o valor
da temperatura calculado pela biblioteca
  Blynk.virtualWrite(V2, valorntc); //Escreve no pino virtual V2 o valor de
valorntc
}
```

O código do projeto Kit Iniciante encontra-se detalhado nas seguintes etapas:

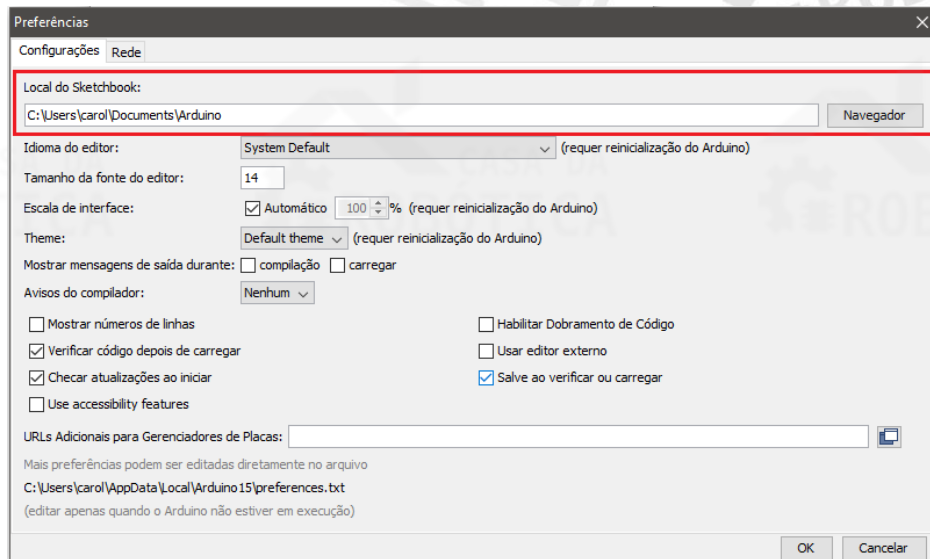
- 1- A primeira etapa consiste na inclusão das bibliotecas Blynk e Thermistor por meio das seguintes instruções:

```
#include <BlynkSimpleStream.h> //Biblioteca do Blynk
#include <Thermistor.h> //Inclusão da biblioteca Thermistor
```

- 2- Na segunda etapa as variáveis para uso e armazenamento dos dados recebidos pelo termistor e LDR devem ser declaradas.
- 3- Em seguida, devemos substituir o Auth Token. Você pode obter seu Auth Token por meio do e-mail recebido ou no aplicativo Blynk na área de configuração do projeto (Project Setting, ícone 2 da Figura 67).
- 4- No setup, faremos a inicialização da comunicação serial por meio da instrução `Serial.begin(9600);` e inicializaremos a comunicação serial do Blynk passando como parâmetro o Auth Token através da instrução `Blynk.begin(Serial, auth);`.
- 5- No loop, chamaremos todas as funções necessárias para o funcionamento do programa.
- 6- A função `Blynk.run()` é responsável por processar os comandos recebidos e executar as tarefas da conexão Blynk.
- 7- A função `luminosidade()` é responsável pela leitura do LDR e envio desse dado ao Blynk, através da instrução `Blynk.virtualWrite(V1, converter1);`.
- 8- De modo semelhante, a função `temperatura()` realizará a leitura do termistor NTC e enviar ao pino V2 do Blynk pela instrução `Blynk.virtualWrite(V2, valorntc);`.
- 9- Verifique o código e faça o upload para a placa UNO.

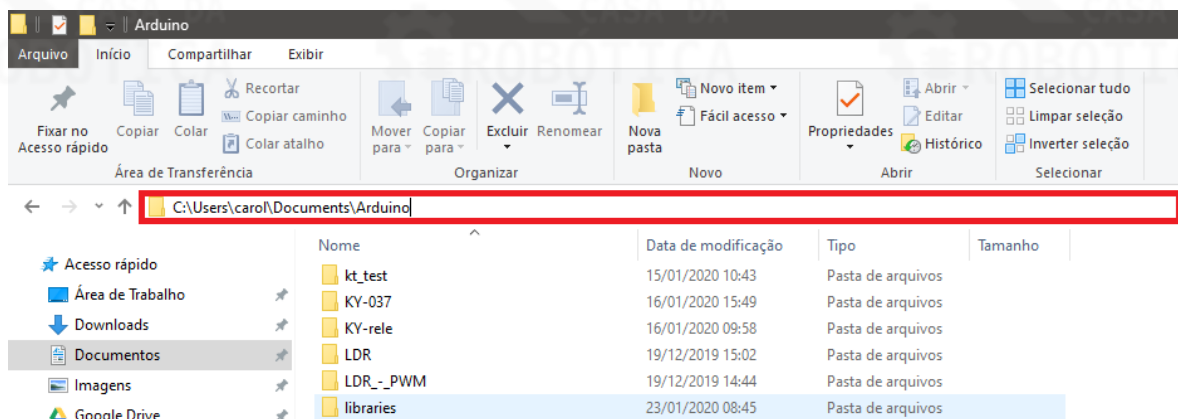
Para executar o programa em conjunto ao aplicativo criado no Blynk, no caso da conexão via USB, é necessário executar um arquivo que encontra-se disponível na pasta de arquivos do Arduino IDE. Você pode localizar o caminho desta pasta pelo seguinte caminho: Toolbar > Arquivo > Preferência - Local do Sketchbook, conforme mostra a Figura 80.

Figura 80 - Local de pasta de arquivos do Arduino.



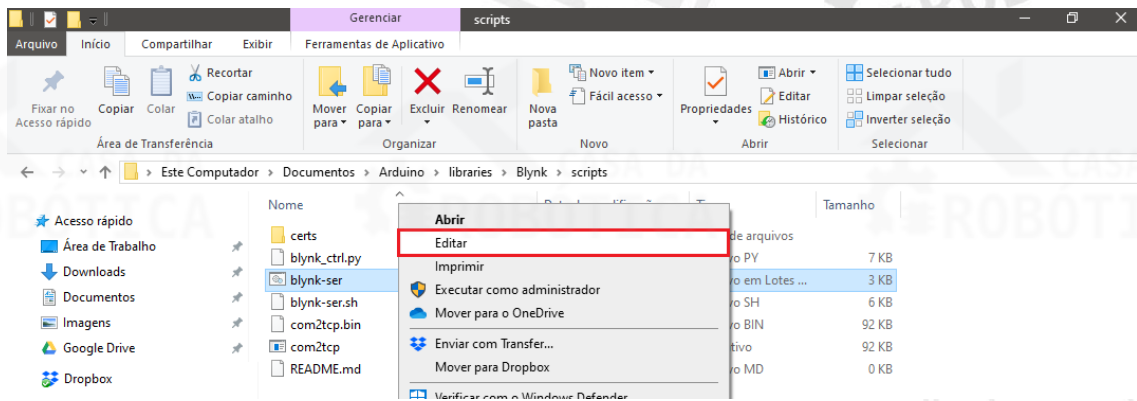
Você pode copiar esse caminho e colar no Explorar do Windows para abrir a pasta de Destino, conforme a Figura 81.

Figura 81 - Cole o caminho da pasta Arduino no Explorar do Windows.



Logo após abra as seguintes pastas: Libraries > Blynk > scripts e busque o arquivo blynk-ser. Ao encontrá-lo, clique nele com o botão direito e selecione a opção Editar, conforme Figura 82.

Figura 82 – Edite o arquivo blynk-ser.



Após clicar em Editar, o arquivo blynk-ser será aberto no Bloco de Notas. Verifique se a porta COM na qual sua placa UNO encontra-se conectada é a mesma configurada no código, caso não seja substitua-a e Salve o arquivo. A linha do código em que a COM é configurada encontra-se destacada em vermelho na Figura 83.

Figura 83 - Código do arquivo blynk-ser.

```
@echo off
setlocal EnableDelayedExpansion

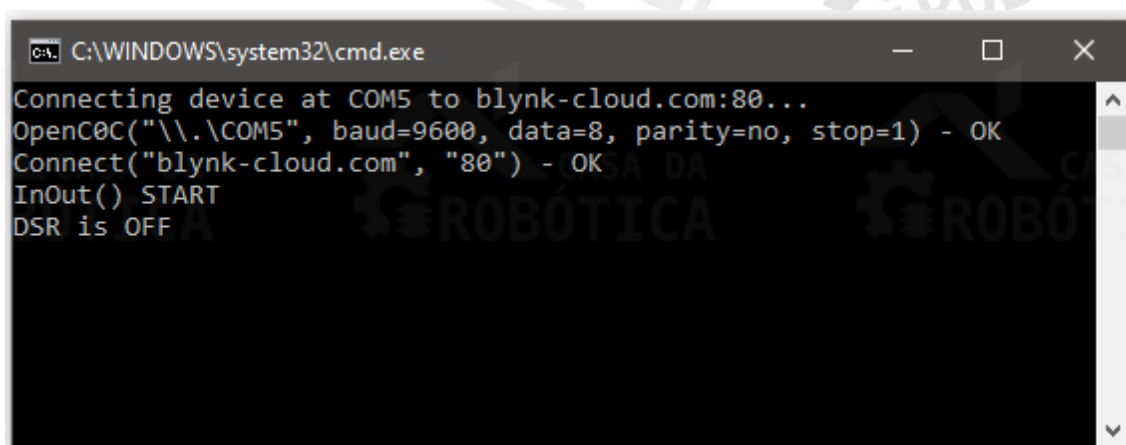
REM === Edit these lines to match your need ===
set COMM_PORT=COM1
set COMM_BAUD=9600
set SERV_ADDR=blynk-cloud.com
set SERV_PORT=80

REM === Edit lines below only if absolutely sure what you're doing ===

rem Get command line options
set SCRIPTS_PATH=%~dp0
```

Em seguida, salve as modificações feitas no código e execute o arquivo blynk-ser. A tela da Figura 84 será exibida.

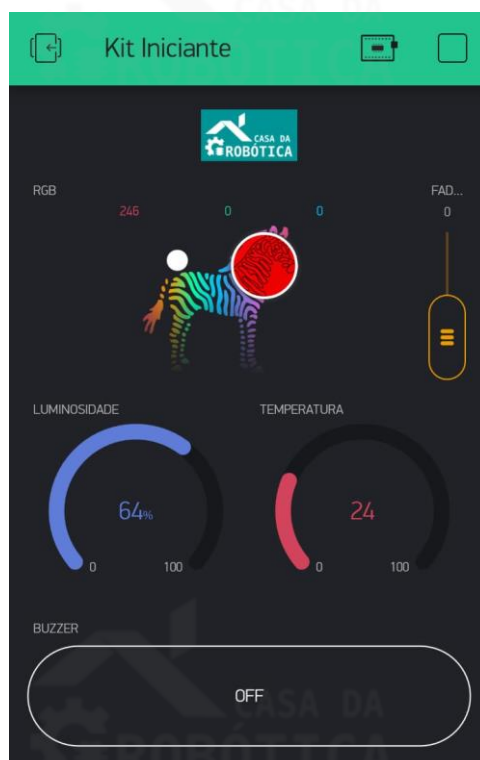
Figura 84 - Arquivo blynk-ser sendo executado.



```
C:\WINDOWS\system32\cmd.exe
Connecting device at COM5 to blynk-cloud.com:80...
OpenC0C("\\.\COM5", baud=9600, data=8, parity=no, stop=1) - OK
Connect("blynk-cloud.com", "80") - OK
InOut() START
DSR is OFF
```

Com o blynk-ser em execução, abra o aplicativo criado no Blynk App e selecione o ícone Play para iniciar a comunicação entre o aplicativo e a placa UNO. Se tudo estiver correto aparecerá o ícone da placa UNO, ao lado do ícone Stop, apresentando informações de que o dispositivo encontra-se online.

Figura 85 - Ícone da placa UNO e Stop no Blynk App.



7. CONSIDERAÇÕES FINAIS

Chegamos ao final do nosso material de apoio Kit Iniciante.

Esperamos que este material tenha auxiliado e contribuído no seu aprendizado de eletrônica e programação.

Caso tenha alguma dúvida entre em contato conosco, afinal “a dúvida é o princípio da sabedoria” (Aristóteles).

Caso você tenha encontrado algum problema no material ou possui alguma sugestão, por favor, entre em contato conosco. Sua opinião é muito importante para nós.

contato@casadarobotica.com

Acompanhe as novidades em nossas redes sociais:

Facebook: @casadaroboticaoficial

Instagram: @casadarobotica

Até a próxima,

Equipe Casa da Robótica

KITS CASA DA ROBÓTICA

KIT AUTOMAÇÃO

KIT Automatize seu Quarto:

Ideal para você aprender sobre o projeto Arduino e obter conhecimentos em Smart Home/Casas Inteligentes e Internet das Coisas - IoT.



KIT INICIANTE IOT

Kit Iniciante IoT - Estação Meteorológica:

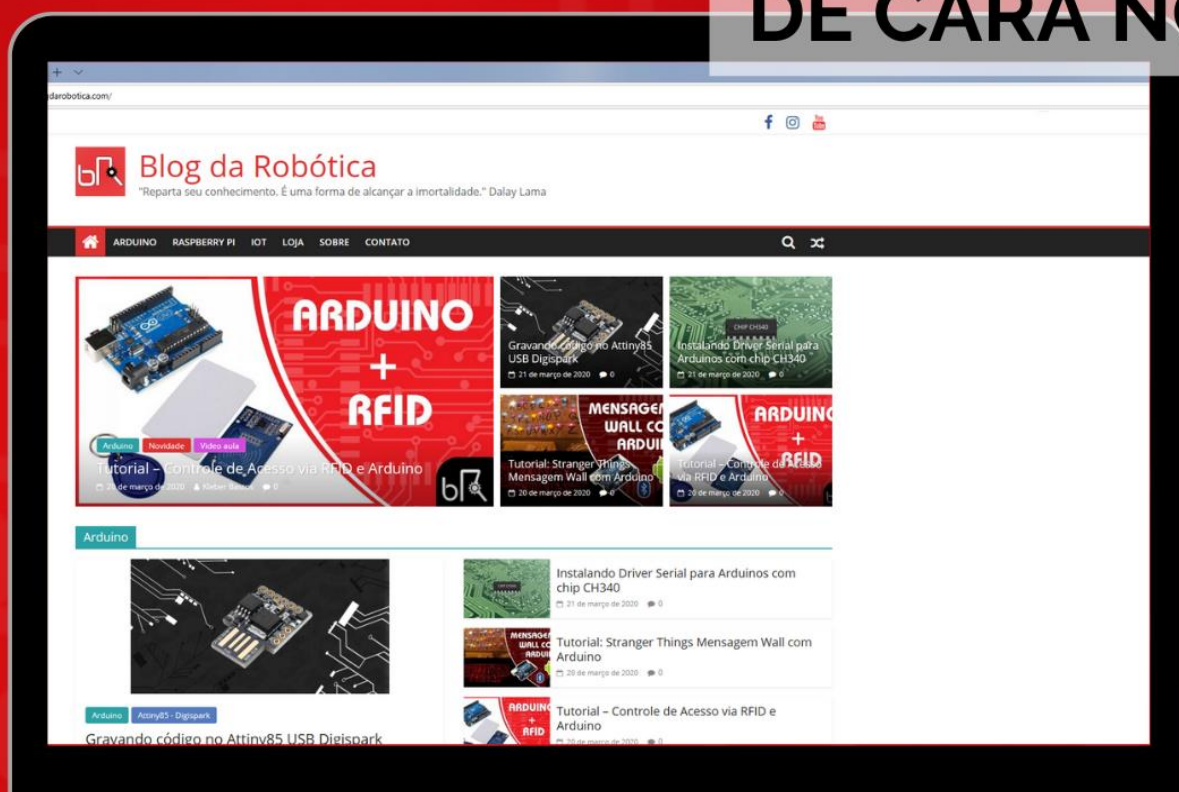
Ideal para você aprender sobre Internet das Coisas com a placa NODEMCU ESP8266. Além de obter conhecimentos em eletrônica e programação com a construção de uma estação meteorológica.



blog da ROBÓTICA

#ELETRÔNICA #INOVAÇÃO #TECNOLOGIA #PROGRAMAÇÃO
#ROBÓTICA #CIÊNCIAS #MAKER

DE CARA NOVA



blogda**robotica**.com





CASA DA ROBÓTICA



www.casadarobotica.com



contato@casadarobotica.com



(77) 99151-2820



[@casadaroboticaoficial](https://www.facebook.com/casadaroboticaoficial)



[@casadarobotica](https://www.instagram.com/casadarobotica)